常见面试题 100 问|20220808

一、为什么连接的时候是三次握手，关闭的时候却是四次握手？

答：因为当 Server 端收到 Client 端的 SYN(同步序列编号)连接请求报文后，可以直接发送 SYN+ACK 报文。其中 ACK 报文是用来应答的，SYN 报文是用来同步的。但是关闭连接时，当 Server 端收到 FIN 报文时，很可能并不会立即关闭 SOCKET，所以只能先回复一个 ACK 报文，告诉 Client 端，"你发的 FIN 报文我收到了"。只有等到我 Server 端所有的报文都发送完了，我才能发送 FIN 报文，因此不能一起发送。故需要四步握手。

传输层 TCP 传输报文

交流Q 群：953306497

位于传输层的 TCP 协议为传输报文提供可靠的字节流服务。它为了方便传输，将大块的数据分割成以报文段为单位的数据包进行管理，并为它们编号，方便服务器接收时能准确地还原报文信息。TCP 协议通过“三次握手”等方法保证传输的安全可靠。
客户端发送一个带有 SYN 标志的数据包给服务端，在一定的延迟时间内等待接收的回复。服务端收到后，回传一个带有 SYN/ACK 标志的数据包以示传达确认信息，最后客户端再回传一个带 ACK 标志的数据包，代表握手结束，连接成功。

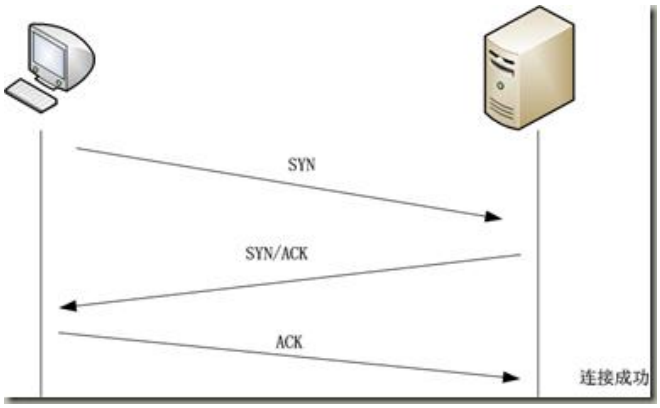
SYN （Synchronize Sequence Numbers）同步序列编号
ACK (Acknowledgement）确认字符

FIN（Finish）结束标志

交流Q 群：953306497

发送端发送一个 SYN=1，ACK=0 标志的数据包给接收端，请求进行连接，这是第一次握手；接收端收到请求并且允许连接的话，就会发送一个 SYN=1，ACK=1 标志的数据包给发送端，告诉它，可以通讯了，并且让发送端发送一个确认数据包，这是第二次握手；最后，发送端发送一个 SYN=0，ACK=1 的数据包给接收端，告诉它连接已被确认，这就是第三次握手。之后，一个 TCP 连接建立，开始通讯。

下图也可以这么理解：
客户端：“你好，在家不，有你快递。”---SYN
服务端：“在的，送来就行。”-----SYN/ACK
客户端：“好嘞。”-----ACK



关闭 TCP 连接

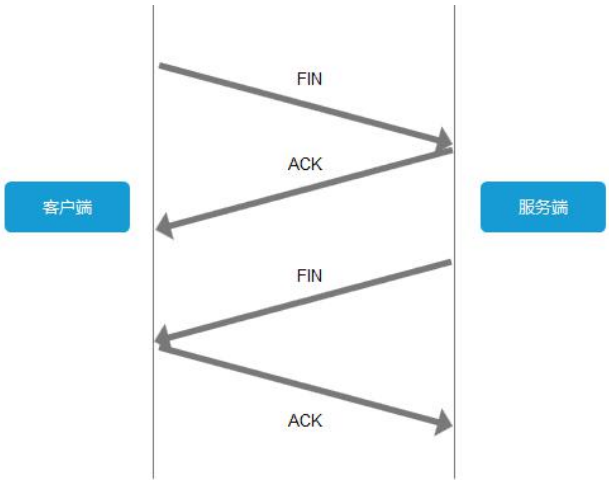
交流Q 群：953306497

为了避免服务器与客户端双方的资源占用和损耗，当双方没有请求或响应传递时，任意一方都可以发起关闭请求。与创建 TCP 连接的 3 次握手类似，关闭 TCP 连接，需要 4 次握手。

SYN （Synchronize Sequence Numbers）同步序列编号

ACK (Acknowledgement）确认字符

FIN（Finish）结束标志



4 次握手

上图可以这么理解：

客户端："兄弟，我这边没数据要传了，咱关闭连接吧。"----FIN

服务端："收到，我看看我这边有木有数据了。"----ACK

服务端："兄弟，我这边也没数据要传你了，咱可以关闭连接了。"----FIN

客户端："好嘞。"----ACK

二、TCP 连接为 什么 C lient 会在发送出 ACK 之后进入到 TIME_WAIT 状态需要经过 2MSL(最大报文段生存时间)才能返回到 CLOSE 状态？

答：虽然按道理，四个报文都发送完毕，我们可以直接进入 CLOSE 状态了，但是我们必须假象网络是不可靠的，有可能最后一个 ACK 丢失。所以 TIME_WAIT 状态就是用来重发可能丢失的 ACK 报文。在 Client 发送出最后的 ACK 回复，但该 ACK 可能丢失。Server 如果没有收到 ACK，将不断重复发送 FIN 片段。所以 Client 不能立即关闭，它必须确认 Server 接收到了该 ACK。Client 会在发送出 ACK 之后进入到 TIME_WAIT 状态。Client 会设置一个计时器，等待 2MSL 的时间。如果在该时间内再次收到 FIN，那么 Client 会重发 ACK 并再次等待 2MSL。所谓的 2MSL 是两倍的 MSL(Maximum Segment Lifetime)。MSL 指一个片段在网络中最大的存活时间，2MSL 就是一个发送和一个回复所需的最大时间。如果直到 2MSL，Client 都没有再次收到 FIN，那么 Client 推断 ACK 已经被成功接收，则结束 TCP 连接。

三、为什么不能用两次握手进行连接？

交流Q 群：953306497

答：3 次握手完成两个重要的功能，既要双方做好发送数据的准备工作(双方都知道彼此已准备好)，也要允许双方就初始序列号进行协商，这个序列号在握手过程中被发送和确认。

现在把三次握手改成仅需要两次握手，死锁是可能发生的。作为例子，考虑计算机 S 和 C 之间的通信，假定 C 给 S 发送一个连接请求分组，S 收到了这个分组，并发 送了确认应答分组。按照两次握手的协定，S 认为连接已经成功地建立了，可以开始发送数据分组。可是，C 在 S 的应答分组在传输中被丢失的情况下，将不知道 S 是否已准备好，不知道 S 建立什么样的序列号，C 甚至怀疑 S 是否收到自己的连接请求分组。在这种情况下，C 认为连接还未建立成功，将忽略 S 发来的任何数据分组，只等待连接确认应答分组。而 S 在发出的分组超时后，重复发送同样的分组。这样就形成了死锁。

四、如果已经建立了连接，但是客户端突然出现故障了怎么办，Server 端怎么处理？

TCP 心跳包的发送，通常有两种技术：

交流Q 群：953306497

1.应用层自己实现的心跳包
由应用程序自己发送心跳包来检测连接是否正常，服务器每隔一定时间向客户端发送一个短小的数据包，然后启动一个线程，在线程中不断检测客户端的回应， 如果在一定时间内没有收到客户端的回应，即认为客户端已经掉线；同样，如果客户端在一定时间内没有收到服务器的心跳包，则认为连接不可用。
2.使用 SO_KEEPALIVE 套接字选项
在 TCP 的机制里面，本身是存在有心跳包的机制的，也就是 TCP 的选项. 不论是服务端还是客户端，一方开启 KeepAlive 功能后，就会自动在规定时间内向对方发送心跳包， 而另一方在收到心跳包后就会自动回复，以告诉对方我仍然在线。因为开启 KeepAlive 功能需要消耗额外的宽带和流量，所以 TCP 协议层默认并不开启默认的 KeepAlive 超时需要 7,200，000

MilliSeconds， 即 2 小时，探测次数为 5 次。对于很多服务端应用程序来说，2 小时的空闲时间太长。因此，我们需要手工开启 KeepAlive 功能并设置合理的 KeepAlive 参数

五、HTTPS 和 HTTP 的区别：

交流 Q 群：953306497

https 协议需要到 ca 申请证书，一般免费证书很少，需要交费。

http 是超文本传输协议，信息是明文传输。

https 是一种通过计算机网络进行安全通信的传输协议，经由 http 进行通信，利用 ssl/tls 建立全信道，加密数据包。tls 是传输层加密协议，前身是 ssl 协议，由网景公司 1995 年发布，有时候两者不区分。

http 和 https 使用的是完全不同的连接方式用的端口也不一样，http 是 80，https 是 443。

http 是一个基于请求与响应，无状态的，应用层的协议，常基于 tcp/ip 协议传输数据

https 协议是由 ssl+http 协议构建的可进行加密传输、身份认证的网络协议，要比 http 协议安全。

HTTPS 和 HTTP 的区别主要如下：

1、https 协议需要到 ca 申请证书，一般免费证书较少，因而需要一定费用。

2、http 是超文本传输协议，信息是明文传输，https 则是具有安全性的 ssl 加密传输协议。

3、http 和 https 使用的是完全不同的连接方式，http 基于 tcp/ip 协议构建的，https 协议是由 ssl+httpP 协议构建的，用的端口也不一样，前者是 80，后者是 443。

4、http 的连接很简单，是无状态的；HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，比 http 协议安全。

http协议，超文本传输协议，基于tcp/ip协议进行传输，现在使用的是HTTP 1.1
主要特点：
1、简单快速：客户向服务器请求服务时，只需传送请求方法(method)和路径(url)。
请求方法常用的有GET,POST,PUT,DELETE。每种方法规定了客户与服务器联系的类型不同。
由于HTTP协议简单，使得HTTP服务器的程序规模小，因而通信速度很快。
2、灵活：HTTP允许传输任意类型的数据对象。传输类型由Content-Type加以标记。
3、支持B/S及C/S模式
4、无连接：无连接的含义是限制每次连接只处理一个请求。
服务器处理完客户的请求，并收到客户的应答后，即断开连接。
采用这种方式可以节省传输时间。(HTTP1.1开始支持长连接，需要服务端配置相应的参数)
5、无状态：HTTP协议是无状态协议。无状态是指协议对于事务处理没有记忆能力。缺少状态意味着如果后续处理需要前面的信息，则它必须重传，这样可能导致每次连接传送的数据量增大。
另一方面，在服务器不需要先前信息时它的应答就较快。

六、HTTPS 的优点

交流 Q 群：953306497

尽管 HTTPS 并非绝对安全，掌握根证书的机构、掌握加密算法的组织同样可以进行中间人形式的攻击，但 HTTPS 仍是现行架构下最安全的解决方案，主要有以下几个好处：

（1）使用 HTTPS 协议可认证用户和服务器，确保数据发送到正确的客户机和服务器；

（2）HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，要比 http 协议安全，可防止数据在传输过程中不被窃取、改变，确保数据的完整性。

（3）HTTPS 是现行架构下最安全的解决方案，虽然不是绝对安全，但它大幅增加了中间人攻击的成本。

（4）谷歌曾在 2014 年 8 月份调整搜索引擎算法，并称"比起同等 HTTP 网站，采用 HTTPS 加密的网站在搜索结果中的排名将会更高"。

七、HTTPS 的缺点

交流Q 群：953306497

虽然说 HTTPS 有很大的优势，但其相对来说，还是存在不足之处的：

（1）HTTPS 协议握手阶段比较费时，会使页面的加载时间延长近 50%，增加 10%到 20%的耗电；

（2）HTTPS 连接缓存不如 HTTP 高效，会增加数据开销和功耗，甚至已有的安全措施也会因此而受到影响；

（3）SSL 证书需要钱，功能越强大的证书费用越高，个人网站、小网站没有必要一般不会用。

（4）SSL 证书通常需要绑定 IP，不能在同一 IP 上绑定多个域名，IPv4 资源不可能支撑这个消耗。

（5）HTTPS 协议的加密范围也比较有限，在黑客攻击、拒绝服务攻击、服务器劫持等方面几乎起不到什么作用。最关键的，SSL 证书的信用链体系并不安全，特别是在某些国家可以控制 CA 根证书的情况下，中间人攻击一样可行。

八、http 切换到 HTTPS 如何实现

如果需要将网站从 http 切换到 https 到底该如何实现呢？

这里需要将页面中所有的链接，例如 js，css，图片等等链接都由 http 改为 https。例如：http://www.baidu.com 改为 https://www.baidu.com

BTW，这里虽然将 http 切换为了 https，还是建议保留 http。所以我们在切换的时候可以做 http 和 https 的兼容，具体实现方式是，去掉页面链接中的 http 头部，这样可以自动匹配 http 头和 https 头。例如：将 http://www.baidu.com 改为//www.baidu.com。然后当用户从 http 的入口进入访问页面时，页面就是 http，如果用户是从 https 的入口进入访问页面，页面即使 https 的。

九、网站从 http 过度到 https 需要注意的几个小问题

一、nginx 配置同时访问 http 和 https

交流Q 群：953306497

监听 80 端口和 443ssl 端口。 重定向是 rewrite 关键字，直接改变浏览器地址栏的值；

转发是 proxy_pass 关键字，表示转发，浏览器地址栏不变。

二、https 协议下，不允许请求 http 协议的资源　　交流Q 群：953306497

https 协议的网站，不支持请求 http 协议的资源（如 js，css，视频等），所以，从 http 过度到 https 的时候，需要注意协议问题。有一个巧妙的方法是，请求资源的时候可以采用相对协议。比如，本来是 https://www.xxx.com/js/1.js ，此时可以改为相对协议 //www.xxx.com/js/1.js 。这样一来，无论用户访问的是哪一种协议的网站，都会成功。因为，当遇到相对协议的时候，浏览器会补上协议头（根据访问当前网站的协议）。当然，前提是，1.js 所在的域名要同时支持 http 和 https 协议，不然就瞎搞了。

三、https 协议下播放 http 协议的视频　　交流Q 群：953306497

如果引入的视频只有 http 能访问（如优酷），那在 https 环境下，则无法播放。解决办法：使用 iframe。利用 iframe 标签，可以在里面放置一个 http 页面（播放 http 视频），直接引入就好。（这个暂时没有实践过，目前只有理论，以后找个时间试试）

十、POST 和 GET 的区别

GET 提交的数据放在 URL 中，POST 则不会。这是最显而易见的差别。这点意味着 GET 更不安全（POST 也不安全，因为 HTTP 是明文传输抓包就能获取数据内容，要想安全还得加密）
GET 回退浏览器无害，POST 会再次提交请求（GET 方法回退后浏览器再缓存中拿结果，POST 每次都会创建新资源）
GET 提交的数据大小有限制（是因为浏览器对 URL 的长度有限制，GET 本身没有限制），POST 没有
GET 可以被保存为书签，POST 不可以。这一点也能感受到。
GET 能被缓存，POST 不能
GET 只允许 ASCII 字符，POST 没有限制
GET 会保存再浏览器历史记录中，POST 不会。这点也能感受到。
GET 在浏览器回退时是无害的，而 POST 会再次提交请求。
GET 产生的 URL 地址可以被 Bookmark，而 POST 不可以。
GET 请求会被浏览器主动 cache（缓存），而 POST 不会，除非手动设置。
GET 请求只能进行 ASCII 编码，而 POST 支持多种编码方式。
URL 编码是一种浏览器用来打包请求参数及表单参数的格式，参数和参数之间使用&分割，非 ASCII 码使用%加 16 进制编码替换
如：https://www.sojson.com/open/api/weather/json.shtml?city=北京
编码后为：
https://www.sojson.com/open/api/weather/json.shtml?city=%E5%8C%97%E4%BA%AC
GET 请求参数会被完整保留在浏览器历史记录里，而 POST 中的参数不会被保留。
GET 请求在 URL 中传送的参数是有长度限制的，而 POST 么有。
对参数的数据类型，GET 只接受 ASCII 字符，而 POST 没有限制。
GET 比 POST 更不安全，因为参数直接暴露在 URL 上，所以不能用来传递敏感信息。
GET 参数通过 URL 传递，POST 放在 Request body 中。
GET 和 POST 都是 TCP 链接。GET 产生一个 TCP 数据包；而 POST 会产生两个 TCP 数据包。

原因：Get，浏览器会把 http header 和 data 一并发送，服务器返回 200
Post，浏览器先发送 header，服务器响应 100，在发送 data，服务器响应 200

十一、HTTP 常见状态码介绍

交流Q 群：953306497

200：请求成功
301：请求重定向到另外一个接口，资源（网页等）被永久转移到其它 URL
302：请求重定向到另外一个接口，资源（网页等）临时转移到其它 URL
304：请求被重定向到客户端本地缓存
400：客户端请求语法错误（post 请求用 get 方法）
401：客户端请求没有经过授权
403：客户端请求资源没有访问权限，拒绝访问
404：客户端请求的资源（网页等）不存在（有可能是请求 url 错误或参数不正确）
405：请求方法不被允许（比如接口只允许 Post,使用 Get 请求接口）
415：content-type 不正确
500：服务器内部错误（通常是服务器挂了或接口 Bug)
502：网关失效
503：服务端发生临时错误
504：网关请求超时

分类	分类描述
1**	信息，服务器收到请求，需要请求者继续执行操作
2**	成功，操作被成功接收并处理
3**	重定向，需要进一步的操作以完成请求
4**	客户端错误，请求包含语法错误或无法完成请求
5**	服务器错误，服务器在处理请求的过程中发生了错误

十二、什么是泛型？

泛型将接口的概念进一步延伸，“泛型”的字面意思就是广泛的类型。类、接口和方法代码可以应用于非常广泛的类型，代码与它们能够操作的数据类型不再绑定在一起，同一套代码可以用于多种数据类型，这样不仅可以复用代码，降低耦合性，而且还提高了代码的可读性以及安全性。

十三、IO 密集型与 CPU 密集型操作

交流Q 群：953306497

IO 密集型，主要涉及到网络、硬盘 IO 的任务，这类任务消耗 cpu 操作很少，任务大部分时

间都在等待 IO 操作的完成，对于 IO 密集型任务，任务越多，CPU 效率越高，但也有一个限度。常见的大部分任务都是 IO 密集型任务，比如 Web 应用。对于 IO 密集型操作，对于语言的运行效率不高，所以一般追求开发效率，使用 python 等脚本语言，

cpu 密集型操作，消耗 CPU 资源，比如计算圆周率、对视频进行高清解码等等，全靠 CPU 的运算能力。这种计算密集型任务虽然也可以用多任务完成，但是任务越多，花在任务切换的时间就越多，CPU 执行任务的效率就越低，所以，要最高效地利用 CPU，计算密集型任务同时进行的数量应当等于 CPU 的核心数。对于 CPU 密集型操作，一般使用 C 等效率高的语言。

十四、什么是多态？为什么使用多态？

父类类型的变量指向子类创建的对象，使用该变量调用父类中一个被子类重写的方法，则父类中的方法呈现出不同的行为特征，这就是多态。
简单的来说编译时方法行为表现的是父类中的方法，运行时方法行为表现的是子类中重写该方法的行为特征

二、为什么使用多态？

1、实现代码的复用，避免代码的冗余；

2、减少代码之间的关联性，即耦合度，方便后期对代码的修改，功能的改善，不必牵一发而动全身，减少不必要的麻烦；

3、能够通过重写子类的方法，使不同的对像具有不同的功能，扩展了功能。

重载、接口、继承三种方法实现多态

十五、动态内存回收对程序员有什么好处

所谓动态内存分配，就是指在程序执行的过程中动态地分配或者回收存储空间的分配内存的方法。动态内存分配不象数组等静态内存分配方法那样需要预先分配存储回空间，而是由系统根据程序的需要即时分配，且分配的大小就是程序要求的大小。动态内存的答好处是归纳为 8 个字：用时分配，不用释放！

十六、浏览器、安卓、ios 系统之间的区别

浏览器：现在有四大渲染引擎在线：WebKit、Blink、Trident 和 Gecko
1、使用 Trident(三叉戟)内核的浏览器：IE、Maxthon、TT、The World 等；IE 从版本 11 开始，初步支持 WebGL 技术。IE8 的 JavaScript 引擎是 Jscript，IE9 开始用 Chakra，这两个版本区别很大，Chakra 无论是速度和标准化方面都很出色
2、使用 Gecko(壁虎)内核的浏览器：Netcape6 及以上版本、火狐 FireFox、MozillaSuite/SeaMonkey；
3、使用 Presto 内核的浏览器：欧朋 Opera7 及以上版本；后期也使用谷歌的 Blink
4、使用 Webkit(极速)内核的浏览器：Safari、Chrome。后期基于 WebKit 的 fork Web 渲染引

擎：Blink

android：https://www.jianshu.com/p/6d7becff8e04
Android 1.0
智能虚拟键盘、视频录制分享、蓝牙和免提电话等
Android 2.0
网络社交，比如 Facebook、Google 地图导航、前置摄像头
Android3.0
主要用于安卓的平板产品，画面动感，可操控性更强
Android4.0
统一了手机和平板电脑使用的系统，支持在系统中使用虚拟按键，可取代物理键、脸部识别、1080p 视频播放录制
Android5.0
采用全新 Material Design 界面 、由 Dalvik 转用 ART(Android Runtime)编译
Android6.0
应用权限管理 、指纹识别，SD 卡可能和内置存储“合并”
Android7.0
分屏多任务 、同一应用的多条通知提示消息归拢为一项、流量保护模式
Android8.0
长按一个软件后可以弹出子菜单的模式、在权限设置中添加安装限制功能、画中画功能
Android9.0
更像是优化、所有应用都使用 HTTPS、新的 Home 按钮（形状是水平对齐的药丸）、新截图快捷方式（点击 Power 菜单中的图标）
Android10
Foldables（折叠屏）、增加了对 5G 网络的支持、保护隐私权限更严格

Android 11
首先对顶部通知栏和 Android 10 上就已出现的气泡进行了优化，让操作变得更加方便快捷。
键盘可自动填充应用程序特定的内容，形式在类似于你选择某个特定功能框，而后在键盘上出现待选项。
加入了全新的媒体控制中心，将在通知栏中点击开启，并出现在手机下方。通过这个媒体控制中心，我们可以在这里更好的连接、控制其它的无线设备。
允许用户向请求访问麦克风、摄像头、地理位置的应用给予一次性使用权限，即推出这个应用后权限会重新被锁定，应用无法继续使用这些权限，需要在访问时需要再次请求相关权限。

Android 12
停止对 32 位的支持，也就是说安卓系统也将开始切换到 64 位操作系统了

ios：https://www.jianshu.com/p/06b9a163a74c， https://www.jianshu.com/p/082e987a4efb
https://blog.csdn.net/u013896628/article/details/79800734
iOS 5
icloud 通知中心 ARC Stroyboard airplay
iOS 6
自动布局 IAP

iOS 7

全新扁化风格　力学模型　　交流Q 群：**953306497**

iOS8
Health Kit (健康工具包)和　Home Kit(家庭工具包)
iOS9
3Dtouch　swift2 编程语言　统一用 https　手机电池的低功耗设置
iOS 10
SiriKit 所有第三方应用都可以用 Siri　　Widget Enhancements 锁屏设计小功能
IOS11
UIKit Bars 标题视图的改版　iOS11 导航栏　无线部署支持无线充电　切换到 64 位应用
安全区域：iPhone X 由于状态栏高度的变化，对于状态栏高度预设值的视图需要，动态获取，由于新增底部功能区域，需要对视图高度进行适配
IOS12
睡前免打扰模式　屏幕时间　共享精确位置　Siri 为日常任务添加快捷指令　测距仪
IOS13
性能提升　深色模式　苹果地图　隐私改进　外部存储　照片提示年月日并可编辑

十七、什么是幂等性

HTTP/1.1 中对幂等性的定义是：一次和多次请求某一个资源对于资源本身应该具有同样的结果（网络超时等问题除外）。也就是说，其任意多次执行对资源本身所产生的影响均与一次执行的影响相同。

这里需要关注几个重点：

幂等不仅仅只是一次（或多次）请求对资源没有副作用（比如查询数据库操作，没有增删改，因此没有对数据库有任何影响）。

幂等还包括第一次请求的时候对资源产生了副作用，但是以后的多次请求都不会再对资源产生副作用。

幂等关注的是以后的多次请求是否对资源产生的副作用，而不关注结果。

网络超时等问题，不是幂等的讨论范围。

什么情况下需要幂等

业务开发中，经常会遇到重复提交的情况，无论是由于网络问题无法收到请求结果而重新发起请求，或是前端的操作抖动而造成重复提交情况。 在交易系统，支付系统这种重复提交造成的问题有尤其明显，比如：

用户在 APP 上连续点击了多次提交订单，后台应该只产生一个订单；

向支付宝发起支付请求，由于网络问题或系统 BUG 重发，支付宝应该只扣一次钱。 很

显然，声明幂等的服务认为，外部调用者会存在多次调用的情况，为了防止外部多次调用对系统数据状态的发生多次改变，将服务设计成幂等。

什么情况下需要保证幂等性

以 SQL 为例，有下面三种场景，只有第三种场景需要开发人员使用其他策略保证幂等性：

SELECT col1 FROM tab1 WHER col2=2，无论执行多少次都不会改变状态，是天然的幂等。

UPDATE tab1 SET col1=1 WHERE col2=2，无论执行成功多少次状态都是一致的，因此也是幂等操作。

UPDATE tab1 SET col1=col1+1 WHERE col2=2，每次执行的结果都会发生变化，这种不是幂等的。

为什么要设计幂等性的服务

幂等可以使得客户端逻辑处理变得简单，但是却以服务逻辑变得复杂为代价。满足幂等服务的需要在逻辑中至少包含两点：

首先去查询上一次的执行状态，如果没有则认为是第一次请求

在服务改变状态的业务逻辑前，保证防重复提交的逻辑

幂等的不足

幂等是为了简化客户端逻辑处理，却增加了服务提供者的逻辑和成本，是否有必要，需要根据具体场景具体分析，因此除了业务上的特殊要求外，尽量不提供幂等的接口。

增加了额外控制幂等的业务逻辑，复杂化了业务功能；

把并行执行的功能改为串行执行，降低了执行效率。

防重复提交策略

上述的保证幂等方案是分成两步的，第②步依赖第①步的查询结果，无法保证原子性的。在高并发下就会出现下面的情况：第二次请求在第一次请求第②步订单状态还没有修改为‘已支付状态’的情况下到来。既然得出了这个结论，余下的问题也就变得简单：把查询和变更状态操作加锁，将并行操作改为串行操作。

乐观锁

如果只是更新已有的数据，没有必要对业务进行加锁，设计表结构时使用乐观锁，一般通过 version 来做乐观锁，这样既能保证执行效率，又能保证幂等。例如： UPDATE tab1 SET col1=1,version=version+1 WHERE version=#version# 不过，乐观锁存在失效的情况，就是常说的 ABA 问题，不过如果 version 版本一直是自增的就不会出现 ABA 的情况。（从网上找了一张图片很能说明乐观锁，引用过来，出自 Mybatis 对乐观锁的支持）

防重表

使用订单号 orderNo 做为去重表的唯一索引，每次请求都根据订单号向去重表中插入一条数据。第一次请求查询订单支付状态，当然订单没有支付，进行支付操作，无论成功与否，

执行完后更新订单状态为成功或失败，删除去重表中的数据。后续的订单因为表中唯一索引而插入失败，则返回操作失败，直到第一次的请求完成（成功或失败）。可以看出防重表作用是加锁的功能。

分布式锁

这里使用的防重表可以使用分布式锁代替，比如 Redis。订单发起支付请求，支付系统会去 Redis 缓存中查询是否存在该订单号的 Key，如果不存在，则向 Redis 增加 Key 为订单号。查询订单支付已经支付，如果没有则进行支付，支付完成后删除该订单号的 Key。通过 Redis 做到了分布式锁，只有这次订单订单支付请求完成，下次请求才能进来。相比去重表，将放并发做到了缓存中，较为高效。思路相同，同一时间只能完成一次支付请求。

token 令牌

这种方式分成两个阶段：申请 token 阶段和支付阶段。 第一阶段，在进入到提交订单页面之前，需要订单系统根据用户信息向支付系统发起一次申请 token 的请求，支付系统将 token 保存到 Redis 缓存中，为第二阶段支付使用。 第二阶段，订单系统拿着申请到的 token 发起支付请求，支付系统会检查 Redis 中是否存在该 token，如果存在，表示第一次发起支付请求，删除缓存中 token 后开始支付逻辑处理；如果缓存中不存在，表示非法请求。 实际上这里的 token 是一个信物，支付系统根据 token 确认，你是你妈的孩子。不足是需要系统间交互两次，流程较上述方法复杂。

支付缓冲区

把订单的支付请求都快速地接下来，一个快速接单的缓冲管道。后续使用异步任务处理管道中的数据，过滤掉重复的待支付订单。优点是同步转异步，高吞吐。不足是不能及时地返回支付结果，需要后续监听支付结果的异步返回。

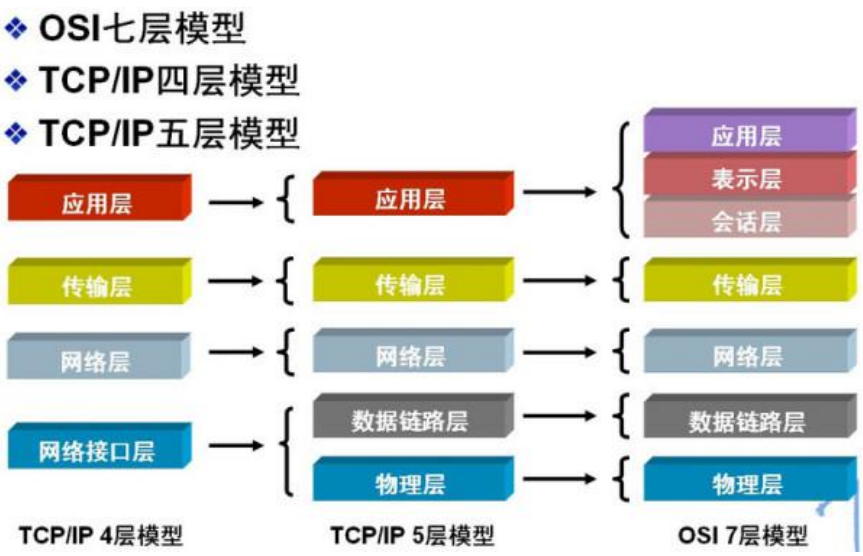
十八、在 TCP/IP 协议有四层。

1、应用层：应用层是 TCP/IP 协议的第一层，是直接为应用进程提供服务的。

2、运输层：作为 TCP/IP 协议的第二层，运输层在整个 TCP/IP 协议中起到了中流砥柱的作用。且在运输层中，TCP 和 UDP 也同样起到了中流砥柱的作用。

3、网络层：网络层在 TCP/IP 协议中的位于版第三层。在 TCP/IP 协议中网络层可以进行网络连接的建立和终止以及 IP 地址的寻找等功能。

4、网络接口层：在 TCP/IP 协议中，网络接口层位于第四层。由于网络接口层兼并了物理层和数据链路层所以，网络接口层既是传输数据的物理媒介，也可以为网络层提供一条准确无误的线路。



TCP／IP 参考模型中，应用层协议常用的有：

TCP/IP协议集

应用层	Telnet, FTP, SMTP, DNS, HTTP 以及其他应用协议
传输层	TCP, UDP
网络层	IP, ARP, RARP, ICMP
网络接口	各种通信网络接口（以太网等） （物理网络）

HTTP 请求：Hyper Text Transfer Protocol 的缩写，即超文本传输协议。

一、HTTP 协议的结构

HTTP 报文由客户机到服务器的请求和从服务器到客户机的相应构成。

1、request **请求报文的组成：**

请求行　信息头　请求头　实体头　报文主体

2、respond **响应报文的组成：**

状态行　信息头　响应头　实体头　报文主体

十九、http 协议中的信息头 header 包含什么

1、HTTP 请求方式，POST /test/tupian/cm HTTP/1.1

2、Host：请求的 web 服务器域名地址，URL：http://www.baidu.com:8088/test

3、User-Agent：HTTP 客户端运行的浏览器类型的详细信息。通过该头部信息，web 服务器可以判断到当前 HTTP 请求的客户端浏览器类别。

User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; zh-CN; rv:1.8.1.11) Gecko/20071127 Firefox/2.0.0.11

4、Accept：指定客户端能够接收的内容类型，内容类型中的先后次序表示客户端接收的先后次序。

Accept:text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5

5、Accept-Language：指定 HTTP 客户端浏览器用来展示返回信息所优先选择的语言。

Accept-Language: zh-cn,zh;q=0.5

6、Accept-Encoding：指定客户端浏览器可以支持的 web 服务器返回内容压缩编码类型。表示允许服务器在将输出内容发送到客户端以前进行压缩，以节约带宽。而这里设置的就是客户端浏览器所能够支持的返回压缩格式。ccept-Encoding: gzip,deflate

7、Accept-Charset：浏览器可以接受的字符编码集。Accept-Charset: gb2312,utf-8;q=0.7,*;q=0.7

8、Content-Type：显示此 HTTP 请求提交的内容类型。一般只有 post 提交时才需要设置该属性。

Content-type: application/x-www-form-urlencoded;charset:UTF-8 、 text/xml 、 application/json、form-data

9、Connection：表示是否需要持久连接。如果 web 服务器端看到这里的值为“Keep-Alive”，或者看到请求使用的是 HTTP 1.1（HTTP 1.1 默认进行持久连接），它就可以利用持久连接的优点，当页面包含多个元素时（例如 Applet，图片），显著地减少下载所需要的时间。要实现这一点， web 服务器需要在返回给客户端 HTTP 头信息中发送一个 Content-Length（返回信息正文的长度）头，最简单的实现方法是：先把内容写入 ByteArrayOutputStream，然 后在正式写出内容之前计算它的大小。Connection: keep-alive

10、Keep-Alive：显示此 HTTP 连接的 Keep-Alive 时间。使客户端到服务器端的连接持续有效，当出现对服务器的后继请求时，Keep-Alive 功能避免了建立或者重新建立连接。Keep-Alive: 300

11、cookie：HTTP 请求发送时，会把保存在该请求域名下的所有 cookie 值一起发送给 web 服务器。

12、Referer：包含一个 URL，用户从该 URL 代表的页面出发访问当前请求的页面

二十、TCP 的优点

可靠，稳定 TCP 的可靠体现在 TCP 在传递数据之前，会有三次握手来建立连接，而且

在数据传递时，有确认、窗口、重传、拥塞控制机制，在数据传完后，还会断开连接用来节约系统资源。 **TCP 的缺点：** 慢，效率低，占用系统资源高，易被攻击 TCP 在传递数据之前，要先建连接，这会消耗时间，而且在数据传递时，确认机制、重传机制、拥塞控制机制等都会消耗大量的时间，而且要在每台设备上维护所有的传输连接，事实上，每个连接都会占用系统的 CPU、内存等硬件资源。 而且，因为 TCP 有确认机制、三次握手机制，这些也导致 TCP 容易被人利用，实现 DOS、DDOS、CC 等攻击。

二十一、UDP 的优点

快，比 TCP 稍安全 UDP 没有 TCP 的握手、确认、窗口、重传、拥塞控制等机制，UDP 是一个无状态的传输协议，所以它在传递数据时非常快。没有 TCP 的这些机制，UDP 较 TCP 被攻击者利用的漏洞就要少一些。但 UDP 也是无法避免攻击的，比如：UDP Flood 攻击…… **UDP 的缺点：** 不可靠，不稳定 因为 UDP 没有 TCP 那些可靠的机制，在数据传递时，如果网络质量不好，就会很容易丢包。 基于上面的优缺点，那么： 什么时候应该使用 TCP： 当对网络通讯质量有要求的时候，比如：整个数据要准确无误的传递给对方，这往往用于一些要求可靠的应用，比如 HTTP、HTTPS、FTP 等传输文件的协议，POP、SMTP 等邮件传输的协议。 在日常生活中，常见使用 TCP 协议的应用如下： 浏览器，用的 HTTP FlashFXP，用的 FTP Outlook，用的 POP、SMTP Putty，用的 Telnet、SSH QQ 文件传输 …… 什么时候应该使用 UDP： 当对网络通讯质量要求不高的时候，要求网络通讯速度能尽量的快，这时就可以使用 UDP。 比如，日常生活中，常见使用 UDP 协议的应用如下： QQ 语音 QQ 视频 TFTP

二十二、TCP 与 UDP 区别总结

1、TCP 面向连接（如打电话要先拨号建立连接）;UDP 是无连接的，即发送数据之前不需要建立连接
2、TCP 提供可靠的服务。也就是说，通过 TCP 连接传送的数据，无差错，不丢失，不重复，且按序到达;UDP 尽最大努力交付，即不保证可靠交付
3、TCP 面向字节流，实际上是 TCP 把数据看成一连串无结构的字节流;UDP 是面向报文的
UDP 没有拥塞控制，因此网络出现拥塞不会使源主机的发送速率降低（对实时应用很有用，如 IP 电话，实时视频会议等）
4、每一条 TCP 连接只能是点到点的;UDP 支持一对一，一对多，多对一和多对多的交互通信
5、TCP 首部开销 20 字节;UDP 的首部开销小，只有 8 个字节
6、TCP 的逻辑通信信道是全双工的可靠信道，UDP 则是不可靠信道

TCP 和 UDP 都是传输层的协议
TCP：传输控制协议
UDP: 数据报文协议

TCP 和 UDP 的区别
UDP 的特点如下：

无链接
UDP 使用尽最大努力交付，不保证可靠性
UDP 是面向报文的，UDP 对应用层交付下来的报文，既不合并，也不拆分，而是保留这些报文的边界。应用层交给 UDP 多长的报文，UDP 就照样发送，即一次发送一个报文
UDP 没有拥塞控制
UDP 支持一对一、一对多、多对一和多对多的交互通信
UDP 的首部开销小，只有 8 字节
TCP 的特点：
TCP 是面向连接的
每条 TCP 连接只能用于两个断点，一对一
TCP 提供可靠交付的服务：连接传输数据、无差错、不丢失、不重复、并且按序到达
TCP 提供全双工通信
面向字节流。TCP 根据对方给出的窗口和当前网络拥塞的程度来决定一个报文应该包含多少个字节

二十三、TCP 如何保证可靠传输

1)应用数据被分割成 TCP 认为最适合发送的数据块。
2)超时重传：当 TCP 发出一个段后，它启动一个定时器，等待目的端确认收到这个报文段。如果不能及时收到一个确认，将重发这个报文段。
3)TCP 给发送的每一个包进行编号，接收方对数据包进行排序，把有序数据传送给应用层。
4)校验和：TCP 将保持它首部和数据的检验和。这是一个端到端的检验和，目的是检测数据在传输过程中的任何变化。如果收到段的检验和有差错，TCP 将丢弃这个报文段和不确认收到此报文段。
5)TCP 的接收端会丢弃重复的数据。
6)流量控制：TCP 连接的每一方都有固定大小的缓冲空间，TCP 的接收端只允许发送端发送接收端缓冲区能接纳的我数据。当接收方来不及处理发送方的数据，能提示发送方降低发送的速率，防止包丢失。TCP 使用的流量控制协议是可变大小的滑动窗口协议。
7)拥塞控制：当网络拥塞时，减少数据的发送。

二十四、单向链表和双向链表的优缺点及使用场景

单向链表：只有一个指向下一个节点的指针。
优点：单向链表增加删除节点简单。遍历时候不会死循环；
缺点：只能从头到尾遍历。只能找到后继，无法找到前驱，也就是只能前进。
适用于节点的增加删除。

双向链表：有两个指针，一个指向前一个节点，一个后一个节点。
优点：可以找到前驱和后继，可进可退；
缺点：增加删除节点复杂，需要多分配一个指针存储空间。
适用于需要双向查找节点值的情况。

二十五、range＆xrange 有什么区别？

在大多数情况下，xrange 和 range 在功能方面完全相同。它们都提供了一种生成整数列表的方法，唯一的区别是 range 返回一个 Python 列表对象，xrange 并不是生成序列，而是作为一个生成器。即他的数据生成一个取出一个。因此如果你有一个非常巨大的列表，那么就要考虑 xrange。xrange 比 range 性能优化很多，因为他不需要一下子开辟一块很大的内存，特别是数据量比较大的时候

二十六、内存泄漏和内存溢出的区别和联系

1、内存泄漏 memory leak :是指程序在申请内存后，无法释放已申请的内存空间，一次内存泄漏似乎不会有大的影响，但内存泄漏堆积后的后果就是内存溢出。
2、内存溢出 out of memory :指程序申请内存时，没有足够的内存供申请者使用，或者说，给了你一块存储 int 类型数据的存储空间，但是你却存储 long 类型的数据，那么结果就是内存不够用，此时就会报错 OOM,即所谓的内存溢出。

3、二者的关系

1.内存泄漏的堆积最终会导致内存溢出

2.内存溢出就是你要的内存空间超过了系统实际分配给你的空间，此时系统相当于没法满足你的需求，就会报内存溢出的错误。

3.内存泄漏是指你向系统申请分配内存进行使用(new)，可是使用完了以后却不归还(delete)，结果你申请到的那块内存你自己也不能再访问（也许你把它的地址给弄丢了），而系统也不能再次将它分配给需要的程序。就相当于你租了个带钥匙的柜子，你存完东西之后把柜子锁上之后，把钥匙丢了或者没有将钥匙还回去，那么结果就是这个柜子将无法供给任何人使用，也无法被垃圾回收器回收，因为找不到他的任何信息。

4.内存溢出：一个盘子用尽各种方法只能装 4 个果子，你装了 5 个，结果掉倒地上不能吃了。这就是溢出。比方说栈，栈满时再做进栈必定产生空间溢出，叫上溢，栈空时再做退栈也产生空间溢出，称为下溢。就是分配的内存不足以放下数据项序列,称为内存溢出。说白了就是我承受不了那么多，那我就报错

隐式内存泄漏。程序在运行过程中不停的分配内存，但是直到结束的时候才释放内存。严格的说这里并没有发生内存泄漏，因为最终程序释放了所有申请的内存。但是对于一个服务器程序，需要运行几天，几周甚至几个月，不及时释放内存也可能导致最终耗尽系统的所有内存。所以，我们称这类内存泄漏为隐式内存泄漏。

5、内存溢出的原因及解决方法：

内存溢出原因：
1.内存中加载的数据量过于庞大，如一次从数据库取出过多数据，尽量采用分页的方式查询
2 . List、MAP 集合类中有对对象的引用，使用完后未清空，使得 JVM 不能回收；
3.代码中存在死循环或循环产生过多重复的对象实体；
4.使用的第三方软件中的 BUG；

5.启动参数内存值设定的过小
内存溢出的解决方案：
第一步，修改 JVM 启动参数，直接增加内存。(-Xms，-Xmx 参数一定不要忘记加。)

第二步，检查错误日志，查看“OutOfMemory”错误前是否有其 它异常或错误。

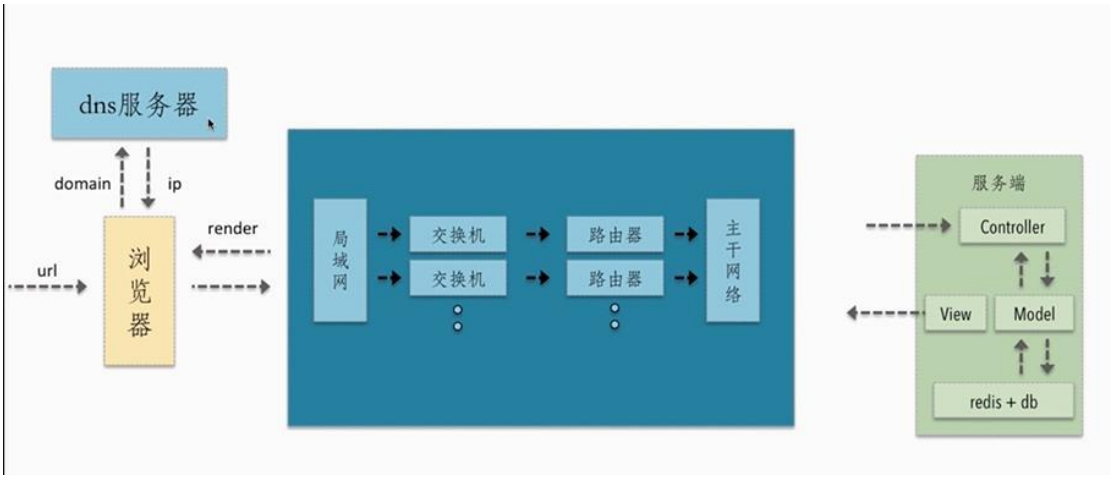
第三步，对代码进行走查和分析，找出可能发生内存溢出的位置。

二十七、从客户端发起请求的数据的流向，会经过哪些服务？

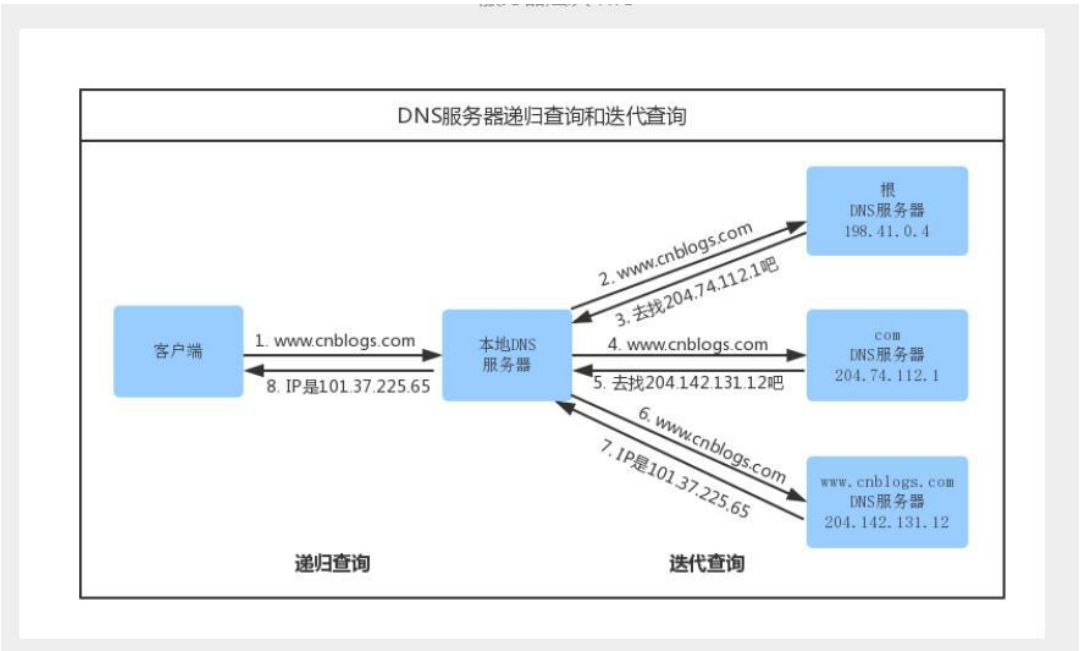
比如 nginx，zuul 网关，服务器，缓存，数据库等等

地址：https://www.cnblogs.com/echo-hui/p/9298203.html

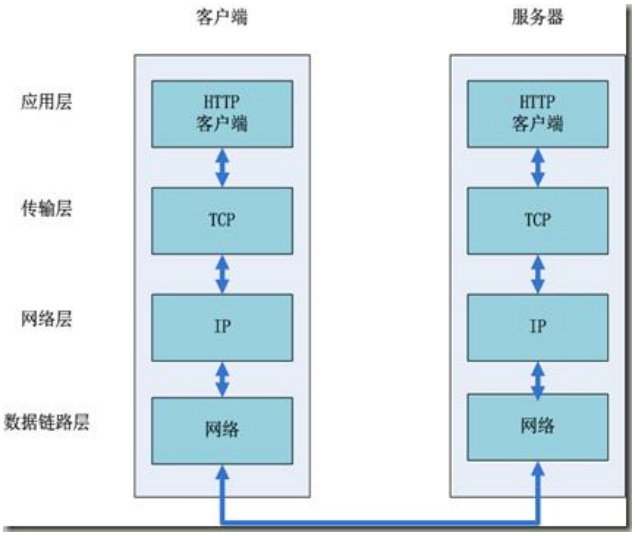
1、用户输入 url，浏览器内部代码将 url 进行拆分解析



2、浏览器首先去找本地的 hosts 文件，检查在该文件中是否有相应的域名、IP 对应关系，如果有，则向其 IP 地址发送请求，如果没有就会将 domain（域）发送给 dns（域名服务器）进行解析（解析如下图），将域名解析成对应的服务器 IP 地址，发回给浏览器

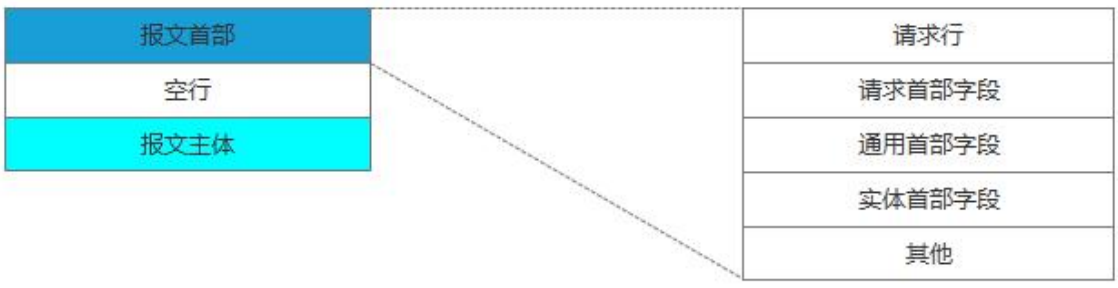


3.浏览器费了一顿周折终于拿到了服务器 IP，接下来就是网络通信（过程如下图），分层由高到低分别为：应用层、传输层、网络层、数据链路层。发送端从应用层往下走，接收端从数据链路层往上走



首先：应用层客户端发送 HTTP 请求

HTTP 请求包括**请求报头和请求主体两个部分，其中请求报头包含了至关重要的信息，包括请求的方法（GET / POST）、目标 url、遵循的协议（http / https / ftp...），返回的信息是否需要缓存，以及客户端是否发送 cookie 等。**



请求报文

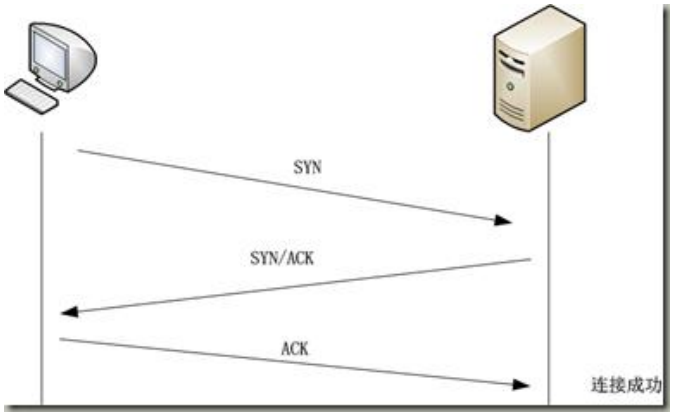
然后：传输层 TCP 传输报文

位于传输层的 TCP 协议为传输报文提供可靠的**字节流服务。它为了方便传输，将大块的数据分割成以报文段为单位的数据包进行管理，并为它们编号，方便服务器接收时能准确地还原报文信息。TCP 协议通过"三次握手"等方法保证传输的安全可靠。**

客户端发送一个带有 SYN 标志的数据包给服务端，在一定的延迟时间内等待接收的回复。服务端收到后，回传一个带有 SYN/ACK 标志的数据包以示传达确认信息，最后客户端再回传一个带 ACK 标志的数据包，代表握手结束，连接成功。

SYN （*Synchronize Sequence Numbers*）*同步序列编号*
ACK （Acknowledgement）确认字符

下图也可以这么理解：
客户端："你好，在家不，有你快递。"---SYN
服务端："在的，送来就行。"-----SYN/ACK
客户端："好嘞。"-----ACK



接着：网络层 IP 协议查询 MAC 地址

IP 协议的作用是把 TCP 分割好的各种**数据包传送给接收方。而要保证确实能传到接收方还需要接收方的 MAC 地址，也就是物理地址。IP 地址和 MAC 地址是一一对应的关系，一个网络设备的 IP 地址可以更换，但是 MAC 地址一般是固定不变的。ARP 协议可以将 IP 地址解析成对应的 MAC 地址。当通信的双方不在同一个局域网时，需要多次中转才能到达最终的目标，在中转的过程中需要通过下一个中转站的 MAC 地址来搜索下一个中转目标。**

最后：数据到达数据链路层

在找到对方的 MAC 地址后，就将数据发送到数据链路层传输。这时，客户端发送请求的阶段**结束**

再次：服务器接收数据

接收端的服务器在链路层接收到数据包，**再层层向上直到应用层。这过程中包括在运输层通过 TCP 协议将分段的数据包重新组成原来的 HTTP 请求报文。**

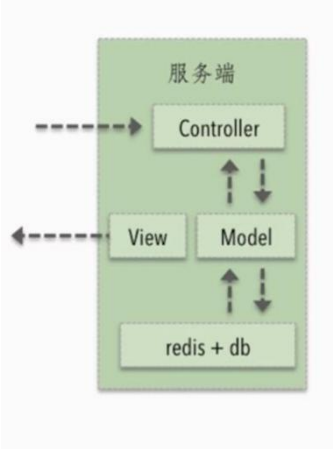
服务器响应请求

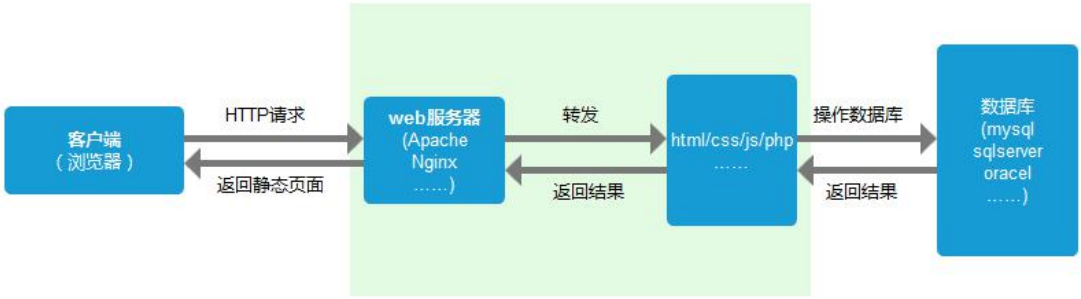
服务接收到客户端发送的 HTTP 请求后，查找客户端请求的资源，并返回响应报文，响应报文中包括一个重要的信息——状态码。状态码由三位数字组成，
其中比较常见的是 200 OK 表示请求成功。

301 表示永久重定向，即请求的资源已经永久转移到新的位置。在返回 301 状态码的同时，响应报文也会附带重定向的 url，客户端接收到后将 http 请求的 url 做相应的改变再重新发送。
404 not found 表示客户端请求的资源找不到。

最后： 服务器返回相应文件

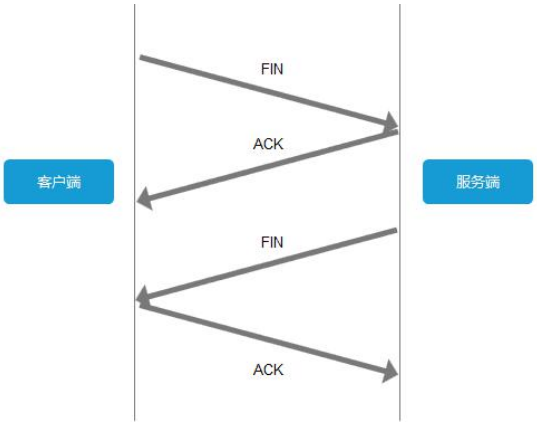
服务器端收到请求后的由 web 服务器（准确说应该是 http 服务器）处理请求，诸如 Apache、Ngnix、IIS 等。web 服务器解析用户请求，知道了需要调度哪些资源文件，再通过相应的这些资源文件处理用户请求和参数，并调用数据库信息，最后将结果通过 web 服务器返回给浏览器客户端。
服务器有自己的 MVC 结构（如下图）





关闭 TCP 连接

为了避免服务器与客户端双方的资源占用和损耗，当双方没有请求或响应传递时，任意一方都可以发起关闭请求。与创建 TCP 连接的 3 次握手类似，关闭 TCP 连接，需要 4 次握手。



4 次握手

上图可以这么理解：

客户端：“兄弟，我这边没数据要传了，咱关闭连接吧。”----FIN

服务端：“收到，我看看我这边有木有数据了。”----ACK

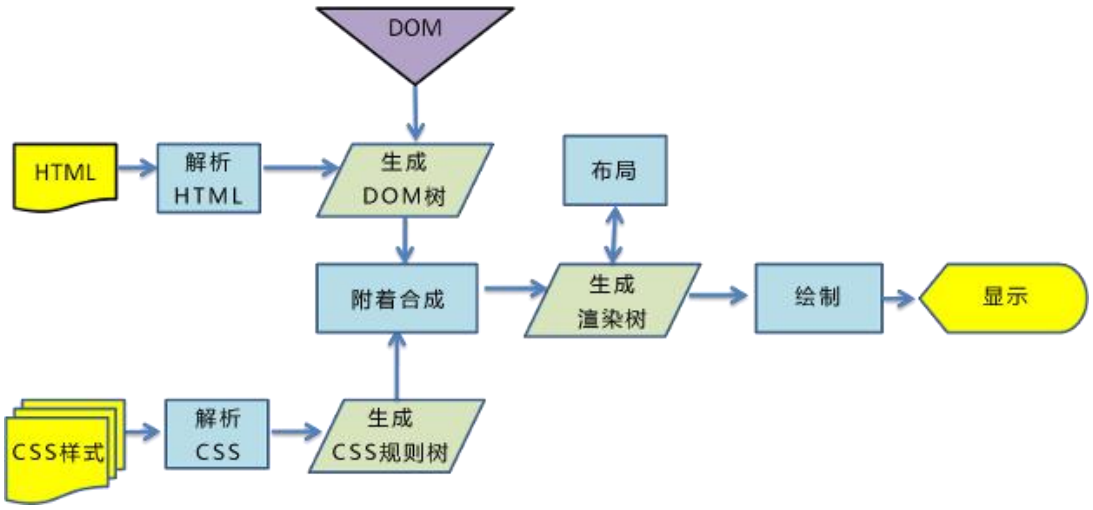
服务端：“兄弟，我这边也没数据要传你了，咱可以关闭连接了。”----FIN

客户端：“好嘞。”----ACK

4.页面的渲染阶段

流程：

1. 解析 HTML 生成 DOM 树。
2. 解析 CSS 生成 CSSOM 规则树。
3. 将 DOM 树与 CSSOM 规则树合并在一起生成渲染树。
4. 遍历渲染树开始布局，计算每个节点的位置大小信息。
5. 将渲染树每个节点绘制到屏幕。



webkit 渲染引擎流程

二十八、Linux 和 Windows 内存管理的区别

【1】Linux 系统：

Linux 优先使用物理内存，当物理内存还有空闲时，linux 是不会释放内存的，即时占用内存的程序已经被关闭了（这部分内存就用来做缓存了）。也就是

说，即时你 有很大的内存,用过一段时间后，也会被占满。这样做的好处是，启动那些刚开启过的程序、或是读取刚存取过得数据会比较快，对于服务器很有好处。

【2】Windows 系统：

windows 则总是给内存留下一定的空闲空间，即时内存有空闲也会让程序使用一些虚拟内存，这样做的好处是，启动新的程序比较快，直接分给它些空闲 内存就可以了，而 linux 下呢？由于内存经常处于全部被使用的状态，则要先清理出一块内存，再分配给新的程序使用，因此，新程序的启动会慢一些。

二十九、jmeter 引用 jar 包的 3 种方式及启动方式

方式一：把 jar 包加入到测试计划里的 classpath

方式二：把 jar 包放到 lib/ext 下

方式三：把 jar 包放到自定义目录下，在 bin 目录下的 jmeter.properties 文件中，搜索 dependencies

`jmx file`：Jmeter 保存的脚本；

`results file`:结果保存文件类型

Jmeter 命令行启动

Path to web report folder：Hmtl 报告保存路径

jmeter -n -t [jmx file] -l [results file] -e -o [Path to web report folder]

jmeter -n -t Runtime_demo.jmx -l 7dg.jtl -e -o 7dhtml

命令参数	说明
-n	非 GUI 模式 -> 在非 GUI 模式下运行 JMeter
-t	测试文件 -> 要运行的 JMeter 测试脚本文件
-l	日志文件 -> 记录结果的文件
-e	负载测试后生成报告仪表板
-o	负载测试后在其中生成报告仪表板的输出文件夹。文件夹不能存在或为空
-H	[代理服务器主机名或 IP 地址]
-P	[代理服务器端口]
-R	[远程服务器列表]在指定的远程服务器中运行测试

三十、jmeter,如何做加密的

1、beanshell 脚本，导入 jar 包

2、函数助手实现参数的 MD5 加密

地址：https://blog.csdn.net/qq_28863851/article/details/84776952

三十一、jmeter，如何做参数化，关联，断言？

参数化：

1、 CSV Data Set Config 组件
2、 用户定义的变量
3、 函数助手
4、 测试计划 Test Plan 里增加参数，${变量名}方式引用
5、 User Defined Variables 组件，定义的变量和 Test Plan 中定义的变量一样，不管这个组件在任何位置，在整个 test plan 中都可以引用这些变量。

关联：

1、 JSON Path Extractor
2、 正则

断言：

1、 响应断言

三十二、lr 中，get,post 函数名，什么是关联，关联的函数是什么

web_url　方法可以 GET 类型的请求

web_custom_request 方法可以发送 POST 和 GET 类型的请求

web_submit_data 只能发送 POST 类型的请求，提供了所有的数据，不管 Cache 存在不存在 Web_submit_data 都是可以工作的

Web_submit_form。只提供了和 Cache 中有差别的数据，其余的数据会自动从 Cache 中取

关联函数 web_reg_save_param

三十三、如何用 mysql 命令进行备份和恢复？

以 test 库为例，创建一个备份，并再用此备份 进行恢复。

mysqldump -u root -p test > test.sql

mysql -u root -p test < test.sql

（1）备份所有的数据库

mysqldump -u root -p --all-databases > bak.sql

（2）备份指定的数据库 db1、db2 以及 db3

mysqldump -u root -p --databases db1 db2 db3 > bak.sql

（3）备份 db 数据库，当仅备份一个数据库时，--databases 可以省略

mysqldump -u root -p db4 > bak.sql

mysqldump -u root -p --databases db4 > bak.sql

两者之间的差别在于不使用 --databases 选项，则备份输出信息中不会包含 CREATE DATABASE 或 USE 语句。不使用 --databases 选项备份的数据文件，在后期进行数据还原操作时，如果该数据库不存在，必须先创建该数据库。

使用 mysql 命令读取备份文件，实现数据还原功能：

mysql -u root -p < bak.sql

mysql -u root -p db4 < bak.sq1

三十四、LoadRunner 由哪些部件组成？

1、Virtual(虚拟) User Generator(生成器)-生成用于“Controlier”部署的，在 VU 左边导航栏中，有三个 LR 框架函数，分别是 vuser_init(),Action(),vuser_end()。这三个函数存在与任何 Vuser 类型的脚本中。

- ●vuser_init()：虚拟用户的初始化函数，存放用户初始化操作，如登录、分配内存等。（只运行一次）
- ●Action：虚拟用户要做的业务，也就是测试内容的主体。参数化只对 Action 有效，Action 重复运行。
- ●vuser_end()：与 init()相对，做收尾工作，如退出登录，free 内存。（同样只运行一次）

2、Vusers LoadRunner Controlier-产生压力和搜集测试结果数据

3、LoadRunner Analysis-通过图表分析工具来编译和显示测试结果

三十五、LoadRunner 使用 IP 欺骗作用是什么？

如何模拟 IP 欺骗测试

地址：https://wenku.baidu.com/view/26c9ecabf90f76c661371a71.html

1、当某个 IP 的访问过于频繁，或者访问量过大是，服务器会拒绝访问请求，这时候通过 IP 欺骗可以增加访问频率和访问量，以达到压力测试的效果。

2、某些服务器配置了负载均衡，使用同一个 IP 不能测出系统的实际性能。LR 中的 7a686964616fe59b9ee7ad9431333264633437IP 欺骗通过调用不同的 IP，可很大程度上的模拟实际使用中多 IP 访问和并测试服务器均衡处理的能力。

3、有一些网站会限制同一个用户同一个 IP 的登陆。为了更加真实的模拟实际情况，LoadRunner 允许运行的虚拟用户使用不 同的 IP 访问同一网站。

添加 IP　在开始菜单中打开 LoadRunner 目录下的 IP Wizard(向导)

在 Controller 中打开 IP Spoofer(勺子)开关，点击 Enable IP Spoofer，使其被勾选上

2. 打开开关　在 Controller 中打开 IP Spoofer 开关，点击 Enable(启用) IP Spoofer，使其被勾选上。

然后在 Tools 菜单勾选上 Export Mode,然后再打开 Options 菜单。必须先勾选上 Export Mode，否则 Option 菜单中将看不到后面需要选择的 General 菜单

在 General 菜单中的 Multiple IP address mode 选择 IP address allocation per thread。这里要与脚本当中的 Run-time Setting 的 Multithreading 选择保持一致。

三十六、Nginx 配置及负载均衡时的方式

1、轮询（默认）
每个请求按时间顺序逐一分配到不同的后端服务器，如果后端服务器 down 掉，能自动剔除。
2、weight（权重）
指定轮询几率，weight 和访问比率成正比，用于后端服务器性能不均的情况。
3、ip_hash
每个请求按访问 ip 的 hash 结果分配，这样每个访客固定访问一个后端服务器，可以解决 session 的问题。
4、fair（公平）
按后端服务器的响应时间来分配请求，响应时间短的优先分配。
5、url_hash（第三方）
按访问 url 的 hash 结果来分配请求，使每个 url 定向到同一个后端服务器，后端服务器为缓存时比较有效。

使用 wget 安装的配置文件在 /usr/local/nginx/conf，使用 yum 安装的配置文件在 /etc/nginx 下
nginx -t 可以查看当前使用的 nginx 配置文件地址；locate nginx.conf 可以查看所有的 nginx.conf
启动 nginx 时可以使用-c 配置项指定配置文件 sudo nginx -c /etc/nginx/nginx.conf。

nginx.conf 的主体结构：

1、全局块，从配置文件开始到 events 块之间的内容，主要会设置一些影响 nginx 服务器整体运行的配置指令，主要包括配置运行 Nginx 服务器的用户（组）、允许生成的 worker process 数，进程 PID 存放路径、日志存放路径和类型以及配置文件的引入等。
worker_processes 1024;
这是 Nginx 服务器并发处理服务的关键配置，worker_processes 值越大，可以支持的并发处理量也越多，但是会受到硬件、软件等设备的制约。
2、events(事件) 块，events 块涉及的指令主要影响 Nginx 服务器与用户的网络连接，常用的设置包括是否开启对多 work process 下的网络连接进行序列化，是否允许同时接收多个网络连接，选取哪种事件驱动模型来处理连接请求，每个 word process 可以同时支持的最大连接数等。
3、http 块，这算是 Nginx 服务器配置中最频繁的部分，代理、缓存和日志定义等绝大多数功能和第三方模块的配置都在这里。
①、http 全局块， http 全局块配置的指令包括文件引入、MIME-TYPE 定义、日志自定义、连接超时时间、单链接请求数上限，是否压缩等。
②、server 块，这块和虚拟主机有密切关系，虚拟主机从用户角度看，和一台独立的硬件主机是完全一样的，该技术的产生是为了节省互联网服务器硬件成本。每个 http 块可以包括多个 server 块，而每个 server 块就相当于一个虚拟主机。而每个 server 块也分为全局 server 块，以及可以同时包含多个 locaton(设置) 块。
1、全局 server 块
最常见的配置是本虚拟机主机的监听配置和本虚拟主机的名称或 IP 配置。
2、location (设置)块
一个 server 块可以配置多个 location 块。这块的主要作用是基于 Nginx 服务器接收到的请求字符串（例如 server_name/uri-string），对虚拟主机名称（也可以是 IP 别名）之外的字符串（例如 前面的 /uri-string）进行匹配，对特定的请求进行处理。地址定向、数据缓存和应答控制等功能，还有许多第三方模块的配置也在这里进行。

三十七、深拷贝和浅拷贝的区别

假设 B 复制了 A，当修改 A 时，看 B 是否会发生变化，如果 B 也跟着变了，说明这是浅拷贝（修改堆内存中的同一个值），拿人手短，如果 B 没变，那就是深拷贝，自食其力（修改堆内存中的不同的值）
python 的浅拷贝是对其对象引用的拷贝，并没有将其引用的对象内容进行拷贝， 只是增加了这个对象的引用计数，数据内容会被共享。
python 的深拷贝是对其引用的内容的拷贝，其引用的地址也会不同，数据内容不会被共享。
深拷贝需要导入 copy 模块，而深拷贝还有两种不同的拷贝：
深拷贝的 copy,只是对其地址的拷贝，其地址内容的元素和浅拷贝一样只是拷贝了地址的引用，一旦修改这个元素，都会发生更改。
深拷贝的 deepcopy 是对其地址内容的拷贝以及元素的地址完全拷贝，内容都不会进行共享。

```
import copy
a = [1, 2, 3, 4, ['a', 'b']] #原始对象
b = a                        #赋值，传对象的引用
c = copy.copy(a)             #对象拷贝，浅拷贝
d = copy.deepcopy(a)         #对象拷贝，深拷贝

a.append(5)                  #修改对象 a
a[4].append('c')             #修改对象 a 中的['a', 'b']数组对象

print( 'a = ', a )          ('a = ', [1, 2, 3, 4, ['a', 'b', 'c'], 5])
print( 'b = ', b )          ('b = ', [1, 2, 3, 4, ['a', 'b', 'c'], 5])
print( 'c = ', c )          ('c = ', [1, 2, 3, 4, ['a', 'b', 'c']])
print( 'd = ', d )          ('d = ', [1, 2, 3, 4, ['a', 'b']])
```

三十八、CDN、DNS 有什么作用

CDN 尽可能避开互联网上有可能影响数据传输速度和稳定性的瓶颈和环节，使内容传输的更快、更稳定。通过在网络各处放置节点服务器所构成的在现有的互联网基础之上的一层智能虚拟网络，CDN 系统能够实时地根据网络流量和各节点的连接、负载状况以及到用户的距离和响应时间等综合信息将用户的请求重新导向离用户最近的服务节点上。
DNS 即域名解析系统，域名和 IP 地址相互映射的一个分布式数据库，提供域名转到对应 ip 的服务

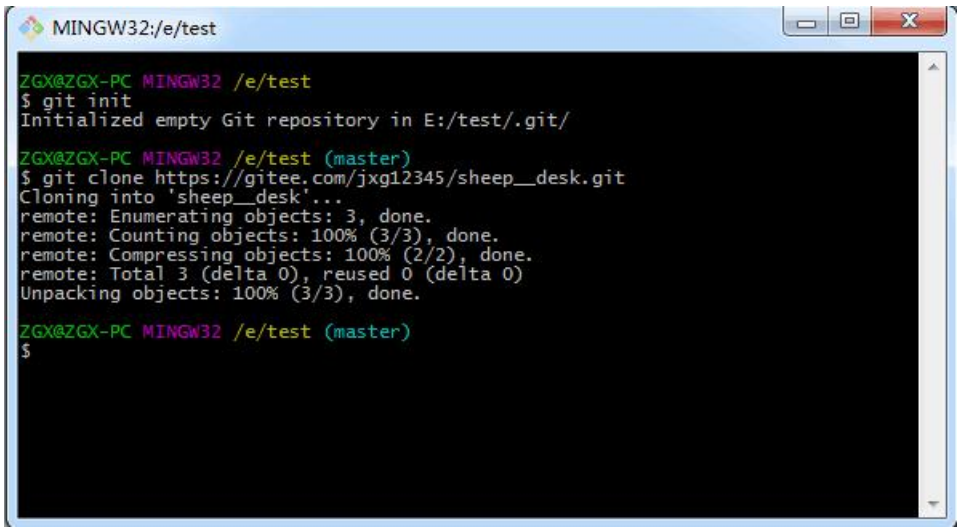
三十九、Git 命令



单独下载内核，下载代码命令

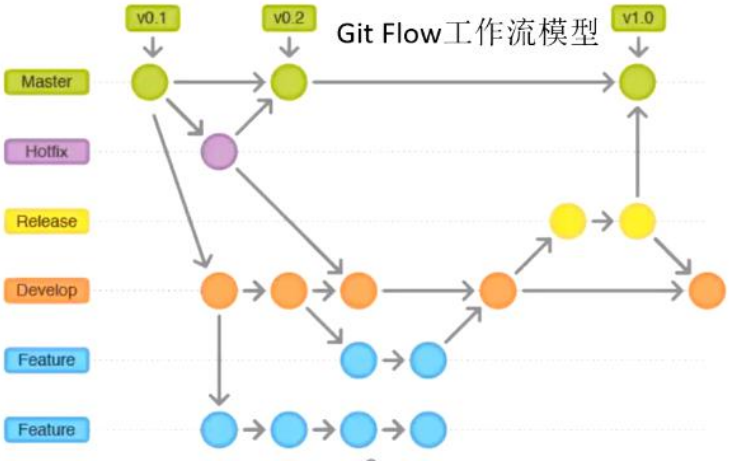
git clone git://android.git.kernel.org/kernel/linux-2.6.git

第一步：创建一个本地的版本库（即新建一个文件夹）
第二步：选中文件夹右键--Git bash here--进入控制面板，输入命令 git init 初始化化文件夹，把这个文件夹变成 Git 可管理的仓库
第三步：把 gitee（码云）上的项目地址复过来，git clone "你的仓库地址"，点击回车如下图



流程：本地目录---暂存区---本地仓库---远程仓库
1、 git init 初始化，变成本地仓库
2、 git add . 提交到暂存区，点代表该路径下的所有文件目录，或者直接写文件名称
3、 git status –s 简化查看的状态信息

4、 git commit –m “增加的注释信息” 提交本地仓库
5、 git log 查看日志
6、 git remote add test1 git@地址 增加远程版本库名字叫 test1
7、 git push test1 master 推送到 test1 的主干，默认不写推送到主干
8、 git diff 不加任何参数 用于比较当前工作区跟暂存区的差异
9、 git checkout a.txt 丢弃工作区的修改，修改后还没放到暂存区，此时撤回修改就回到和版本库一样的状态，如果已经增加到了缓存区，又做了修改，在撤回就回到增加暂存区后的状态，也作用切换分支使用
10、git reset –hard HEAD 本地仓库版本回退、回滚，每次提交的代码都有一个唯一的 commit id（版本号），例如：git reset --hard b498237e6dc1fc4861c79d3314d07285995b
11、git branch 分支名 创建一个分支
12、git branch –d 分支名 删除分支
13、master(主分支)、hitfix(热修复)、release(测试分支)、develop(开发分支)、feature(新需求分支)



git clone 和 git pull 的区别：
1、 git clone 是将整个工程复制下来所以，不需要本地是仓库（没有.git 文件夹）
2、 git pull 需要先初始化本地文件夹文一个仓库
3、 切换分支，clone 远端已经有的分支可以直接切换，pull 只能在当前分支
4、 Pull 问题，clone 直接指定分支可以推送到远端，pull 的话需要执行 add 命令来指定源，然后才能 pull
GitHub 与 GitLab 的区别：GitLab 是可以部署到自己的服务器上

四十、Maven 命令

1 通过 git 命令下载项目：
git clone 项目 git 地址
git clone 项目 git 地址 git clone http://gitmy.wanmei.com/parden-services/parden-provider.git

2 maven 打包 springboot 项目：
cd 项目根目录（和 pom.xml 同级）

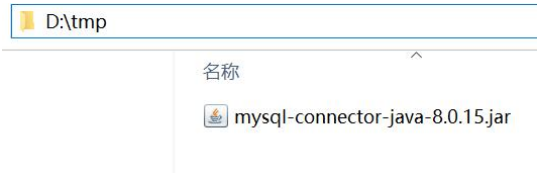
mvn clean package
(打包出来的是 war 包还是 jar 包取决于你项目 pom.xml 文件中 packaging 是怎么指定的。

```
<packaging>jar</packaging>
```

package 和 install 区别：
mvn clean package 只会打包,自动生成 target 目录
mvn install mvn install 包含打包过程，并将打包好的包安装到本地 maven 仓库

maven 安装 jar 包 ---分两种场景
场景一 ---如果整个项目已经 git 下来了：
cd 项目根目录（和 pom.xml 同级）
mvn clean install 或 直接 mvn install

场景二 ---如果只是从别处拷贝过来的一个单独的 jar 包
举例：比如我从 manven 官方仓库上下载了一个 mysql 驱动的 jar 包 mysql-connector-java-8.0.15.jar，把它放过在了 D 盘 tmp 目录下。



在下图中已经给出这个 jar 包的 groupId，artifactId，version 信息，
手动安装的时候这些信息不要改，否则 Maven 项目移植的话，jar 包下载就会失败。



那么 mvn 安装 jar 包的命令为：
mvn install:install-file -Dfile=jar 包的位置 -DgroupId=上面的 groupId -DartifactId=上面的 artifactId -Dversion=上面的 version -Dpackaging=jar

mvn install:install-file -Dfile=D:\tmp\mysql-connector-java-8.0.15.jar -DgroupId=mysql -DartifactId=mysql-connector-java -Dversion=8.0.15 -Dpackaging=jar

四十一、Docker 开源的应用容器引擎

docker 是一个开源的应用容器引擎，让开发者可以打包他们的应用以及依赖包到一个可移植的容器中，然后发布到任何流行的 Linux 机器上，也可以实现虚拟化，容器是完全使用沙箱机制，相互之间不会有任何接口。简言之，就是可以在 Linux 上镜像使用的这么一个容器。它基于 LXC 容器技术，使用 GO 语言开发，源代码托管在 Github 上，并遵从 Apache2.0 协议。Docker 采用 C/S 结构，可轻松为任何应用创建一个轻量级的、可移植的、自给自足的容器。Docker 技术的三大核心概念，分别是：

镜像（Image），一个镜像可以创建多个容器
容器（Container）
仓库（Repository）
那个放在包里的“镜像”，就是 Docker 镜像。而我的背包，就是 Docker 仓库。我在空地上，用魔法造好的房子，就是一个 Docker 容器。

docker 可以在什么情况下使用
1.web 应用自动化打包发布，像 tomcat 应用的发布。
2.自动化测试和持续集成、发布
3.在服务型环境中部署和调整数据库或其他的后台应用
4.搭建 paas 环境

使用 docker 有什么好处
1.简单易操作，简化了以往复杂的应用程序安装步骤，使用 docker 会变得非常简便。
2.可兼容多种应用，Web 应用、后台应用、数据库应用、大数据应用比如 Hadoop 集群、消息队列等等都可以打包成镜像部署。
3.省钱 docker 结合云可以做到高性能低价格
优点：适应场景多、环境部署快，更新快、用的人多、省钱省力省人工
缺点：以利于操作系、依赖网络(需要拉取应用)、银行 U 盾等不能用(因为要拉取应用)

命令：yum install docker –y(y 代表默认安装)、systemctl start docker(启动服务)
、systemctl stop docker、systemctl restart docker、docker pull python:3.6(安装)
、docker image ls(查看所有安装镜像)、
docker images -q (可以 只看容器的 ID 信息)、docker history -H mongo(显示指定镜像的历史创建，-H 镜像大小和日期，默认为 true；--no-trunc 显示完整的提交记录)、
docker search redis(镜像的搜索)、docker search --filter=stars=600 redis(搜索 redis 筛选出来 stars 大于等于 600 的)、docker pull redis(容器下载)、docker rmi redis(容器删除)
docker version(查看 docker 的版本)、docker info(查看容器的信息)

##新建并启动自定义容器名称，参数：
-i interactive=ture|false 默认是 false 以交互模式运行容器；

-t　tty=true|false 默认是 false 为容器重新分配一个伪输入终端；

--name　为容器指定一个名称

docker run -i -t --name my　或者 docker run --name=test1 -i -t ubuntu，定义为 test1

```
[admin@bogon ~]$ docker run --name=test1 -i -t ubuntu
root@14d7e4718cc1:/# exit
exit
[admin@bogon ~]$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                      PORTS               NAMES
14d7e4718cc1        ubuntu              "/bin/bash"         11 seconds ago      Exited (0) 4 seconds ago                        test1
acd58890320e        ubuntu              "/bin/bash"         13 minutes ago      Exited (0) 13 minutes ago                       vigilant_vil
lani
```

##后台启动容器，参数：-d　已守护方式启动容器

docker run -d my

##重新启动已建立的停止的容器，不必每次建立新的

docker start -i -t test1

##查看已建立的容器状态

可以按 id 或者自动生成唯一的名称查询

docker inspect acd58890320e　　或者 docker inspect vigilant_villani

##查看正在运行的容器

docker ps

##查看正在运行的容器的 ID

docker ps -q

##查看正在运行+历史运行过的容器

docker ps -a

##显示运行容器总文件大小

docker ps -s

##显示最近创建容器

docker ps -l

##显示最近创建的 3 个容器

docker ps -n 3

##停止一个运行中的容器

docker stop redis

##杀掉一个运行中的容器

docker kill redis

##删除一个已停止的容器

docker rm redis　　或者 docker rm 2fc08691bb96　　按 id 删除会返回 id，按名称删除返回名称

##删除一个运行中的容器

docker rm -f redis

##退出交互式容器并在后台运行

Ctrl+p+q

##重新启动后台运行的容器

docker attach test1

##查看容器最新 10 条日志

docker logs -f -t –tall 10 test1

-f　--follows=true|false 默认为 false　　循环读取，一直更新

-t　--timestamps=true|false 默认为 false　显示时间

--tail 0　　如果设为 0，取全部最新的日志，或者不加也可以取到全部日志

##查看守护式容器内进程

前提是容器必须在后台运行中，停止的查看不了

docker top test1

##centos 系统安装 python

docker pull python:3.6　　如果是 ubuntu 系统，apt –get install –y python

```
[admin@localhost ~]$ docker pull python:3.6
3.6: Pulling from library/python
d6ff36c9ec48: Pull complete
c958d65b3090: Pull complete
edaf0a6b092f: Pull complete
80931cf68816: Pull complete
bc1b8aca3825: Pull complete
6bdbc0af74d9: Pull complete
f7ef6253c491: Pull complete
3175f9abc051: Pull complete
34b14a02188e: Pull complete
Digest: sha256:4ec64894c98dd0de6804e046359be6ae0820699d89a98ed4b2f62ccbea37f044
Status: Downloaded newer image for python:3.6
docker.io/library/python:3.6
```

创建容器：docker run –it –name=123 python:3.6 bash

docker run -i -t ubuntu /bin/bash　或者 docker run -i -t ubuntu bash

It 指交互模式，name 起的名字，在 python3.6 镜像上创建容器，已 bash 方式启动容器

直接输入 python 回车后，就可以输入 python 代码了，exit()退出后还在这个容器里，只有再次输入 exit 后才真正退出这个 123 的容器，返回到宿主机里。

docker ps -a 查看有哪些容器，docker rm 123 删除容器

导出容器压缩：docker save python:3.6 –o python_test.tar

解压缩：docker load <python_test.tar

特别之处：Docker 是基于 Linux 64bit 的，在 32 位机器上无法运行。

特性	虚拟机	容器
隔离级别	操作系统级	进程级
隔离策略	Hypervisor	CGroups
系统资源	5~15%	0~5%
启动时间	分钟级	秒级
镜像存储	GB-TB	KB-MB
集群规模	上百	上万
高可用策略	备份、容灾、迁移	弹性、负载、动态

四十二、什么是迭代器 Iterator

顾名思义，迭代器就是用于迭代操作（for 循环）的对象，它像列表一样可以迭代获取其中的每一个元素，任何实现了 __next__ 方法 （python2 是 next）的对象都可以称为迭代器。它与列表的区别在于，构建迭代器的时候，不像列表把所有元素一次性加载到内存，而是以

一种延迟计算（lazy evaluation）方式返回元素，这正是它的优点。比如列表含有中一千万个整数，需要占超过 400M 的内存，而迭代器只需要几十个字节的空间。因为它并没有把所有元素装载到内存中，而是等到调用 next 方法时候才返回该元素（按需调用 call by need 的方式，本质上 for 循环就是不断地调用迭代器的 next 方法）。迭代器的特性：惰性运算。在 Python 中，迭代器是遵循迭代协议的对象。使用 iter()从任何序列对象中得到迭代器（如 list, tuple, dictionary, set 等）。另一种形式的输入迭代器是 generator（生成器）

四十三、什么是生成器 generator

知道迭代器之后，就可以正式进入生成器的话题了。普通函数用 return 返回一个值，和 Java 等其他语言是一样的，然而在 Python 中还有一种函数，用关键字 yield 来返回值，这种函数叫生成器函数，函数被调用时会返回一个生成器对象，生成器本质上还是一个迭代器，也是用在迭代操作中，因此它有和迭代器一样的特性，唯一的区别在于实现方式上不一样，后者更加简洁。生成器也是一种迭代器，但是你只能对其迭代一次。这是因为它们并没有把所有的值存在内存中，而是在运行时生成值。生成器表达式和列表推导式很像，仅仅是将列表推导式中的[]替换为()。

为什么 list、dict、str 等数据类型不是迭代器 Iterator？

这是因为 Python 的迭代器 Iterator 对象表示的是一个**数据流**，Iterator 对象可以被 next()函数调用并不断返回下一个数据，直到没有数据时抛出 StopIteration 错误。可以把这个数据流看做是一个有序序列，但我们却不能提前知道序列的长度，只能不断通过 next()函数实现按需计算下一个数据，所以 Iterator 的计算是惰性的，只有在需要返回下一个数据时它才会计算。

Iterator 甚至可以表示一个无限大的数据流，例如全体自然数。而使用 list 是永远不可能存储全体自然数的。

四十四、迭代器、生成器区别

①生成器是生成元素的，迭代器是访问集合元素的一中方式

②迭代输出生成器的内容

③迭代器是一种支持 next()操作的对象

④迭代器（iterator）：其中 iterator 对象表示的是一个数据流，可以把它看做一个有序序列，但我们不能提前知道序列的长度，只有通过 nex()函数实现需要计算的下一个数据。可以看做生成器的一个子集。

举例如下：

```
#列表生成式
lis = [x*x for x in range(10)]
print(lis)
#生成器
generator_ex = (x*x for x in range(10))
print(generator_ex)
```

结果：
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
<generator object <genexpr> at 0x000002A4CBF9EBA0>

#生成器
generator_ex = (x*x for x in range(10))
for i in generator_ex:
    print(i)

结果：
0
1
4
9

四十五、python+unittest 接口自动化执行多条测试用例怎么实现？

并发用用多线程：@threads(3)，数据驱动用 ddt
地址：https://www.cnblogs.com/garvicker/p/9771276.html

四十六、Selenium-显示等待与隐式等待

当浏览器加载页面时，页面上的元素并不会同时被加载，因此增加了定位元素的困难（ElementNotVisibleException 错误）

WebDriver 提供了 2 种类型的等待

显示等待（WebDriverWait 类）：推荐使用，使 WebDriver 等待某个条件成立时继续执行，否则在达到最大时间长时抛出超出时间异常（TimeoutException）
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("http://www.baidu.com")
element = WebDriverWait(driver, 10).until(
EC.presence_of_element_located((By.ID, "kw")))

隐式等待（implicitly_wait 类）：通过一定时长的等待页面上元素加载完成。如果超出了设置

的时间长，元素还未被加载，则抛出异常 NoSuchElementException
driver = webdriver.Chrome()
driver.get("http://www.baidu.com")
driver.implicitly_wait(10)，如果不设置默认为 0

xpath 定位方法：
find_element_by_xpath("//span[@id='input-container']/input")

四十七、冒泡排序、斐波那契数列

冒泡一：

```
a_list = [1, 2, 3, 4, 5, 6]

for t in range(len(a_list)-1):

    for i in range(0, len(a_list)-1):
        tmp = a_list[i]
        if a_list[i] < a_list[i+1]:
            a_list[i] = a_list[i+1]
            a_list[i+1] =tmp

print(a_list)
```

冒泡二：

```
for i in range(len-1):
  for j in range(len-1-i):
      if a[j] > a[j+1]:
         a[j], a[j+1] = a[j+1], a[j]
print(a)
```

斐波那契数列一：

```
lis =[]
for i in range(20):
    if i ==0 or i ==1:#第 1,2 项 都为 1
        lis.append(1)
    else:
        lis.append(lis[i-2]+lis[i-1])#从第 3 项开始每项值为前两项值之和
print(lis)
```

四十八、Redis 持久化，缓存雪崩，缓存穿透？

地址：https://blog.csdn.net/Butterfly_resting/article/details/89668661

什么是 Redis 持久化？Redis 有哪几种持久化方式？优缺点是什么？

持久化就是把内存的数据写到磁盘中去，防止服务宕机了内存数据丢失。

Redis 提供了两种持久化方式:RDB（默认） 和 AOF

比较：
1、aof 文件比 rdb 更新频率高，优先使用 aof 还原数据。
2、aof 比 rdb 更安全也更大
3、rdb 性能比 aof 好
4、如果两个都配了优先加载 AOF

存储结构:
内容是 redis 通讯协议(RESP)格式的命令文本存储。RESP 是 redis 客户端和服务端之前使用的一种通讯协议；

RESP 的特点：实现简单、快速解析、可读性好

缓存雪崩：
我们都知道 Redis 不可能把所有的数据都缓存起来(内存昂贵且有限)，所以 Redis 需要对数据设置过期时间，并采用的是惰性删除+定期删除两种策略对过期键删除。
如果缓存数据设置的过期时间是相同的，并且 Redis 恰好将这部分数据全部删光了。这就会导致在这段时间内，这些缓存同时失效，全部请求到数据库中。
这就是缓存雪崩：Redis 挂掉了，请求全部走数据库。

如何避免缓存雪崩：
事发前：实现 Redis 的高可用(主从架构+Sentinel 或者 Redis Cluster)，尽量避免 Redis 挂掉这种情况发生。
事发中：万一 Redis 真的挂了，我们可以设置本地缓存(ehcache)+限流(hystrix)，尽量避免我们的数据库被干掉(起码能保证我们的服务还是能正常工作的)
事发后：redis 持久化，重启后自动从磁盘上加载数据，快速恢复缓存数据。

缓存穿透：
缓存穿透是指查询一个一定不存在的数据。由于缓存不命中，并且出于容错考虑，如果从数据库查不到数据则不写入缓存，这将导致这个不存在的数据每次请求都要到数据库去查询，失去了缓存的意义，请求的数据在缓存大量不命中，导致请求走数据库。

解决缓存穿透也有两种方案：

由于请求的参数是不合法的(每次都请求不存在的参数)，于是我们可以使用布隆过滤器(BloomFilter)或者压缩 filter 提前拦截，不合法就不让这个请求到数据库层！
当我们从数据库找不到的时候，我们也将这个空对象设置到缓存里边去。下次再请求的时候，就可以从缓存里边获取了。这种情况我们一般会将空对象设置一个较短的过期时间。

如果我们的数据在缓存里边有，那么就直接取缓存的。如果缓存里没有我们想要的数据，我们会先去查询数据库，然后将数据库查出来的数据写到缓存中。最后将数据返回给请求。

缓存与数据库双写一致：
从理论上说，只要我们设置了键的过期时间，我们就能保证缓存和数据库的数据最终是一致的。因为只要缓存数据过期了，就会被删除。随后读的时候，因为缓存里没有，就可以查数据库的数据，然后将数据库查出来的数据写入到缓存中。

四十九、Redis 的数据类型，以及每种数据类型的使用场景

回答：一共五种
(一)String
这个其实没啥好说的，最常规的 set/get 操作，value 可以是 String 也可以是数字。一般做一些复杂的计数功能的缓存。
(二)hash
这里 value 存放的是结构化的对象，比较方便的就是操作其中的某个字段。博主在做单点登录的时候，就是用这种数据结构存储用户信息，以 cookieId 作为 key，设置 30 分钟为缓存过期时间，能很好的模拟出类似 session 的效果。
(三)list
使用 List 的数据结构，可以做简单的消息队列的功能。另外还有一个就是，可以利用 lrange 命令，做基于 redis 的分页功能，性能极佳，用户体验好。本人还用一个场景，很合适—取行情信息。就也是个生产者和消费者的场景。LIST 可以很好的完成排队，先进先出的原则。
(四)set
因为 set 堆放的是一堆不重复值的集合。所以可以做全局去重的功能。为什么不用 JVM 自带的 Set 进行去重？因为我们的系统一般都是集群部署，使用 JVM 自带的 Set，比较麻烦，难道为了一个做一个全局去重，再起一个公共服务，太麻烦了。
另外，就是利用交集、并集、差集等操作，可以计算共同喜好，全部的喜好，自己独有的喜好等功能。
(五)sorted set
sorted set 多了一个权重参数 score,集合中的元素能够按 score 进行排列。可以做排行榜应用，取 TOP N 操作。

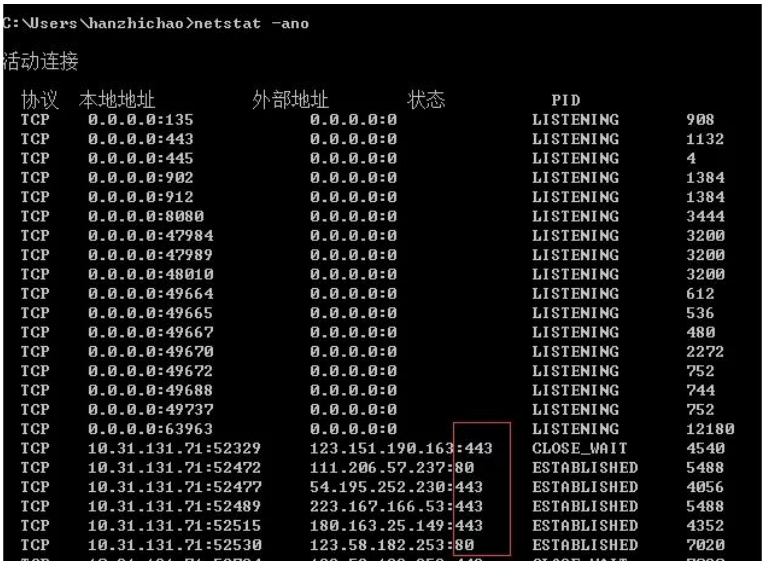
五十、Redis 为什么是单线程的

官方 FAQ 表示，因为 Redis 是基于内存的操作，CPU 不是 Redis 的瓶颈，Redis 的瓶颈最有可能是机器内存的大小或者网络带宽。既然单线程容易实现，而且 CPU 不会成为瓶颈，那就顺理成章地采用单线程的方案了（毕竟采用多线程会有很多麻烦！）Redis 利用队列技术将并发访问变为串行访问
1）绝大部分请求是纯粹的内存操作（非常快速）
2）采用单线程,避免了不必要的上下文切换和竞争条件
3）非阻塞 IO 优点：
1.速度快，因为数据存在内存中，类似于 HashMap，HashMap 的优势就是查找和操作的时间复杂度都是 O(1)

2.支持丰富数据类型，支持 string，list，set，sorted set，hash
3.支持事务，操作都是原子性，所谓的原子性就是对数据的更改要么全部执行，要么全部不执行
4. 丰富的特性：可用于缓存，消息，按 key 设置过期时间，过期后将会自动删除

五十一、常见软件默认端口

Apache/Nginx(HTTP 服务): 80
Tomcat: 8080
Oracle: 1521
MySQL: 3306
SQL Server: 1433
PostgreSQL: 5432
MongoDB: 27017
Redis: 6379
Memcached: 11211
Adb 命令端口：5037
查看端口命令
Windows: netstat -ano
Linux: netstat –ntlp



```
C:\Users\hanzhichao>netstat -ano

活动连接

  协议  本地地址          外部地址        状态           PID
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       908
  TCP    0.0.0.0:443            0.0.0.0:0              LISTENING       1132
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:902            0.0.0.0:0              LISTENING       1384
  TCP    0.0.0.0:912            0.0.0.0:0              LISTENING       1384
  TCP    0.0.0.0:8080           0.0.0.0:0              LISTENING       3444
  TCP    0.0.0.0:47984          0.0.0.0:0              LISTENING       3200
  TCP    0.0.0.0:47989          0.0.0.0:0              LISTENING       3200
  TCP    0.0.0.0:48010          0.0.0.0:0              LISTENING       3200
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING       612
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING       536
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING       480
  TCP    0.0.0.0:49670          0.0.0.0:0              LISTENING       2272
  TCP    0.0.0.0:49672          0.0.0.0:0              LISTENING       752
  TCP    0.0.0.0:49688          0.0.0.0:0              LISTENING       744
  TCP    0.0.0.0:49737          0.0.0.0:0              LISTENING       752
  TCP    0.0.0.0:63963          0.0.0.0:0              LISTENING       12180
  TCP    10.31.131.71:52329     123.151.190.163:443    CLOSE_WAIT      4540
  TCP    10.31.131.71:52472     111.206.57.237:80      ESTABLISHED     5488
  TCP    10.31.131.71:52477     54.195.252.230:443     ESTABLISHED     4056
  TCP    10.31.131.71:52489     223.167.166.53:443     ESTABLISHED     5488
  TCP    10.31.131.71:52515     180.163.25.149:443     ESTABLISHED     4352
  TCP    10.31.131.71:52530     123.58.182.253:80      ESTABLISHED     7020
```

解决端口占用问题
Windows: netstat -ano | findstr "8080",找到占用端口的程序的 PID -> 打开任务管理器 -> 设置显示 PID -> 找到并结束对于程序
Linux: netstat -ntlp | grep "8080", 找到对应的程序 -> ps -ef | grep "程序名" 找到对于的 pid -> kill 相应的 id

五十二、http 八种请求方法

序号	方法	描述
1	GET	请求指定的页面信息，并返回实体主体
2	POST	向指定资源提交数据进行处理请求（例如提交表单或者上传文件）数据被包含在请求体中
3	HEAD	类似于 get 请求，只不过返回的响应中没有具体的内容，用于获取报头
4	PUT	从客户端向服务器传送的数据取代指定的文档的内容
5	DELETE	请求服务器删除指定的页面
6	CONNECT	HTTP/1.1 协议中预留给能够将连接改为管道方式的代理服务器
7	OPTIONS	允许客户端查看服务器的性能
8	TRACE	回显服务器收到的请求，主要用于测试或诊断

五十三、常见 Headers（请求头）

Header	解释	示例
Accept	指定客户端能够接收的内容类型	Accept: text/plain, text/html
Accept-Charset	浏览器可以接受的字符编码集。	Accept-Charset: iso-8859-5
Accept-Encoding	指定浏览器可以支持的web服务器返回内容压缩编码类型。	Accept-Encoding: compress, gzip
Accept-Language	浏览器可接受的语言	Accept-Language: en,zh
Cache-Control	指定请求和响应遵循的缓存机制	Cache-Control: no-cache或max-age=315360000
Expires	缓存过期时间	Sun, 17 Oct 2027 14:03:47 GMT
Connection	表示是否需要持久连接。（HTTP 1.1默认进行持久连接）	Connection: close
Cookie	HTTP请求发送时，会把保存在该请求域名下的所有cookie值一起发送给web服务器。	Cookie: $Version=1; Skin=new;
Content-Length	请求的内容长度	Content-Length: 348
Content-Type	请求的与实体对应的MIME信息	Content-Type: application/x-www-form-urlencoded
Date	请求发送的日期和时间	Date: Tue, 15 Nov 2010 08:12:31 GMT
Host	指定请求的服务器的域名和端口号	Host: www.zcmhi.com
Referer	先前网页的地址，当前请求网页紧随其后,即来路	Referer: http://www.zcmhi.com/archives/71.html
User-Agent	User-Agent的内容包含发出请求的用户信息	User-Agent: Mozilla/5.0 (Linux; X11)
Status	状态码	200
Server	web服务信息	nginx
Via	通知中间网关或代理服务器地址，通信协议	Via: 1.0 fred, 1.1 nowhere.com (Apache/1.1)

五十四、http 请求数据(又称为 Request Body 或 Data)

请求数据类型（Content-Type）（重点）

application/x-www-form-urlencoded：网页表单格式（默认）
application/json：REST 接口常用格式
text/xml：xml 格式，RPC 接口，Dubbo 接口常用格式
test/html：html 格式
multipart/form-data: 混合表单，支持上传图片

数据编码
ASCII 码: 单字节，美国信息交换标准码，包含数字，字母，英文标点及一些控制字符
ISO-8859-1：又称 Latin1，单字节，向下兼容 ASCII，用于支持部分于欧洲使用的语言
ANSI 编码：单字节表示英文，双字节表示汉字，对 ASCII 的扩展，不同的国家和地区制定了不同的标准，中文中的 GBK,GB2312 属于 ANSI 编码
Unicode 编码: 采用二个字节编码（英文和中文的字符都以双字节存放），与 ANSI 码不兼容
UTF-8：是目前互联网上使用最广泛的一种 Unicode 编码方式，又称万国码
指定请求数据编码（解决中文乱码）：
请求 Headers 设置 Content-Type: application/json; charset=utf-8

Base64: 一种用 64 个字符来表示任意二进制数据的方法。
Base64 编码的作用：由于某些系统中只能使用 ASCII 字符。Base64 就是用来将非 ASCII 字符的数据转换成 ASCII 字符的一种方法。
而且 base64 特别适合在 http，mime 协议下快速传输数据。

```
{
    "partnerCode": "0001",
    "partnerUserId": "13800138000",
    "serialNo": "12345678900987654321",
    "timeStamp": "20140515141000",
    "macAlg": "30",
    "mac":"102FCA8CF2C76927A463FCF8EABAE342DB5EF4FE363B920A",
    "data": {
        "registerType": "1",
        "registerSource": "A",
        "authRole": "1",
        "invokeAccount": "",
        "idCard": "611111198310102228",
        "linkPhone": "13788888888",
        "linkAdress": "深圳南山科技园",
        "licenseIssueAddress": "1",
        "idCardPhotoHand": "图片 base64 编码",
        "idCardPhotoFront": "图片 base64 编码",
        "licensePlateNo": "粤 B001",
        "licensePlateType": "02",
        "licensePlateIssueProvinceShort": "粤"
    }
}
```

@51CTO博客

五十五、Cookie 和 Session 的区别

**Cookie/Cookies: **是指某些网站为了辨别用户身份、进行 session 跟踪而储存在用户本地终端上的数据（通常经过加密）。
Session：服务端为客户端访问所建立和维持的会话，通常会生成一个唯一的 id，会话有一定的有效期。session 是另一种记录服务器和客户端会话状态的机制。session 是基于 cookie 实现的，session 存储在服务器端，sessionId 会被存储到客户端的 cookie 中
由于 HTTP 是无状态的，即服务器不知道用户上一次做了什么，默认也无法识别用户身份。
比较流行的做法是：
用户访问时服务端建立会话（Session）
将会话 id（Session ID）随响应返回，并保存在客户端的 Cookies 里
后续的访问中，服务器通过辨识，客户端请求时携带的 Cookies 内容来识别用户
区别：
安全性： Session 比 Cookie 安全，Session 是存储在服务器端的，Cookie 是存储在客户端的。
存取值的类型不同：Cookie 只支持存字符串数据，想要设置其他类型的数据，需要将其转换成字符串，Session 可以存任意数据类型。
有效期不同： Cookie 可设置为长时间保持，比如我们经常使用的默认登录功能，Session 一般失效时间较短，客户端关闭（默认情况下）或者 Session 超时都会失效。
存储大小不同： 单个 Cookie 保存的数据不能超过 4K，Session 可存储数据远高于 Cookie，但是当访问量过多，会占用过多的服务器资源。

响应内容



请求内容



cookie 是存在客户端（浏览器）的进程内存中和客户端所在的机器硬盘上
cookie 只能能够存储少量文本，大概 4K 大小
cookie 是不能在不同浏览器之间共享
Session 存在服务器端，存在网站进程的内存中
Session 在初次设置 session 的时候，会在 session 池中实例化一个 session 对象，以 sessionid 的值作为 key，同时会将 key 以 cookie 的形式保存到客户端的内存中
Session 的作用域只存在当前浏览器的会话中，当浏览器关闭以后就会将 sessionid 丢失，但是服务器的 Session 对象要 20 分钟以后才会回收

五十六、授权与加密

常见的接口安全策略：
Session/Cookie 机制：即需要登录，登录后可访问各个接口，最常用的一种策略，适用于内部接口。
固定 appid 模式：用户注册时会生成一个唯一的 appid，用户调用接口时需要携带 appid，适用于公开接口，安全性较差。
动态 token 模式：token 即身份令牌，用户访问接口需要使用个人 appid 临时申请一个 token，token 有一定有效期，适用于公开接口，安全性较 appid 模式好。
开放协议：Basic Auth/ Oauth1.0 / Oauth2.0: 适用于开放接口。
数字签名：将所有请求参数及参数值进行排列拼接，加上用户私钥，再进行 Md5 或其他加密生成一个请求的签名（sign）,请求是需要携带签名，服务器收到请求后，会对请求重新计算签名并核实与请求所携带签名是否一致。安全性较高，可以有效防止请求被篡改。适用于内部接口及微服务接口。
常见的加密算法
在接口数据传输过程中常对一些敏感数据（如密码）进行 Base64 编码或 MD5 加密，以增加安全性。
加密算法分为对称式加密算法和非对称式加密算法，对称式加解密使用同一个秘钥，非对称式使用不同的秘钥。

对称式加密
DES: 数据加密标准，速度较快，适用于加密大量数据的场合

AES: 高级加密标准，速度快，安全级别高
非对称式加密
RSA: 是一个支持变长密钥的公共密钥算法， 分公钥和私钥，SSH 协议使用该算法
MD5: 最常用的一种加密方法，是一种摘要算法。

五十七、ADB 命令

ADB，即 Android Debug Bridge，它是 Android 开发/测试人员不可替代的强大工具，也是 Android 设备玩家的好玩具。安卓调试桥 (Android Debug Bridge, adb)，是一种可以用来操作手机设备或模拟器的命令行工具。它存在于 android sdk 安装目录的 platform-tools 目录下。虽然现在 Android Studio 已经将大部分 ADB 命令以图形化的形式实现了，但是了解一下还是有必要的。
adb 命令的基本语法如下：前提进入 cmd 模式里
adb [-d|-e|-s]
如果有多个设备/模拟器连接，则需要为命令指定目标设备。

参数	含义
-d	指定当前唯一通过 USB 连接的 Android 设备为命令目标
-e	指定当前唯一运行的模拟器为命令目标
-s <serialNumber>	指定相应 serialNumber 号的设备/模拟器为命令目标

举例
$ adb devices

List of devices attached
cf264b8f device
emulator-5554 device
10.129.164.6:5555 device
输出里的 cf264b8f、emulator-5554 和 10.129.164.6:5555 即为 serialNumber。

比如这时想指定 cf264b8f 这个设备来运行 adb 命令获取屏幕分辨率：

adb -s cf264b8f shell wm size

又如想给 10.129.164.6:5555 这个设备安装应用（这种形式的 serialNumber 格式为 :，一般为无线连接的设备或 Genymotion 等第三方 Android 模拟器）：
adb -s 10.129.164.6:5555 install test.apk
查看当前连接设备：
adb devices
如果发现多个设备：

adb -s 设备号 其他指令　　举例：adb -s devicel install xxx.apk

查看顶部 Activity:
windows 环境下:
adb shell dumpsys activity | findstr "mFocusedActivity"

Linux、Mac 环境下：
adb shell dumpsys activity | grep "mFocusedActivity"

查看日志：
adb logcat　　adb logcat >Desk\log.txt　　把日志放到桌面，方便看，把>换成-f 是放在手机系统里

安装 apk 文件：
adb install xxx.apk
此安装方式，如果已经存在，无法安装；
推荐使用覆盖安装：
adb install -r xxx.apk

比如直接 RUN 出来的包是 test-onlu 的无法安装，推荐使用**-t**
adb install -r -t xxx.apk

卸载 App:
adb uninstall com.zhy.app
如果想要保留数据，则：
adb uninstall -k com.zhy.app

查看包名：findstr 代表筛选
aapt d bagding *.apk|findstr “package”

传递文件：往手机 SDCard 传递文件：
adb push 文件名 手机端 SDCard 路径
例如：adb push Desktop\11.jpg /sdcard/local
从手机端下载文件到电脑桌面：
adb pull /sdcard/xxx.txt Desktop\

查看手机端安装的所有 app 包名:
adb shell pm list packages

启动 Activity:
adb shell am start 包名/完整 Activity 路径
例如：
adb shell am start com.zhy.aaa/com.zhy.aaa.MainActivity
如果需要携带参数(携带一个 Intent,Key 为 name):

adb shell am start com.zhy.aaa/com.zhy.aaa.MainActivity -e name zhy

查看 adb 版本：adb version
进入安卓交互模式：adb -s 10.129.164.6:5555 shell
进入安卓本机系统架监控：vmstat 1　　1 指每格一秒打印一次
不进入安卓本机：adb shell vmstat 1

查看手机设备内存
adb shell cat /proc/meminfo

MemTotal：可以使用的内存总和（小于实际RAM，操作系统预留了一部分）
MemFree：未使用的内存
Cached：缓存（这个也是app可以申请到的内存）
HightTotal：内存中地址高于860M的物理内存总和，只能被用户空间的程序使用。
HightFree：内存中地址高于860M的未使用内存
LowTotal：内存中内核和用户空间程序都可以使用的内存总和（对于512M的RAM: lowTotal= MemTotal）
LowFree: 内存中内核和用户空间程序未使用的内存（对于512M的RAM: lowFree = MemFree）

查看应用软件占用内存
adb shell dumpsys meminfo com.qq.frodo
查看手机设备 cpu
adb shell dumpsys cpuinfo
应用软件占用 cpu，这个不显示字段标题是啥，用 adb shell “top”也可以
adb shell “top –n 1|grep com.qq.frodo”
获取 app 的 pid，findstr 代表筛选
adb shell “ps|findstr com.qq.frodo”　　不加引号也行

五十八、RPC 远程过程调用

RPC(Romote Procedure Call)：远程过程调用，允许一台计算机程序远程调用另外一台计算机的子程序，不用关心底层网络通信。
在socket基础上实现，比socket需要更多资源

RPC 是一种技术思想而非一种规范或协议，常见 RPC 技术和框架有：

应用级的服务框架：阿里的 Dubbo/Dubbox、Google gRPC、Spring Boot/Spring Cloud。
远程通信协议：RMI、Socket、SOAP(HTTP XML)、REST(HTTP JSON)。
通信框架：MINA 和 Netty。

RPC 即远程过程调用（Remote Procedure Call Protocol，简称 RPC），像调用本地服务(方法)

一样调用服务器的服务(方法)。通常的实现有 XML-RPC , JSON-RPC , 通信方式基本相同, 所不同的只是传输数据的格式.

RPC 是分布式架构的核心，按响应方式分如下两种：
同步调用：客户端调用服务方方法，等待直到服务方返回结果或者超时，再继续自己的操作
异步调用：客户端把消息发送给中间件，不再等待服务端返回，直接继续自己的操作。

同步调用的实现方式有 WebService 和 RMI。Web Service 提供的服务是基于 web 容器的，底层使用 http 协议，因而适合不同语言异构系统间的调用。RMI 实际上是 Java 语言的 RPC 实现，允许方法返回 Java 对象以及基本数据类型，适合用于 JAVA 语言构建的不同系统间的调用。

异步调用的 JAVA 实现版就是 JMS(Java Message Service)，目前开源的的 JMS 中间件有 Apache 社区的 ActiveMQ、Kafka 消息中间件，另外有阿里的 RocketMQ。

五十九、进程与线程的区别？多线程及实现方式？

1、进程是资源分配最小单位，线程是程序执行的最小单位。计算机在执行程序时，会为程序创建相应的进程，进行资源分配时，是以进程为单位进行相应的分配。每个进程都有相应的线程，在执行程序时，实际上是执行相应的一系列线程。
2、地址空间：进程有自己独立的地址空间，每启动一个进程，系统都会为其分配地址空间，建立数据表来维护代码段、堆栈段和数据段；线程没有独立的地址空间，同一进程的线程共享本进程的地址空间。
3、资源拥有：进程之间的资源是独立的；同一进程内的线程共享本进程的资源。
4、执行过程：每个独立的进程程有一个程序运行的入口、顺序执行序列和程序入口。但是线程不能独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制。
5、线程是处理机调度的基本单位，但是进程不是。由于程序执行的过程其实是执行具体的线程，那么处理机处理的也是程序相应的线程，所以处理机调度的基本单位是线程。
6、系统开销：进程执行开销大，线程执行开销小。

多线程的几种实现方式：
(1) 继承 Thread 类创建线程
Thread 类本质上是实现了 Runnable 接口的一个实例，代表一个线程的实例。启动线程的唯一方法就是通过 Thread 类的 start()实例方法。start()方法将启动一个新线程，并执行 run（）方法。这种方式实现多线程比较简单，通过自己的类直接继承 Thread，并重写 run（）方法，就可以启动新线程并执行自己定义的 run()方法。
(2) 实现 Runnable 接口创建线程
如果自己的类已经继承了两个类，就无法再继承 Thread，因此可以实现一个 Runnable 接口
(3) 实现 Callable 接口通过 FutureTask 包装器来创建 Thread 线程
(4) 使用 ExecutorService、Callable、Future 实现有返回结果的线程
ExecutorService、Callable、Future 三个接口实际上都是属于 Executor 框架。返回结果的线程是在 JDK1.5 中引入的新特征，有了这种特征就不需要再为了得到返回值而大费周折了。
可返回值的任务必须实现 Callable 接口；无返回值的任务必须实现 Runnabel 接口。
执行 Callable 任务后，可以获取一个 Future 对象，在该对象上调用 get()方法就可以获取到

Callable 任务返回的 Object 了。（get()方法是阻塞的，线程无返回结果，该方法就一直等待）

Thread 类中的 strat()和 run()方法有什么区别?
start()方法被用来启动新创建的线程，而且 start()内部调用了 run()方法，这和直接调用 run()方法的效果不一样。当你调用 run()方法的时候，只会在原来的线程中调用，没有新的线程启动，而调用 start()方法会启动一个新的线程。

有三个线程 T1、T2 和 T3，怎么确保它们按照顺序执行？
在多线程中有多重方法让线程按特定的顺序执行，你可以用线程类的 join()方法在一个线程中启动另一个线程，另外一个线程完成该线程继续执行。为了确保三个线程的顺序你应该先启动最后一个（T3 调用 T2，T2 调用 T1），这样 T1 就会先完成而 T3 最后完成

六十、Restful 规范

RESTFUL特点包括：
1、每一个URI代表1种资源；
2、客户端使用GET、POST、PUT、DELETE4个表示操作方式的动词对服务端资源进行操作：GET用来获取资源，POST用来新建资源（也可以用于更新资源），PUT用来更新资源，DELETE用来删除资源；
3、通过操作资源的表现形式来操作资源；
4、资源的表现形式是XML或者HTML；
5、客户端与服务端之间的交互在请求之间是无状态的，从客户端到服务端的每个请求都必须包含理解请求所必需的信息

REST （Representational State Transfer） 表征性状态转移
SOAP （Simple Object Access Protocol）简单的对象访问协议
XML-RPC （XML Remote Procedure Call）基于 XML 的远程过程调用

1、API 与用户的通信协议，总是使用 HTTPS 协议，尽量使用 HTTPS
2、域名规范，在 URL 中体现是否是 API
https://api.example.com 尽量将 API 部署在专用域名（会存在跨域问题）
https://example.org/api/ API 很简单（推荐）
3、版本规范，在 URL 中体现版本
URL，如：https://api.example.com/v1/
请求头 跨域时，引发发送多次请求
4、路径，视网络上任何东西都是资源，均使用名词表示（可复数），每个 URL 代表一种资源，URL 中尽量不要用动词，要用名词。
https://api.example.com/v1/zoos
5、根据 method 不同，进行不同的操作
GET ： 从服务器取出资源（一项或多项）
POST ： 在服务器新建一个资源
PUT ： 在服务器更新资源（客户端提供改变后的完整资源）
PATCH ： 在服务器更新资源（客户端提供改变的属性）

DELETE ：从服务器删除资源
6、在 URL 中的过滤条件，通过在 url 上传参的形式传递搜索条件
https://api.example.com/v1/zoos?limit=10：指定返回记录的数量
https://api.example.com/v1/zoos?offset=10：指定返回记录的开始位置
https://api.example.com/v1/zoos?page=2&per_page=100：指定第几页，以及每页的记录数

7、状态码，响应设置状态码
8、错误处理，应返回错误信息，error 当做 key，返回值携带错误信息
```
{
    'status':状态码,
    'error':错误信息,
}
```
9、返回结果，针对不同操作，服务器向用户返回的结果应该符合以下规范。
GET /collection：返回资源对象的列表（数组）
GET /collection/resource：返回单个资源对象
POST /collection：返回新生成的资源对象
PUT /collection/resource：返回完整的资源对象
PATCH /collection/resource：返回完整的资源对象
DELETE /collection/resource：返回一个空文档
GET 请求 返回查到所有或单挑数据
POST 请求 返回新增的数据
PUT 请求 返回更新数据
PATCH 请求 局部更新 返回更新整条数据
DELETE 请求 返回值为空
10、Hypermedia API，RESTful API 最好做到 Hypermedia，即返回结果中提供链接，连向其他 API 方法，使得用户不查文档，也知道下一步应该做什么。如果遇到需要跳转的情况 携带调转接口的 URL
```
ret = {
    code: 100,
    data: {
            id: 1,
            name: '小明',
            depart_id: 'http://www.baidu.com/api/v1/depart/4/'
        }
}
```

六十一、性能测试流程

1、需求调研
2、测试计划
3、环境搭建
4、数据准备
5、测试脚本
6、压测执行

7、调优回归
8、测试报告

需求调研包含：
1、 项目背景
2、 测试范围
3、 业务逻辑&数据流向
4、 系统架构
5、 配置信息
6、 测试数据量
7、 外部依赖
8、 系统使用场景&业务比例
9、 日常业务量
10、 预期指标
11、 上线时间

六十二、性能测试计划

1、 项目描述
2、 业务模型及性能指标
3、 测试环境说明
4、 测试资源
5、 测试方法及测试场景设计原则
   基准测试、单交易负载测试、混合场景测试、高可用性测试、异常场景测试、稳定性测试、其他特殊场景
6、 测试进度安排及测试准则

六十三、性能测试报告

1、 概述
2、 测试环境
3、 结果与分析
4、 调优说明
5、 项目时间表
6、 结论
7、 建议

六十四、主流协议

Tcp/ip 协议是网络中使用的基本的通信协议，传输控制协议，TCP/IP 协议族是一个四层协议系统，自底而上分别是数据链路层、网络层、传输层和应用层。HTTP（HyperTextTransferProtocol）是超文本传输协议的缩写。UDP(User Datagram Protocol)用户数据报协议。FTP (文件传输协

议 File Transfer Protocol)。POP3 是 Post Office Protocol 3 的简称，即邮局协议的第 3 个版本。

六十五、计算宽带兆数为什么要除以 8

宽带的单位是百 bps，也就是位/秒 ，转化为度字节/秒，此时需要除以 8
因为 1 字节=8 位
例如 4M 宽带指的是 4Mbps，也就内是 4M 位/秒
转换为常见的下载速度容 KB/s ： 4Mbps 除以 8=0.5MB/s=512KB/s

六十六、26 个 linux 常用命令

地址：https://www.cnblogs.com/xuxinstyle/p/9609551.html
【find、ls、cd、tree、cp、rm、mv、pwd、tar、mkdir、rmdir、ps、kill、free、top、 chmod、chown、chgrp、gzip、vim、grep、rz、sz、sed、awk、group】

tail -300f shopbase.log #倒数 300 行并进入实时监听文件写入模式
grep -wn aaa text.txt 搜索 text.txt 中 aaa 的内容并打印行数
grep forest f.txt #文件查找
grep forest f.txt cpf.txt #多文件查找
grep 'log' /home/admin -r -n #目录下查找所有符合关键字的文件
cat f.txt | grep -i shopbase
grep 'shopbase' /home/admin -r -n --include *.{vm,java} #指定文件后缀
grep 'shopbase' /home/admin -r -n --exclude *.{vm,java} #反匹配
grep -iR 'ultra' *.conf #根据文件内容查找文件
-i ： 忽略模式（匹配字符串 valid、 VALID、 ValID ）和输入文件（匹配 file.c FILE.c FILE.C）的大小写。
-R ： 递归读取每个目录下的所有文件
seq 10 | grep 5 -A 3 #上匹配
seq 10 | grep 5 -B 3 #下匹配
seq 10 | grep 5 -C 3 #上下匹配，平时用这个就妥了
cat f.txt | grep -c 'SHOPBASE'

awk '{print $4,$6}' f.txt
awk '{print NR,$0}' f.txt cpf.txt
awk '{print FNR,$0}' f.txt cpf.txt
awk '{print FNR,FILENAME,$0}' f.txt cpf.txt
awk '{print FILENAME,"NR="NR,"FNR="FNR,"$"NF"="$NF}' f.txt cpf.txt
echo 1:2:3:4 | awk -F: '{print $1,$2,$3,$4}'
awk '/ldb/ {print}' f.txt #匹配 ldb
awk '!/ldb/ {print}' f.txt #不匹配 ldb
awk '/ldb/ && /LISTEN/ {print}' f.txt #匹配 ldb 和 LISTEN
awk '$5 ~ /ldb/ {print}' f.txt #第五列匹配 ldb

sudo -u admin find /home/admin /tmp /usr -name *.log(多个目录去找)
find . -iname *.txt(大小写都匹配)
find . -type d(当前目录下的所有子目录)
find /usr -type l(当前目录下所有的符号链接)
find /usr -type l -name "z*" -ls(符号链接的详细信息 eg:inode,目录)
find /home/admin -size +250000k(超过 250000k 的文件，当然+改成-就是小于了)
find /home/admin f -perm 777 -exec ls -l {} \; (按照权限查询文件)
find /home/admin -atime -1 1 天内访问过的文件
find /home/admin -ctime -1 1 天内状态改变过的文件
find /home/admin -mtime -1 1 天内修改过的文件
find /home/admin -amin -1 1 分钟内访问过的文件
find /home/admin -cmin -1 1 分钟内状态改变过的文件
find /home/admin -mmin -1 1 分钟内修改过的文件

1、FIND 命令
与时间有关的参数：
-mtime n : n 为数字，意思为在 n 天之前的“一天内”被更改过的文件；
-mtime +n : 列出在 n 天之前（不含 n 天本身）被更改过的文件名；
-mtime -n : 列出在 n 天之内（含 n 天本身）被更改过的文件名；
-newer file : 列出比 file 还要新的文件名
例如：
find /root -mtime 0 # 在当前目录下查找今天之内有改动的文件

与用户或用户组名有关的参数：
-user name : 列出文件所有者为 name 的文件
-group name : 列出文件所属用户组为 name 的文件
-uid n : 列出文件所有者为用户 ID 为 n 的文件
-gid n : 列出文件所属用户组为用户组 ID 为 n 的文件
例如：
find /home/hadoop -user hadoop # 在目录/home/hadoop 中找出所有者为 hadoop 的文件

与文件权限及名称有关的参数：
-name filename ：找出文件名为 filename 的文件
-size [+-]SIZE ：找出比 SIZE 还要大（+）或小（-）的文件
-tpye TYPE ：查找文件的类型为 TYPE 的文件，TYPE 的值主要有：一般文件（f)、设备文件（b、c)、
目录（d)、连接文件（l)、socket（s)、FIFO 管道文件（p)；
-perm mode ：查找文件权限刚好等于 mode 的文件，mode 用数字表示，如 0755；
-perm -mode ：查找文件权限必须要全部包括 mode 权限的文件，mode 用数字表示
-perm +mode ：查找文件权限包含任一 mode 的权限的文件，mode 用数字表示
例如：
find / -name passwd # 查找文件名为 passwd 的文件
find . -perm 0755 # 查找当前目录中文件权限的 0755 的文件

find . -size +12k # 查找当前目录中大于 12KB 的文件，注意 c 表示 byte

2、LS 命令

ls –lt|sed -n 2p|awk '{print $9}' 按时间查找最新的文件倒序显示，2p 指从第二行开始，$9 指第 9 个段落的名称

-a ：全部的档案，连同隐藏档(开头为 . 的档案) 一起列出来～

-A ：全部的档案，连同隐藏档，但不包括 . 与 .. 这两个目录，一起列出来～

-d ：仅列出目录本身，而不是列出目录内的档案数据

-f ：直接列出结果，而不进行排序 (ls 预设会以档名排序！)

-F ：根据档案、目录等信息，给予附加数据结构，例如：

*：代表可执行档； /：代表目录； =：代表 socket 档案； |：代表 FIFO 档案；

-h ：将档案容量以人类较易读的方式(例如 GB, KB 等等)列出来；

-i ：列出 inode 位置，而非列出档案属性；

-l ：长数据串行出，包含档案的属性等等数据；

-n ：列出 UID 与 GID 而非使用者与群组的名称 (UID 与 GID 会在账号管理提到！)

-r ：将排序结果反向输出，例如：原本档名由小到大，反向则为由大到小；

-R ：连同子目录内容一起列出来；

-S ：以档案容量大小排序！

-t ：依时间排序

3、CD 命令

cd /root/Docements # 切换到目录/root/Docements

cd ./path # 切换到当前目录下的 path 目录中，“.”表示当前目录

cd ../path # 切换到上层目录中的 path 目录中，“..”表示上一层目录

4、tree 命令，显示树形的层级目录结构，非原生命令，需要安装 tree

5、cp 命令，作用复制，参数如下：

-a ：将文件的特性一起复制

-p ：连同文件的属性一起复制，而非使用默认方式，与-a 相似，常用于备份

-i ：若目标文件已经存在时，在覆盖时会先询问操作的进行

-r ：递归持续复制，用于目录的复制行为

-u ：目标文件与源文件有差异时才会复制

6.、rm 命令作用为删除，参数：

-f ：就是 force 的意思，忽略不存在的文件，不会出现警告消息

-i ：互动模式，在删除前会询问用户是否操作

-r ：递归删除，最常用于目录删除，它是一个非常危险的参数

7、 mv 命令作用为移动文件：

-f ：force 强制的意思，如果目标文件已经存在，不会询问而直接覆盖

-i ：若目标文件已经存在，就会询问是否覆盖

-u ：若目标文件已经存在，且比目标文件新，才会更新

-r 就是向下递归，不管有多少级目录，一并删除

8、pwd 命令，作用为查看”当前工作目录“的完整路径
pwd -P # 显示出实际路径，而非使用连接(link)路径；pwd 显示的是连接路径

9、tar 命令，用于压缩解压：
-c ： 新建打包文件
-t ： 查看打包文件的内容含有哪些文件名
-x ：解打包或解压缩的功能，可以搭配-C（大写）指定解压的目录，注意-c,-t,-x 不能同时出现在同一条命令中
-j ： 通过 bzip2 的支持进行压缩/解压缩
-z ： 通过 gzip 的支持进行压缩/解压缩
-v ： 在压缩/解压缩过程中，将正在处理的文件名显示出来
-f filename ： filename 为要处理的文件
-C dir ： 指定压缩/解压缩的目录 dir

压缩：tar -zxfv filename.tar.bz2 要被处理的文件或目录名称
查询：tar -jtv -f filename.tar.bz2
解压：tar -jxv -f filename.tar.bz2 -C 欲解压缩的目录

10、mkdir 命令创建目录：
mkdir [选项]... 目录...
-m, --mode=模式，设定权限<模式> (类似 chmod)，而不是 rwxrwxrwx 减 umask
-p, --parents 可以是一个路径名称。此时若路径中的某些目录尚不存在,加上此选项后,系统将自动建立好那些尚不存在的目录,即一次可以建立多个目录;
-v, --verbose 每次创建新目录都显示信息

11、rmdir 命令删除目录：
rmdir [选项]... 目录...
-p 递归删除目录 dirname，当子目录删除后其父目录为空时，也一同被删除。如果整个路径被删除或者由于某种原因保留部分路径，则系统在标准输出上显示相应的信息。
-v --verbose 显示指令执行过程

12、ps 命令显示运行的进程，还会显示进程的一些信息如 pid, cpu 和内存使用情况等：

-A ： 所有的进程均显示出来
-a ： 不与 terminal 有关的所有进程
-u ： 有效用户的相关进程
-x ： 一般与 a 参数一起使用，可列出较完整的信息
-l ： 较长，较详细地将 PID 的信息列出
ps –ef|grep java

13、kill 命令用于终止进程，
Kill -9 id

14、free 命令用于显示 Linux 系统中空闲的、已用的物理内存及 swap 内存,及被内核使用的 buffer-b　以 Byte 为单位显示内存使用情况。
-k　以 KB 为单位显示内存使用情况。
-m　以 MB 为单位显示内存使用情况。
-g　以 GB 为单位显示内存使用情况。
-o　不显示缓冲区调节列。
-s<间隔秒数>　持续观察内存使用状况。
-t　显示内存总和列。
-V　显示版本信息。

15、top 命令是 Linux 下常用的性能分析工具，能够实时显示系统中各个进程的资源占用状况，类似于 Windows 的任务管理器：
top［参数］
-b 批处理
-c 显示完整的治命令
-I 忽略失效过程
-s 保密模式
-S 累积模式
-i<时间> 设置间隔时间
-u<用户名> 指定用户名
-p<进程号> 指定进程
-n<次数> 循环显示的次数

16、chmod 命令改变文件权限，r=4，w=2，x=1
chmod [-R] xyz 文件或目录
-R：进行递归的持续更改，即连同子目录下的所有文件都会更改
#同时，chmod 还可以使用 u（user）、g（group）、o（other）、a（all）和+（加入）、-（删除）、=（设置）跟 rwx 搭配来对文件的权限进行更改，编号是各种权限的数字代码，示例：
chmod 0755 file # 把 file 的文件权限改变为-rxwr-xr-x
chmod g+w file # 向 file 的文件权限中加入用户组可写权限
chmod 600 file (等价于 chmod u=rw,g=---,o=--- file 或 chmod u=rw,go-rwx file)

- 设置所有用户可读取文件 a.conf

chmod ugo+r a.sh　或　chmod a+r a.conf

- 设置 c.sh 只有 拥有者可以读写及执行

```
chmod u+rwx c.sh
```

u 表示该档案的拥有者，g 表示与该档案的拥有者属于同一个群体(group)者，o 表示其他以外的人，a 表示所有（包含上面三者）。
[+-=]
+ 表示增加权限，- 表示取消权限，= 表示唯一设定权限。
[rwxX]
r 4 表示可读取，w2 表示可写入，x 1 表示可执行，X 表示只有当该档案是个子目录或者该

档案已经被设定过为可执行。

17、chown 命令改变文件所有者：

chown [para]... [owner][:[group]] file...

-c 显示更改的部分的信息
-f 忽略错误信息
-h 修复符号链接
-R 处理指定目录以及其子目录下的所有文件
-v 显示详细的处理信息
-deference 作用于符号链接的指向，而不是链接文件本身

chown -R -f -c root usr.txt

18、chgrp 命令，改变文件所属组：

-c 当发生改变时输出调试信息
-f 不显示错误信息
-R 处理指定目录以及其子目录下的所有文件
-v 运行时显示详细的处理信息
--dereference 作用于符号链接的指向，而不是符号链接本身
--no-dereference 作用于符号链接本身

chgrp -R -f -c root usr.txt

19、gzip 命令压缩文件或文件夹为 .gz 文件：

gzip[参数][文件或者目录]：gzip a.log

-a or --ascii 使用 ASCII 文字模式。
-c or --stdout or --to-stdout 把压缩后的文件输出到标准输出设备，不去更动原始文件。
-d or --decompress or ----uncompress 解开压缩文件。
-f or --force 强行压缩文件。不理会文件名称 or 硬连接是否存在以及该文件是否为符号连接。
-h or --help 在线帮助。
-l or --list 列出压缩文件的相关信息。
-L or --license 显示版本与版权信息。
-n or --no-name 压缩文件时，不保存原来的文件名称及时间戳记。
-N or --name 压缩文件时，保存原来的文件名称及时间戳记。
-q or --quiet 不显示警告信息。
-r or --recursive 递归处理，将指定目录下的所有文件及子目录一并处理。
-S<压缩字尾字符串> or ----suffix<压缩字尾字符串> 更改压缩字尾字符串。
-t or --test 测试压缩文件是否正确无误。
-v or --verbose 显示指令执行过程。
-V or --version 显示版本信息。
-num 用指定的数字 num 调整压缩的速度，-1 or --fast 表示最快压缩方法（低压缩比），-9 or --best 表示最慢压缩方法（高压缩比）。系统缺省值为 6。

20、vi/vim 是使用 vi 编辑器的命令

i 切换到输入模式，以输入字符。
x 删除当前光标所在处的字符。

: 切换到底线命令模式，以在最底一行输入命令
:q! 退出程序
:wq 保存文件
h 或 向左箭头键(←) 光标向左移动一个字符
j 或 向下箭头键(↓) 光标向下移动一个字符
k 或 向上箭头键(↑) 光标向上移动一个字符
l 或 向右箭头键(→) 光标向右移动一个字符
0 或功能键[Home] 这是数字『 0 』：移动到这一行的最前面字符处 (常用)
$ 或功能键[End] 移动到这一行的最后面字符处(常用)
H 光标移动到这个屏幕的最上方那一行的第一个字符
M 光标移动到这个屏幕的中央那一行的第一个字符
L 光标移动到这个屏幕的最下方那一行的第一个字符
G 移动到这个档案的最后一行(常用)
nG n 为数字。移动到这个档案的第 n 行。例如 20G 则会移动到这个档案的第 20 行(可配合 :set nu)
gg 移动到这个档案的第一行，相当于 1G 啊！ (常用)

21、gerp 用于查找文件里符合条件的字符串
ps –ef|grep java 查看 java 进程
grep -wn aaa text.txt 搜索 text.txt 中 aaa 的内容
如果需要显示搜索文本在文件中的行数，可以使用参数-n
如果搜索时需要忽略大小写问题，可以使用参数-i
搜索、查找匹配的行数，可以使用参数-c
如果只想获取和整个搜索字符匹配的内容，那么可以使用参数 w

22、rz 用于上传
先安装：yum install lrzsz
输入 rz，然后回车，弹出上传的窗口

23、sz 用于下载
sz install.log，下载文件到本地

24、sed 替换
s 表示替换，d 表示删除
示例：sed -i "s/shan/hua/g" lishan.txt
把当前目录下 lishan.txt 里的 shan 都替换为 hua

通过 vi 编辑器来替换。
vi/vim 中可以使用 :s 命令来替换字符串。
:s/well/good/ 替换当前行第一个 well 为 good
:s/well/good/g 替换当前行所有 well 为 good
:n,$s/well/good/ 替换第 n 行开始到最后一行中每一行的第一个 well 为 good
:n,$s/well/good/g 替换第 n 行开始到最后一行中每一行所有 well 为 good
n 为数字，若 n 为 .，表示从当前行开始到最后一行

25、awk 对于文件中一行行的独处来执行操作

awk -F ：'{print $1,$4}'　:使用‘:’来分割这一行，把这一行的第一第四个域打印出来，-F 域分隔符]是可选的，awk 默认使用空格分隔

26、group 组相关的命令

groupadd 组名　添加组
groupdel 组名　删除组
cat /etc/group　查看组信息

显示当前用户所在组
groups

查看当前系统所有的组
cat /etc/group

增加一个组
groupadd java

删除一个组
groupdel java

增加用户到组，-a：添加用户到组；
gpasswd -a zhangsan java

从组中删除用户，-d：从组删除用户
gpasswd -d zhangsan java

groupmod 命令用于修改用户组的相关信息，命令格式如下：
[root@localhost ~]# groupmod [选现] 组名

选项：
-g GID：修改组 ID；
-n 新组名：修改组名；

例子：
[root@localhost ~]# groupmod -n testgrp group1
#把组名 group1 修改为 testgrp
[root@localhost ~]# grep "testgrp" /etc/group
testgrp:x:502:
#注意 GID 还是 502，但是组名已经改变

六十七、python 小例子

1、例子 1




```
import random
c=[1,2,3,4,5]
b=['a','b']
g=random.randint(1,4)#随机生成 1,2,3,4 任意一个数
print('随机的一个数：',g)
for j in range(len(b)):
    g
    c.insert(g,b[j])#按随机数的下标来插入 b 的值
    g+=2#不能取连续的数，只能大于 2 的数，否则出现 a 和 b 挨着
print (c)
```

2、例子 2

从文本里读取 5 行数据，第三行有空行，按空行分割成两部分存在列表里

```
import io
with open("C:\\Users\\Desktop\\1.txt") as file:
    filetxt=file.read()
    a=filetxt.split('\n\n')
    b=[]
    for i in a:
        b.append(i.replace('\n',''))

    print(b)
```

结果：['ab','cde']

3、例子 3
c=[('a', 'b'), ('c', 'd'), ('e', 'f')]　　变成 a=b&c=d&e=f

```
e=''
for i in c:
    e=e+'&'+'='.join(i)

print(e[1:])
```

六十八、Python 字典和 JSON 格式区别

字典中的引号支持单引号和双引号，JSON 格式只支持双引号
字典中的 True/False 首字母大写，JSON 格式为 true/false
字典中的空值为 None, JSON 格式为 null

JSON 格式转换操作方法

字段转 json：序列化（字典 -> 文本/文件句柄）：json.dumps()/json.dump()
Json 转字段：反序列化（文本/文件句柄 -> 字典) : json.loads()/json.load()

json 格式传输需要遵循以下几点：
必须是严格的 JSON 格式字符串，里面必须用双引号，k-v 之间必须有逗号，布尔值必须是小写的 true/false 等等，不能有中文，直接传字符串不会自动编码

json.dumps(res_dict, indent=2, sort_keys=True, ensure_ascii=False)
indent: 缩进空格数，indent=0 输出为一行
sork_keys=True: 将 json 结果的 key 按 ascii 码排序
ensure_ascii=Fasle: 不确保 ascii 码，如果返回格式为 utf-8 包含中文，不转化为\u...
例子：

```
{"code":100000,"text":"111"}  # res.text，有些接口中文会返回为\u..
{
  "code": 100000,
  "text": "111"   #  树状格式，比较清晰，显示中文
}
```

六十九、HashTable 和 HashMap 区别

高并发面试题：https://blog.csdn.net/weixin_41050155/article/details/88047556
散列表（Hash table，也叫哈希表），是根据关键码值(Key value)而直接进行访问的数据结构。它通过把关键码值映射到表中一个位置来访问记录，以加快查找的速度。这个映射函数叫做

散列函数，存放记录的数组叫做散列表。
Hashmap 基于哈希表的 Map 接口的实现。此实现提供所有可选的映射操作，并允许使用 null 值和 null 键。(除了非同步和允许使用 null 之外，HashMap 类与 Hashtable 大致相同。)此类不保证映射的顺序，特别是它不保证该顺序恒久不变。
1、继承的父类不同
Hashtable 继承自 Dictionary 类，而 HashMap 继承自 AbstractMap 类。但二者都实现了 Map 接口。
2、线程安全性不同
hashMap 没有考虑同步，是线程不安全的。hashTable 是线程安全的，给 api 套上了一层 synchronized 修饰
javadoc 中关于 hashmap 的一段描述如下：此实现不是同步的。如果多个线程同时访问一个哈希映射，而其中至少一个线程从结构上修改了该映射，则它必须保持外部同步。ashtable 中的方法是 Synchronize 的，而 HashMap 中的方法在缺省情况下是非 Synchronize 的。在多线程并发的环境下，可以直接使用 Hashtable，不需要自己为它的方法实现同步，但使用 HashMap 时就必须要自己增加同步处理。Hashtable 线程安全很好理解，因为它每个方法中都加入了 Synchronize(使同步发生)。HashMap 为什么是线程不安全的：HashMap 底层是一个 Entry(进入)数组，当发生 hash 冲突的时候，hashmap 是采用链表的方式来解决的，在对应的数组位置存放链表的头结点。对链表而言，新加入的节点会从头结点加入。
3、key 和 value 是否允许 null 值
HashMap 允许 Key-value 为 null，hashTable 不允许
其中 key 和 value 都是对象，并且不能包含重复 key，但可以包含重复的 value。Hashtable 中，key 和 value 都不允许出现 null 值。但是如果在 Hashtable 中有类似 put(null,null)的操作，编译同样可以通过，因为 key 和 value 都是 Object 类型，但运行时会抛出 NullPointerException 异常，这是 JDK 的规范规定的。
HashMap 中，null 可以作为键，这样的键只有一个；可以有一个或多个键所对应的值为 null。当 get()方法返回 null 值时，可能是 HashMap 中没有该键，也可能使该键所对应的值为 null。因此，在 HashMap 中不能由 get()方法来判断 HashMap 中是否存在某个键， 而应该用 containsKey()方法来判断。
4、两个遍历方式的内部实现上不同
迭代器(Iterator)。HashMap 的迭代器(Iterator)是 fail-fast 迭代器，而 Hashtable 的 enumerator 迭代器不是 fail-fast 的。所以当有其它线程改变了 HashMap 的结构（增加或者移除元素），将会抛出 ConcurrentModificationException。
Hashtable、HashMap 都使用了 Iterator。而由于历史原因，Hashtable 还使用了 Enumeration 的方式。HashMap 的 Iterator 是 fail-fast 迭代器。当有其它线程改变了 HashMap 的结构（增加，删除，修改元素），将会抛出 ConcurrentModificationException。不过，通过 Iterator 的 remove()方法移除元素则不会抛出 ConcurrentModificationException 异常。但这并不是一个一定发生的行为，要看 JVM。
5、hash 值不同
添加 key-value 时的 hash 值算法不同：HashMap 添加元素时，是使用自定义的哈希算法。Hashtable 没有自定义哈希算法，而直接采用的 key 的 hashCode()
哈希值的使用不同，HashTable 直接使用对象的 hashCode。而 HashMap 重新计算 hash 值。hashCode 是 jdk 根据对象的地址或者字符串或者数字算出来的 int 类型的数值。Hashtable 计算 hash 值，直接用 key 的 hashCode()，而 HashMap 重新计算了 key 的 hash 值，Hashtable 在求 hash 值对应的位置索引时，用取模运算，而 HashMap 在求位置索引时，则用与运算，

且这里一般先用 hash&0x7FFFFFFF 后，再对 length 取模，&0x7FFFFFFF 的目的是为了将负的 hash 值转化为正值，因为 hash 值有可能为负数，而&0x7FFFFFFF 后，只有符号外改变，而后面的位都不变。

6、内部实现使用的数组初始化和扩容方式不同

容量的初始值和增加方式都不一样：HashMap 默认的容量大小是 16；增加容量时，每次将容量变为"原始容量 x2"。Hashtable 默认的容量大小是 11；增加容量时，每次将容量变为"原始容量 x2 + 1"

HashTable 在不指定容量的情况下的默认容量为 11，而 HashMap 为 16，Hashtable 不要求底层数组的容量一定要为 2 的整数次幂，而 HashMap 则要求一定为 2 的整数次幂。Hashtable 扩容时，将容量变为原来的 2 倍加 1，而 HashMap 扩容时，将容量变为原来的 2 倍。

Hashtable 和 HashMap 它们两个内部实现方式的数组的初始大小和扩容的方式。HashTable 中 hash 数组默认大小是 11，增加的方式是 old*2+1。

7、是否提供 contains 方法

HashMap 把 Hashtable 的 contains 方法去掉了，改成 containsValue 和 containsKey，因为 contains 方法容易让人引起误解。Hashtable 则保留了 contains，containsValue 和 containsKey 三个方法，其中 contains 和 containsValue 功能相同。

这两个类最大的不同之处在于：

HashTable 是线程安全的，它的方法是同步的，可以直接用于多线程环境中，不允许 Key-value 为 null

HashMap 是线程不安全的，在多线程环境中，需要手动实现同步机制，允许 Key-value 为 null

七十、Monkey 命令

monkey 是一个运行在您的 模拟器或设备上，并生成伪随机的用户事件流如点击， 触摸或手势， 以及一些系统级事件的程序。 您可以使用 monkey 以随机但可重复的方式对您正在开发的应用程序进行压力测试。

例子：

abd shell monley –s 123456 –ignore-crashes –ignore-timoouts –ignore-security-excptions –p com.qq.frodo –pct-touch 60 –tcp-motion 40 –throttle 1000 -v –v –v 100 >C:\Desk\log.txt

查看日志里的 pointers，如果 pointers=2 说明有两个事件失败

-s 123456 指随机生成的种子值，出问题时方便回放，这个值可以随便写

–ignore-crashes 指忽略异常，将继续执行

–ignore-timoouts 指忽略超时，将继续执行

–ignore-security-excptions 指忽略由权限等报的错误，将继续执行

-p 指要执行包时必须写的，com.qq.frodo 指 qq 的包名

-throttle 1000 指每次事件延时 1000 毫秒也就是 1 秒，100 指 100 的随机事件，但并不是真的要执行 100 秒，实际也就 20 多秒

三个-v 指最详细的日志，一个-v 只是一点日志

--pct-touch 60 指 60%的触摸

--tcp-motion 40 指 40%的滑动事件

事件	解释
-s <seed>	伪随机数发生器的种子值。如果用相同的种子值重新运行monkey，它将生成相同的事件序列。
--throttle <milliseconds>	在事件之间插入一个固定的延迟。你可以使用这个选项来减慢monkey的速度。如果没有指定，则不会有任何延迟，事件会尽快生成。
--pct-touch <percent>	调整触摸事件的百分比。（触摸事件是一个down-up事件，她发生在屏幕的某一个位置）
--pct-motion <percent>	调整运动事件的百分比。运动事件由屏幕某处的down事件，一系列伪随机运动和up事件组成。滑动事件
--pct-trackball <percent>	调整轨迹球事件的百分比。（轨迹球事件包含一个或多个随机移动，有时还伴随着点击。）轨迹球事件
--pct-nav <percent>	调整“基本”导航事件的百分比。（导航事件由上/下/左/右，作为来自定向输入设备的输入。）

事件	解释
--pct-majornav <percent>	调整“主要”导航事件的百分比。（例如后退键或菜单键。）
--pct-syskeys <percent>	调整“系统”键事件的百分比。（这些是通常保留供系统使用的操作，如主页，返回，开始呼叫，结束呼叫或音量控制。）
--pct-appswitch <percent>	调整activity启动的百分比。在随机的时间间隔内，Monkey会发出一个startActivity（）调用，以此来最大限度地覆盖包中的所有activity。切换activity
--pct-anyevent <percent>	调整其他类型事件的百分比。对于所有其他类型的事件，如按键，设备上其他使用较少的按钮等等，这是一个全面的方法。任意事件

约束	解释
-p <allowed-package-name>	如果以这种方式指定一个或多个软件包，Monkey将只允许系统访问这些软件包中的活动。如果您的应用程序需要访问其他包中的活动（例如选择联系人），则还需要指定这些包。如果您不指定任何软件包，Monkey将允许系统在所有软件包中启动活动。要指定多个软件包，请多次使用-p选项 - 一个软件包一个-p选项。
-c <main-category>	如果您以这种方式指定一个或多个类别，Monkey将只允许系统访问使用指定类别之一列出的活动。如果您没有指定任何类别，Monkey将选择列出的类别为Intent.CATEGORY_LAUNCHER或Intent.CATEGORY_MONKEY的活动。要指定多个类别，请多次使用-c选项 - 一个类别一个-c选项。

调试	解释
--dbg-no-events	当指定时，Monkey将执行初始启动到一个测试活动，但不会产生任何进一步的事件。为了获得最佳效果，结合使用-v，一个或多个包限制，以及一个非零的油门来保持猴子运行30秒或更长时间。这提供了一个可以监视应用程序调用的程序包转换的环境。
--hprof	如果设置，则此选项将在Monkey事件序列之前和之后立即生成分析报告。这会在数据/杂项中生成大的（～5Mb）文件，所以请谨慎使用
--ignore-crashes	通常，当应用程序崩溃或遇到任何类型的未处理的异常时，monkey将停止。如果你指定了这个选项，Monkey将继续向系统发送事件，直到计数完成。
--ignore-timeouts	通常，当应用程序遇到任何类型的超时错误（如“应用程序未响应”对话框）时，Monkey将停止。如果你指定了这个选项，Monkey将继续向系统发送事件，直到计数完成。

调试	解释
--ignore-security-exceptions	通常，当应用程序遇到任何类型的权限错误时，monkey将停止，例如，如果它尝试启动需要某些权限的活动。如果你指定了这个选项，Monkey将继续向系统发送事件，直到计数完成。
--kill-process-after-error	通常，当monkey因错误而停止时，失败的应用程序将保持运行。当这个选项被设置时，它会通知系统停止发生错误的过程。请注意，在正常（成功）完成的情况下，启动的进程不会停止，并且在最终事件之后，设备仍然处于最后一个状态。
--monitor-native-crashes	监视并报告Android系统本机代码中发生的崩溃。如果错误设置了--kill-process-after，系统将停止。
--wait-dbg	停止执行Monkey直到调试器被连接到它。

七十一、LoadRunner 常用函数

地址：https://blog.csdn.net/huangjin0507/article/details/52152285

1、web_reg_save_param 关联函数("参数名”,"LB=左边界”,"RB=右边界","Ord=All",LAST) /当参数有多个值时，加上"Ord=All”后可获取所有的数值。注册成功后，{参数名_count}表示取得的数值个数，{参数名_1}为第一个数值，{参数名_2}为第二个数值。

2、lr_save_string(“字符串变量”,"参数名")/将字符变量里的值传递给指定参数。通过该函数来改变 DataFile 类型参数的数值。

3、lr_message("str: %s",str);

4、web_find()增加检查点

事务函数

5、lr_start_transaction/标记 LoadRunner 事务的开始

6、lr_end_transaction/标记 LoadRunner 事务的结束

7、lr_stop_transaction/停止事务数据的收集

lr_get_transaction_duration/获取事务的持续时间（按事务的名称）

lr_get_transaction_think_time/获取事务的思考时间（按事务的名称）

lr_get_transaction_wasted_time/获取事务浪费的时间（按事务的名称）

lr_think_time 思考时间,用户在执行连续操作之间等待的时间

命令行分析函数

8、lr_get_attrib_double/检索脚本命令行中使用的 double 类型变量

9、lr_get_attrib_long/检索脚本命令行中使用的 long 类型变量

10、lr_get_attrib_string/检索脚本命令行中使用的字符串

字符串函数

11、lr_eval_string/用参数的当前值替换参数

例如：lr_output_message(lr_eval_string("{parm}"));

12、lr_save_string/将以 NULL 结尾的字符串保存到参数中

13、lr_save_var/将变长字符串保存到参数中

14、lr_save_datetime/将当前日期和时间保存到参数中

15、web_save_timestamp_param("tStamp", LAST);//获取时间戳，并存到 tStamp 这个参数中备用

消息函数

16、lr_debug_message/将调试消息发送到输出窗口

17、lr_error_message/将错误消息发送到输出窗口

18、lr_get_debug_message/检索当前的消息类

19、lr_log_message/将输出消息直接发送到 output.txt 文件，此文件位于 Vuser 脚本目录中. 该函数有助于防止输出消息干扰 TCP/IP 通信。

20、lr_output_message/将消息发送到输出窗口

21、lr_set_debug_message/为输出消息设置消息类

22、lr_vuser_status_message/生成格式化输出并将其打印到 ControllerVuser 状态区域。

23、lr_message/将消息发送到 Vuser 日志和输出窗口

操作函数

24、web_custom_request 允许您使用 HTTP 支持的任何方法来创建自定义 HTTP 请求

25、web_image 在定义的图像上模拟鼠标单击

26、web_link 在定义的文本链接上模拟鼠标单击

27、web_submit_data 执行“无条件”或“无上下文”的表单

28、web_submit_form 模拟表单的提交

29、web_url 加载由“URL”属性指定的 URL

缓存函数

30、缓存函数 web_cache_cleanup 清除缓存模拟程序的内容

检查函数

31、web_find 在 HTML 页内搜索指定的文本字符串

32、web_global_verification 在所有后面的 HTTP 请求中搜索文本字符串

33、web_image_check 验证指定的图像是否存在于 HTML 页内

34、web_reg_find 在后面的 HTTP 请求中注册对 HTML 源或原始缓冲区中文本字符串的搜索

并发组

35、web_concurrent_start 标记并发组的开始

36、web_concurrent_end 标记并发组的结束

cook 函数

37、web_add_cookie 添加新的 Cookie 或修改现有的 Cookie

38、web_cleanup_cookies 删除当前由 Vuser 存储的所有 Cookie

39、web_remove_cookie 删除指定的 Cookie

七十二、手机专项测试

1、查看手机设备内存

adb shell cat /proc/meminfo

MemTotal：可以使用的内存总和（小于实际RAM，操作系统预留了一部分）
MemFree：未使用的内存
Cached：缓存（这个也是app可以申请到的内存）
HightTotal：内存中地址高于860M的物理内存总和，只能被用户空间的程序使用。
HightFree：内存中地址高于860M的未使用内存
LowTotal：内存中内核和用户空间程序都可以使用的内存总和（对于512M的RAM: lowTotal= MemTotal）
LowFree: 内存中内核和用户空间程序未使用的内存（对于512M的RAM: lowFree = MemFree）

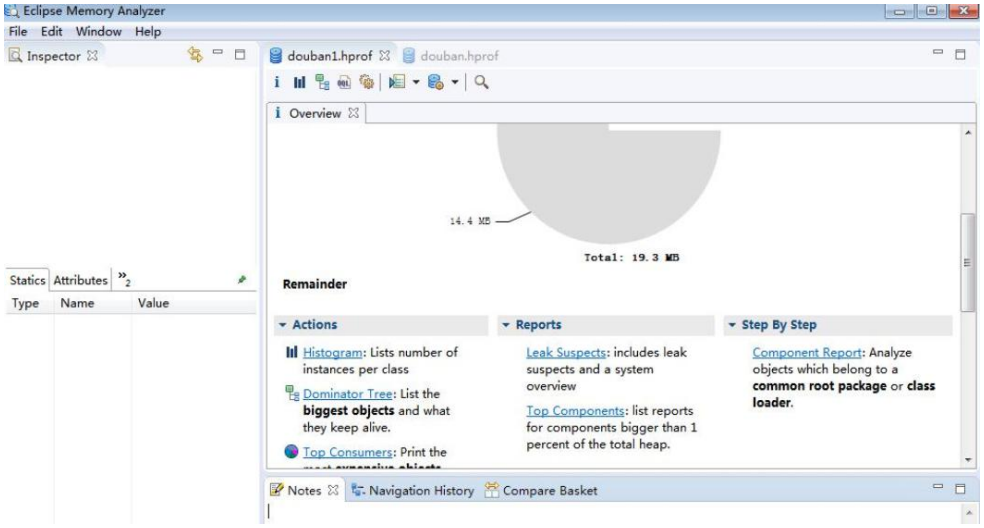
查看应用软件占用内存
adb shell dumpsys meminfo com.qq.frodo

2、手机内存采集

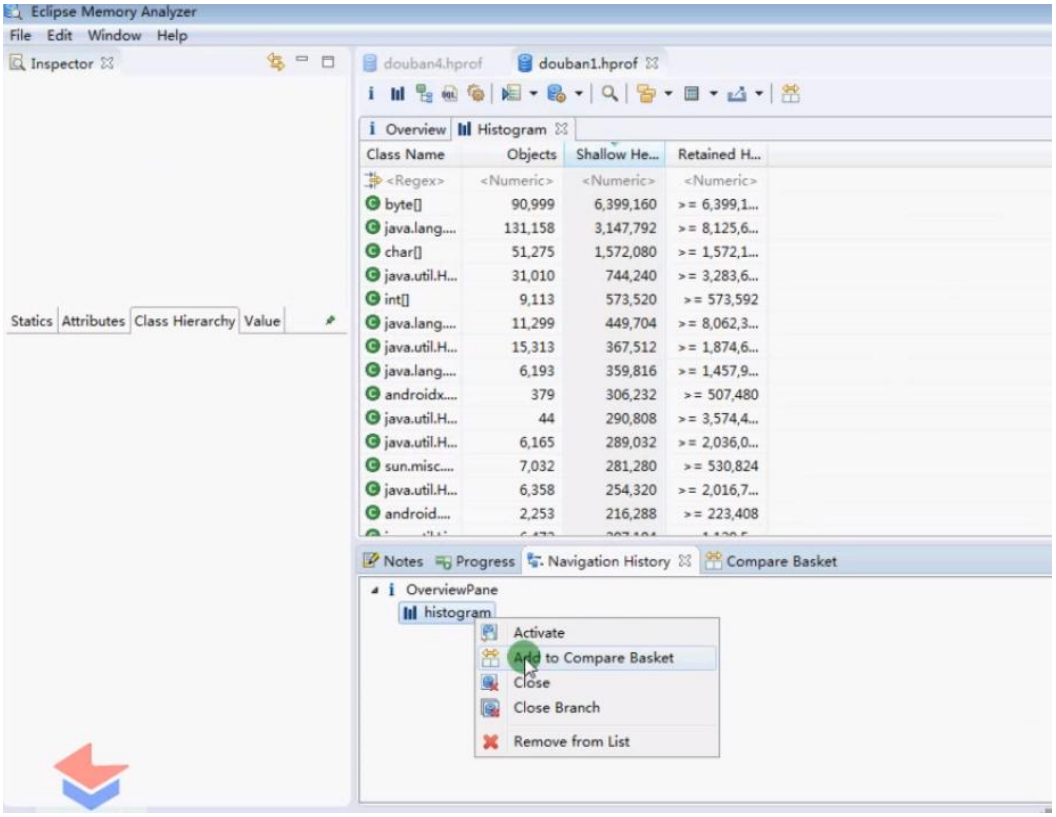
在 sdk 安装目录的 tools 文件夹里，monitor.bat 文件，先选择左侧的应用包名，点击上面的 update heap 绿色按钮，在点右侧标签 Heap 查看数据，操作 app，点击绿色下载箭头的图标，采集后生成*.hprof 文件

3、内存数据分析 MAT(Memory Analyzer Tool)

下载 memoryanalyzer 工具，在 sdk 安装目录的 platform-tools 里的 hprof-conv.exe 文件，先配置环境变量，进入 cmd 界面的*.hprof 文件路径下，输入 hprof-conv *.hprof 1.hprof 命令，用 memoryanalyzer 工具打开 1.hprof 文件



增加对比文件：先点击历史数据 histogram，点击标签 navigation history，选中 histogram，右键点击 add to compare basket，数据就会存到右边的 compare basket 里，在新增一个之前的*.hprof 文件，按步骤在增加到 compare basket 里，点击最右边的红色叹号就可以对比了。



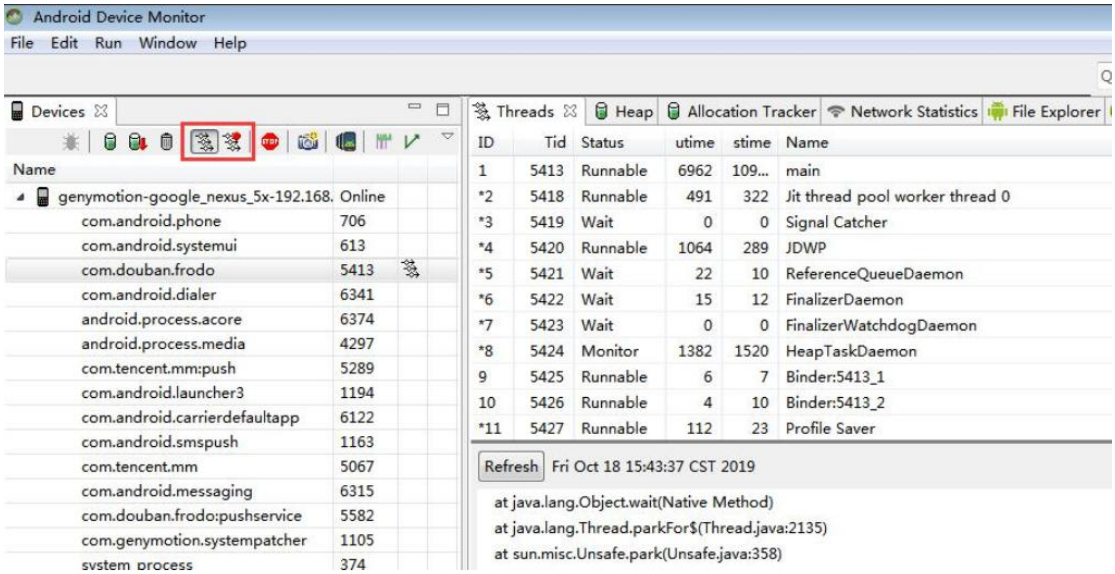
4、查看手机设备 cpu

adb shell dumpsys cpuinfo

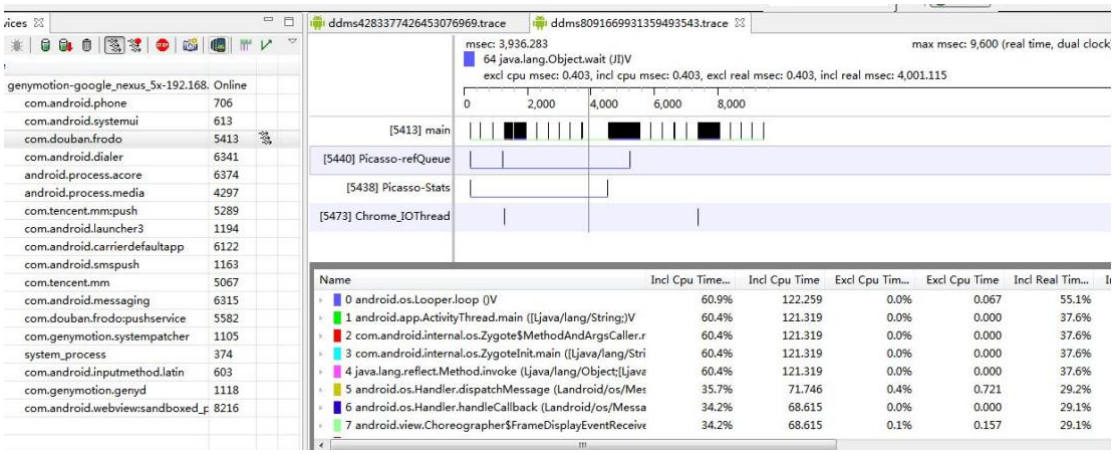
查看某应用软件占用 cpu，这个不显示字段标题是啥，用 adb shell “top”也可以
adb shell “top –n 1|grep com.qq.frodo”

5、手机 cpu 采集

在 sdk 安装目录的 tools 文件夹里，monitor.bat 文件，先选择左侧的应用包名，点击上面的 start method profiling 按钮，操作 app，点击 stop method profiling 按钮，出现黑色条纹，代表函数调用，点击黑框显示相关函数，红色图标表示时间长

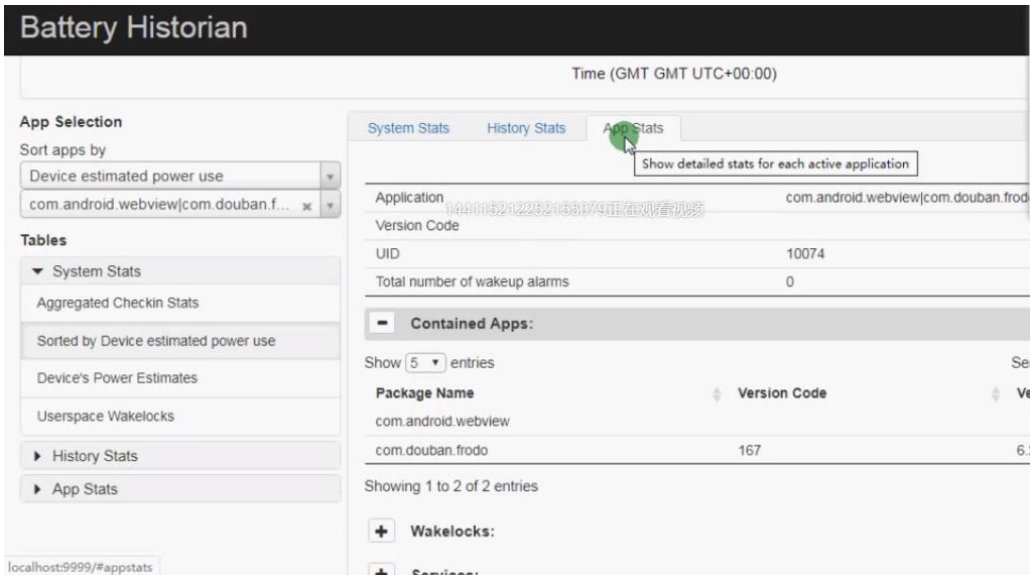


出现黑色条纹，代表函数调用，点击黑框显示相关函数，红色图标表示时间长



6、电量分析工具

battery historian，Google 开源的分析设备电量用量情况的测试工具，go 语言开发的工具，适用于 android5.0 以后的版本，需要安装 go 语言，git 工具



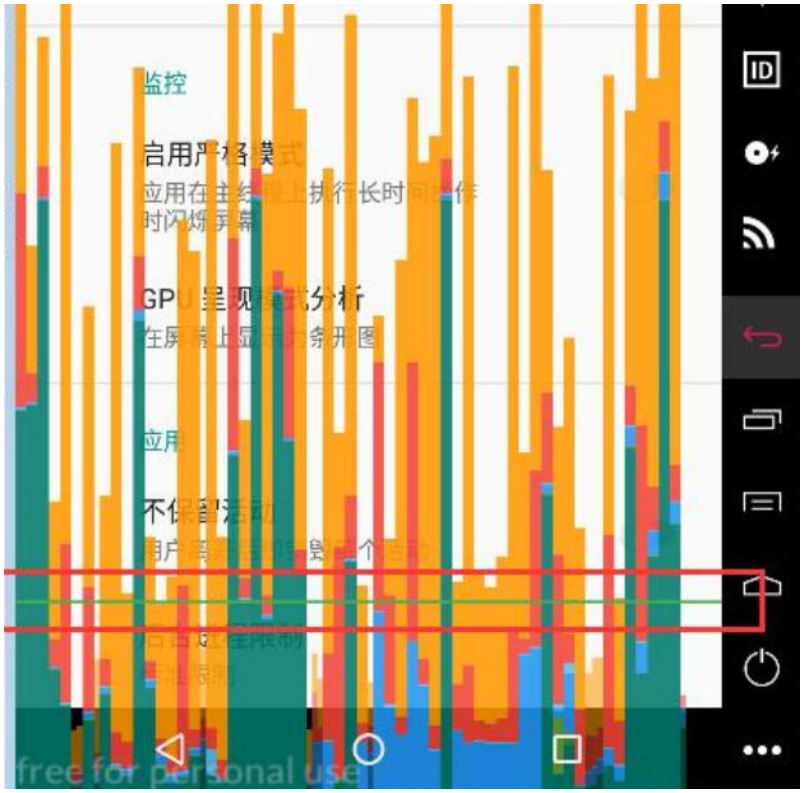
7、GPU 渲染之流畅度

流畅度（SM：smoothness）指每秒 60 帧的速度运行（每帧是 16.6ms(毫秒)），就是 60fps。和丢帧相对，在 VSync 机制中 1s 内 Loop 运行的次数

丢帧（SF：skipped frame）指在 16.6ms 内完成的工作没有做完，占用了后面 n 个 1.6ms 的时间，相当于丢了 n 帧。

VSync 机制：android 系统每隔 16ms 发出 VSync 信号，触发对 UI 进行渲染，VSync 是 Vertical Synchronization(垂直同步)的缩写。

打开手机开发者模式，打开“GPU 呈现模式分析”开关，选择“在屏幕上显示为条形图”即可。



这条绿色的横线是 16.6ms 的分界线，超过这个线说明有丢帧的地方

竖条区段	渲染阶段	说明
	交换缓冲区	表示 CPU 等待 GPU 完成其工作的时间。如果此竖条升高，则表示应用在 GPU 上执行太多工作。
	发出命令	表示 Android 的 2D 渲染程序向 OpenGL 发出绘制和重新绘制显示列表的命令所花的时间。此竖条的高度与执行每个显示列表所花的时间的总和成正比。显示列表越多，红色竖条就越高。
	同步和上传	表示将位图信息上传到 GPU 所花的时间。大区段表示应用花费大量的时间加载大量图形。
	绘制	表示用于创建和更新视图显示列表的时间。如果竖条的此部分很高，表明可能有许多自定义视图绘制，或 onDraw 方法执行的工作很多。
	测量/布局	表示在视图层次结构中的 onLayout 和 onMeasure 回调上所花的时间。大区段表示处理视图层次结构需要很长时间。
	动画	表示评估运行该帧的所有动画程序所花的时间。如果此区段很大，则表示您的应用可能在使用性能欠佳的自定义动画程序，或因更新属性而导致一些意料之外的工作。
	输入处理	表示应用执行输入事件回调中的代码所花的时间。如果此区段很大，则表示应用花太多时间处理用户输入。不妨考虑将此类处理任务分流到其他线程。
	其他时间/VSync 延迟	表示应用执行两个连续帧之间的操作所花的时间。它可能表示界面线程中进行的处理太多，而这些处理任务本可以分流到其他线程。

8、GPU 之过度绘制

打开开发者选项，打开“调试 GPU 过度绘制”开关，选择“显示过度绘制区域”。

Android 将按如下方式为界面元素着色，以确定过度绘制的次数：

- **真彩色**：没有过度绘制
- **蓝色**：过度绘制 1 次
- **绿色**：过度绘制 2 次
- **粉色**：过度绘制 3 次
- **红色**：过度绘制 4 次或更多

解决方法：

1、删除布局中不需要的背景

2、展平视图层次结构

3、降低透明度

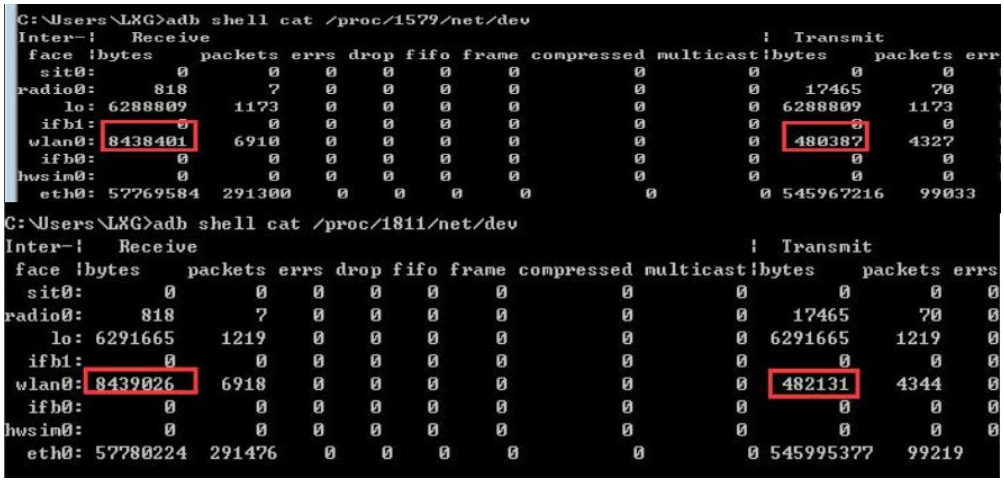
9、流量测试

方法一：获取 app 的 pid，查询“com.qq.frodo”的 pid

adb shell “ps|findstr com.qq.frodo”　不加引号也行，可能会出现两个 pid 的情况

通过 pid 获取流量

adb shell cat /proc/1111/net/dev　如果有两个 pid，执行两次，两次结果相加，看 wlan0 的值，receive 指接收的数据，transmit 指发送的数据，单位是字节，8bit(位)=1Byte(字节) 1024Byte(字节)=1KB，电脑里一个中文字是占两个字节。先第一次获取一个初值，然后操作 app，在获取值，后值减去前值得出结果。



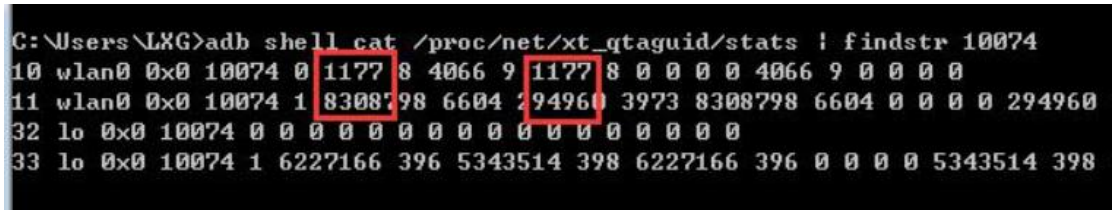
方法二：通过 userid 获取流量，查询“com.qq.frodo”的 userid

abd shell dumpsys package com.qq.frodo |findstr userid

通过 userid 获取流量，findstr 代表筛选

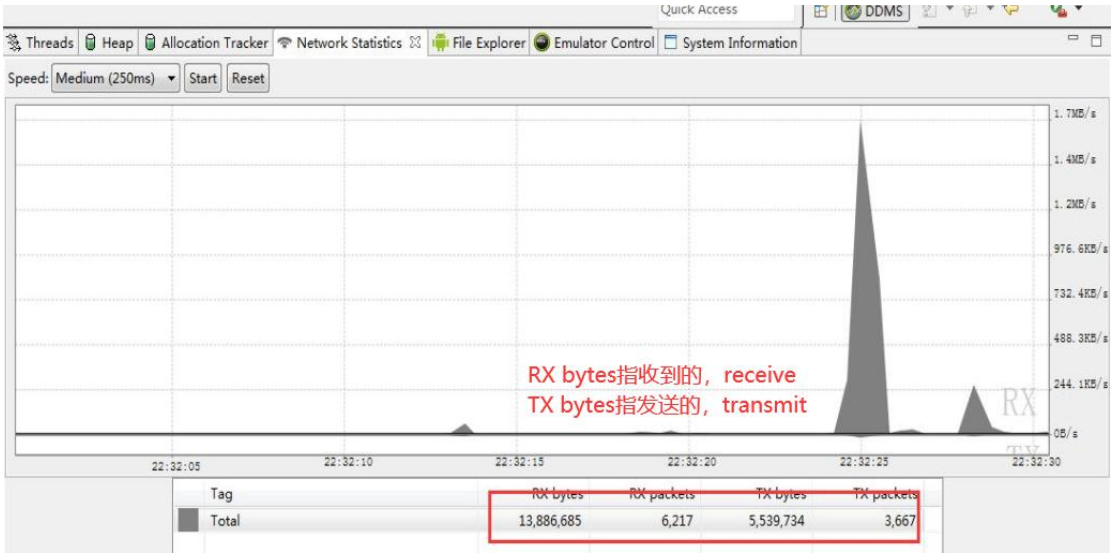
adb shell cat /proc/net/xy_qtaguid/stats |findstr 10074

看 wlan0 的值，第六列代表下载，第八列代表上传，单位是字节，查出两值相加是这次操作的结果，先第一次获取一个初值，然后操作 app，在获取值，后值减去前值得出结果。



方法三：用 monitor 工具查看流量

先左侧点击测试的应用包名，右侧点击 Speed 的采集速度随便选择一个值 fast(100ms)，点击 start 按钮开始采集，RX bytes(receive)指收到(下载)的数据，TX bytes(transmit)指发送(上传)的数据，单位是字节，常规操作 app，发现上传的数据变化比较大，收到的数据变化较小(不一定都是这样)



10、弱网测试

弱网测试的目的就是尽可能保证用户体验，关注的关键点包括:
(1)页面响应时间是否可接受，关注包括热启动、冷启动时间，页面切换，前后台切换，首字时间，首屏时间等。
(2)页面呈现是否完整一-致
(3)超时文案是否符合定义，异常信息是否显示正常。
(4)是否会有超时重连.
(5)安全角度:是否会发生 dns 劫持、登录 ip 更换频繁、单点登录异常等。
(6)大流量事件风险:是否会在弱网下进行更新 apk 包、下载文件等大流量

fiddler 主要是使用 Rules->Performance- >Simulate Modem Speeds 功能进行的网络延迟模拟，点击 Rules->Customize Rules 进行设置，打开自定义脚本编辑器，搜索 Delay 关键字，默认是延时 300ms 毫秒，保存后把 Simulate Modem Speeds 功能勾选上即可

11、常用手机性能测试软件

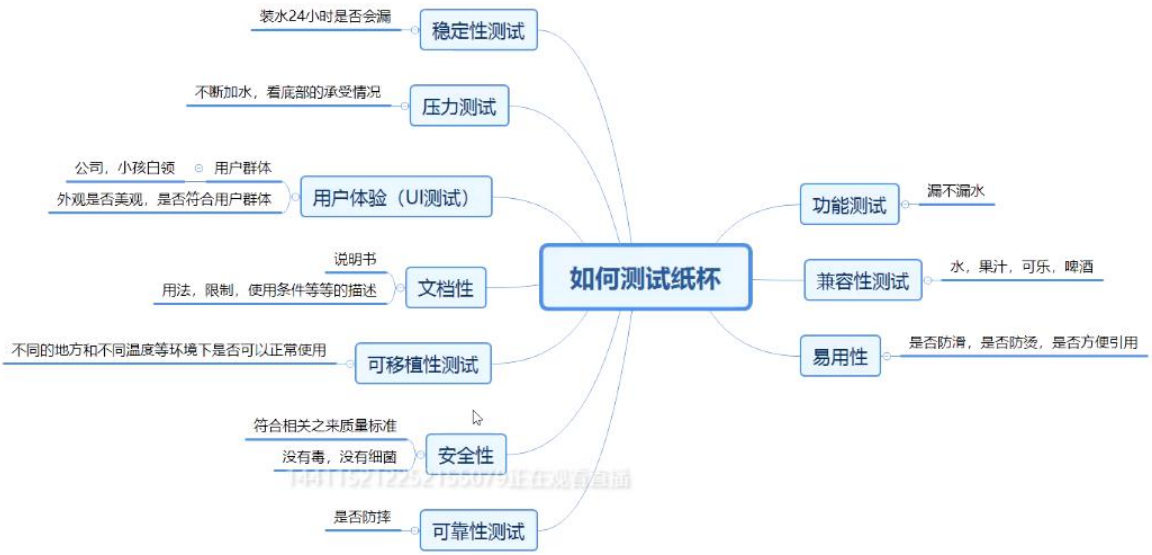
腾讯的 GT，现在已不跟新了，网易的 Emmagee，安卓 7 以上用不了，阿里的 solopi 可以用，solopi 学习地址：https://blog.csdn.net/q_catherine/article/details/94433828



2019 年腾讯新出的 PerfDog 移动性能测试工具，需要官网注册登录账号


PerfDog(v3.0.9153)
SceneTab
Coolpad A8-930
手机淘宝
Device
Setting
About
Version:
v3.0.9153
Help:
FPS
CPU Usage
Memory Usage
Total
App
Memory
SwapMemory
VirtualMemory
Log
https://blog.csdn.net/asstsui

七十三、纸杯测试用例


如何测试纸杯
稳定性测试
装水24小时是否会漏
压力测试
不断加水，看底部的承受情况
用户体验（UI测试）
公司，小孩日领
用户群体
外观是否美观，是否符合用户群体
文档性
说明书
用法，限制，使用条件等等的描述
可移植性测试
不同的地方和不同温度等环境下是否可以正常使用
安全性
符合相关之来质量标准
没有毒，没有细菌
可靠性测试
是否防摔
功能测试
漏不漏水
兼容性测试
水，果汁，可乐，啤酒
易用性
是否防滑，是否防烫，是否方便引用

七十四、Mysql 数据库

1.ORDER BY 要写在 LIMIT 前
例子 SELECT * FROM app ORDER BY id DESC LIMIT 1,10;
select * from 简历表 where age<30 and degree='硕士' order by age desc limit 10

mysql> SELECT * FROM table LIMIT 5,10; // 检索记录行 6-15

//为了检索从某一个偏移量到记录集的结束所有的记录行，可以指定第二个参数为 -1：
mysql> SELECT * FROM table LIMIT 95,-1; // 检索记录行 96-last.

//如果只给定一个参数，它表示返回最大的记录行数目：
mysql> SELECT * FROM table LIMIT 5; //检索前 5 个记录行

//换句话说，LIMIT n 等价于 LIMIT 0,n。

关键字：left join on / left outer join on，关键字：right join on / right outer join on

语句：SELECT * FROM a_table a left join b_table b ON a.a_id = b.b_id;

请查出 2006-6-1 到 2006-7-1 之间举行的所有比赛
地址：https://blog.csdn.net/zjt980452483/article/details/82945663

```
SELECT t1.tname,t1.mres,t2.tname,t1.time FROM
(SELECT m.mid,t.tname,m.mres,m.matime as time FROM m LEFT JOIN t ON t.tid = m.hid)
as t1
LEFT JOIN
(SELECT m.mid,t.tname,m.mres,m.matime as time FROM m LEFT JOIN t ON t.tid = m.gid)
as t2
ON t1.mid = t2.mid WHERE t1.time BETWEEN '2006-06-01' AND '2006-07-01';
```

Group by 配合聚合函数
聚合函数进行操作了。如执行 select name,sum(id) from test group by name,number

删除语句
1、drop table tb
drop 将表格直接删除，没有办法找回
2、truncate (table) tb
删除表中的所有数据，不能与 where 一起使用
3、delete from tb (where)
删除表中的数据(可制定某一行)

表字段的增、删、改、查
增：alter table 表名 add 字段名 数据类型 【位置】;
删：alter table 表名 drop 字段名；
改：alter table 表名 modify 字段 数据类型 【位置】
重命名：alter table 表名 change oldname newname 数据类型 【位置】

数据的增、删、改、查
增：insert into 表名（例 1，例 2，.....）values (v1,v2,.....);
删：delete from 表名 where 条件；
改：update 表名 set 字段=‘值’【where 条件】;
查：select */字段列表 from 表名 【where 条件】;

1.需要数据库表 1.学生表
Student(SID,Sname,Sage,Ssex) --SID 学生编号,Sname 学生姓名,Sage 出生年月,Ssex 学生性别
2.课程表
Course(CID,Cname,TID) --CID --课程编号,Cname 课程名称,TID 教师编号
3.教师表
Teacher(TID,Tname) --TID 教师编号,Tname 教师姓名
4.成绩表
SC(SID,CID,score) --SID 学生编号,CID 课程编号,score 分数

```
添加测试数据1.学生表
create table Student(SID varchar(10),Sname nvarchar(10),Sage datetime,Ssex nvarchar(10));
insert into Student values('01' , '赵雷' , '1990-01-01' , '男');
insert into Student values('02' , '钱电' , '1990-12-21' , '男');
insert into Student values('03' , '孙风' , '1990-05-20' , '男');
insert into Student values('04' , '李云' , '1990-08-06' , '男');
insert into Student values('05' , '周梅' , '1991-12-01' , '女');
insert into Student values('06' , '吴兰' , '1992-03-01' , '女');
insert into Student values('07' , '郑竹' , '1989-07-01' , '女');
insert into Student values('08' , '王菊' , '1990-01-20' , '女');

2.课程表
create table Course(CID varchar(10),Cname nvarchar(10),TID varchar(10));
insert into Course values('01' , '语文' , '02');
insert into Course values('02' , '数学' , '01');
insert into Course values('03' , '英语' , '03');

3.教师表
create table Teacher(TID varchar(10),Tname nvarchar(10));
insert into Teacher values('01' , '张三');
insert into Teacher values('02' , '李四');
insert into Teacher values('03' , '王五');

4.成绩表
create table SC(SID varchar(10),CID varchar(10),score decimal(18,1));
insert into SC values('01' , '01' , 80);
insert into SC values('01' , '02' , 90);
insert into SC values('01' , '03' , 99);
insert into SC values('02' , '01' , 70);
insert into SC values('02' , '02' , 60);
insert into SC values('02' , '03' , 80);
```

1、查询同时存在"01"课程和"02"课程的情况

```
select a.* , b.score 课程 01 的分数,c.score 课程 02 的分数 from Student a , SC b , SC c
where a.SID = b.SID and a.SID = c.SID and b.CID = '01' and c.CID = '02' and b.score > c.score
```

2、查询同时存在"01"课程和"02"课程的情况

```
select a.* , b.score 课程 01 的分数 ,c.score 课程 02 的分数 from Student a , SC b , SC c
where a.SID = b.SID and a.SID = c.SID and b.CID = '01' and c.CID = '02' and b.score < c.score
```

3、查询平均成绩大于等于 60 分的同学的学生编号和学生姓名和平均成绩

```
select a.SID , a.Sname , cast(avg(b.score) as decimal(18,2)) avg_score
from Student a , sc b where a.SID = b.SID group by a.SID , a.Sname having cast(avg(b.score) as decimal(18,2)) >= 60 order by a.SID
```

4、查询所有有成绩的 SQL。

```
select a.SID 学生编号 , a.Sname 学生姓名 , count(b.CID) 选课总数, sum(score) 所有课程的总成绩
from Student a , SC b where a.SID = b.SID group by a.SID,a.Sname order by a.SID
```

5、查询"李"姓老师的数量

```
select count(Tname) 李姓老师的数量 from Teacher where Tname like '李%'
```

6、查询学过"张三"老师授课的同学的信息

```
select distinct Student.* from Student , SC , Course , Teacher
where Student.SID = SC.SID and SC.CID = Course.CID and Course.TID = Teacher.TID and Teacher.Tname = '张三' order by Student.SID
```

7、检索"01"课程分数小于 60，按分数降序排列的学生信息

```
select student.* , sc.CID , sc.score from student , sc where student.SID = SC.SID and sc.score < 60 and sc.CID = '01' order by sc.score desc
```

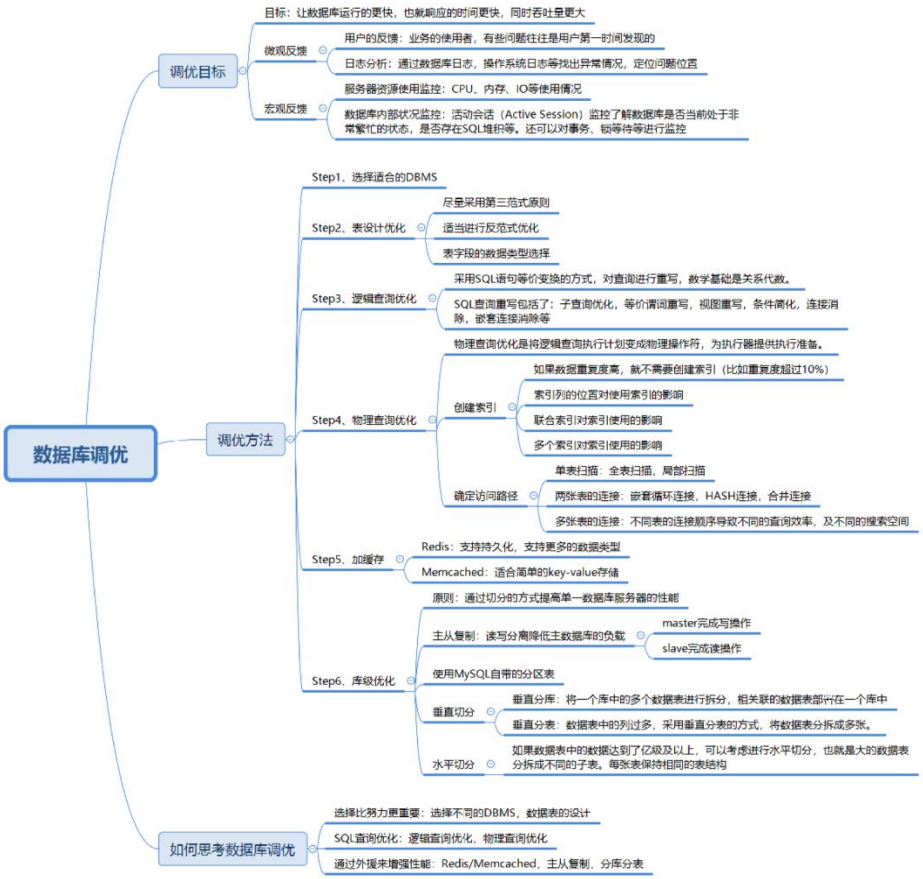
8、查询课程名称为"数学"，且分数低于 60 的学生姓名和分数

```
select sname , score from Student , SC , Course where SC.SID = Student.SID and SC.CID = Course.CID and Course.Cname = N'数学' and score < 60
```

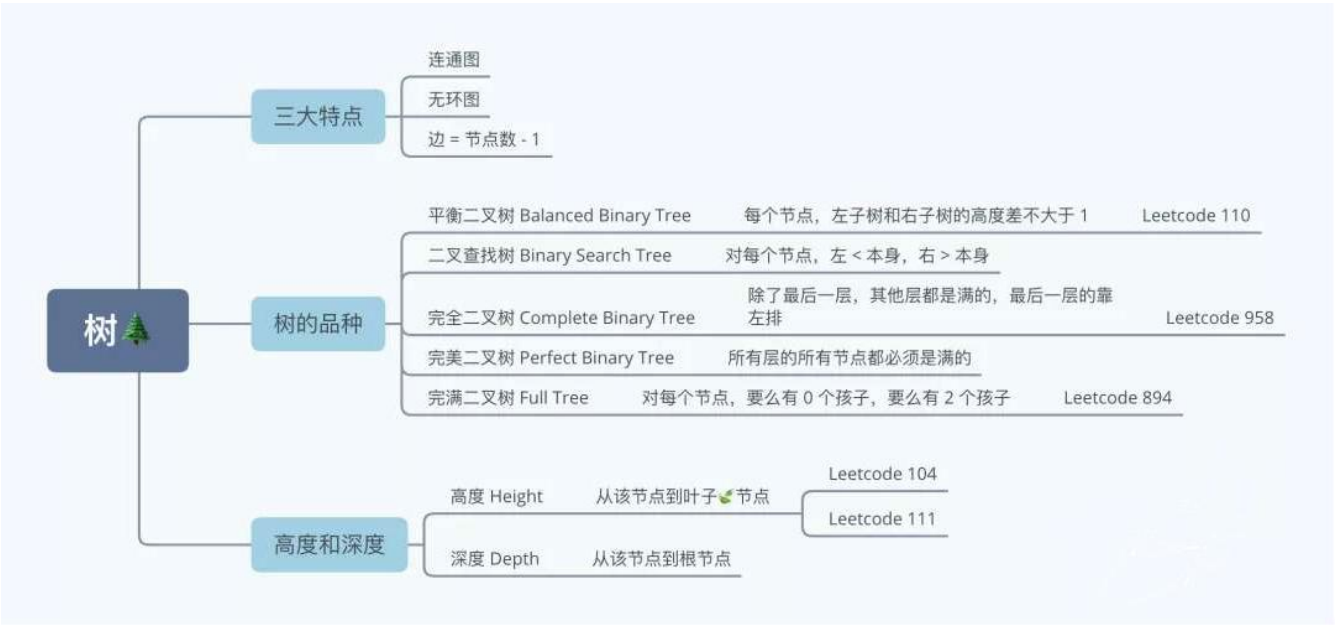
9、查询学生表前十行的数据

```
select * from Student limit 10
```

七十五、数据库调优



七十六、二叉树



文章地址：https://mp.weixin.qq.com/s/JDehhchwjXIpKfh5EP9F_g

1、二叉树的概念：一棵二叉树是节点的一个有限集合，该集合或者为空，或者由一个根节点加上两棵左子树和右子树组成

2、二叉树是树的特殊一种，具有如下特点：

1、每个结点最多有两颗子树，结点的度最大为 2。

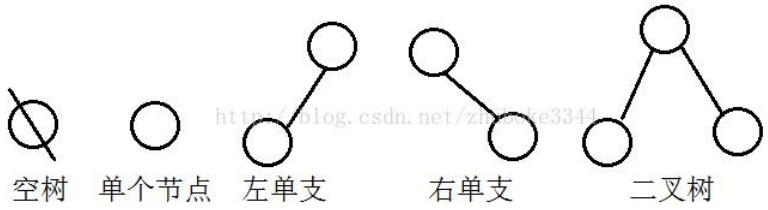
2、左子树和右子树是有顺序的，次序不能颠倒。

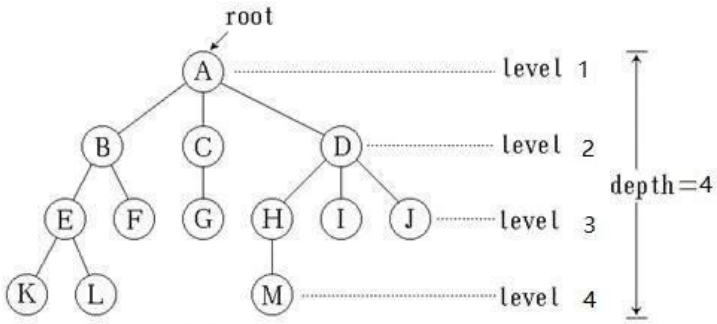
3、即使某结点只有一个子树，也要区分左右子树。

3、二叉树的类型：

二叉树、完全二叉树、满二叉树、线索二叉树、霍夫曼树、二叉排序树、平衡二叉树、红黑树、B 树等

4、二叉树的形式





5、满二叉树

满二叉树：在一棵二叉树中。如果所有分支结点都存在左子树和右子树，并且所有叶子都在同一层上，这样的二叉树称为满二叉树。

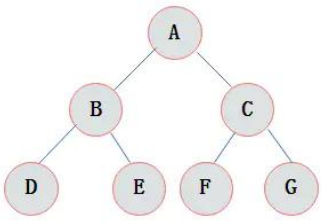
空树：不包含任何节点的二叉树为空树，只有一个节点的二叉树称为单点树，一个节点的子节点的个数称为该节点的度。如果每个分支节点的度都为 2，则称之为满二叉树。T4、T5 就是两棵满二叉树

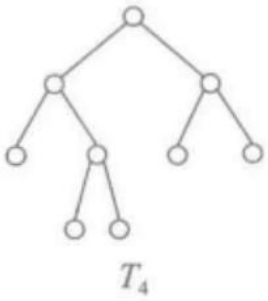
满二叉树的特点有：

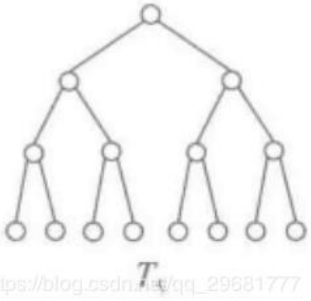
1）叶子只能出现在最下一层。出现在其它层就不可能达成平衡。

2）非叶子结点的度一定是 2。

3）在同样深度的二叉树中，满二叉树的结点个数最多，叶子数最多。

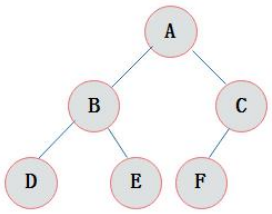


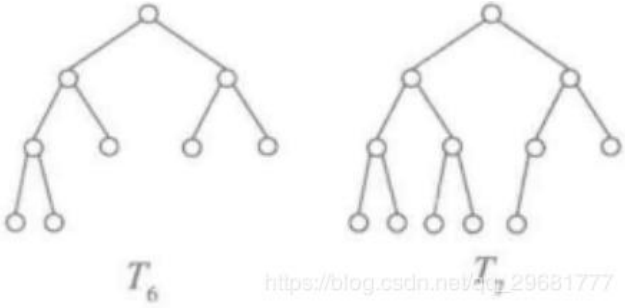




6、完全二叉树

完全二叉树：对一颗具有 n 个结点的二叉树按层编号，如果编号为 i(1<=i<=n)的结点与同样深度的满二叉树中编号为 i 的结点在二叉树中位置完全相同，则这棵二叉树称为完全二叉树。如果一棵二叉树，除最后一层外，其它层的节点都是满的，而最后一层节点在最左边连续排列，空位都在右边，这样的二叉树叫做完全二叉树，如 T6、T7 所示





特点：

1）叶子结点只能出现在最下层和次下层。

2）最下层的叶子结点集中在树的左部。

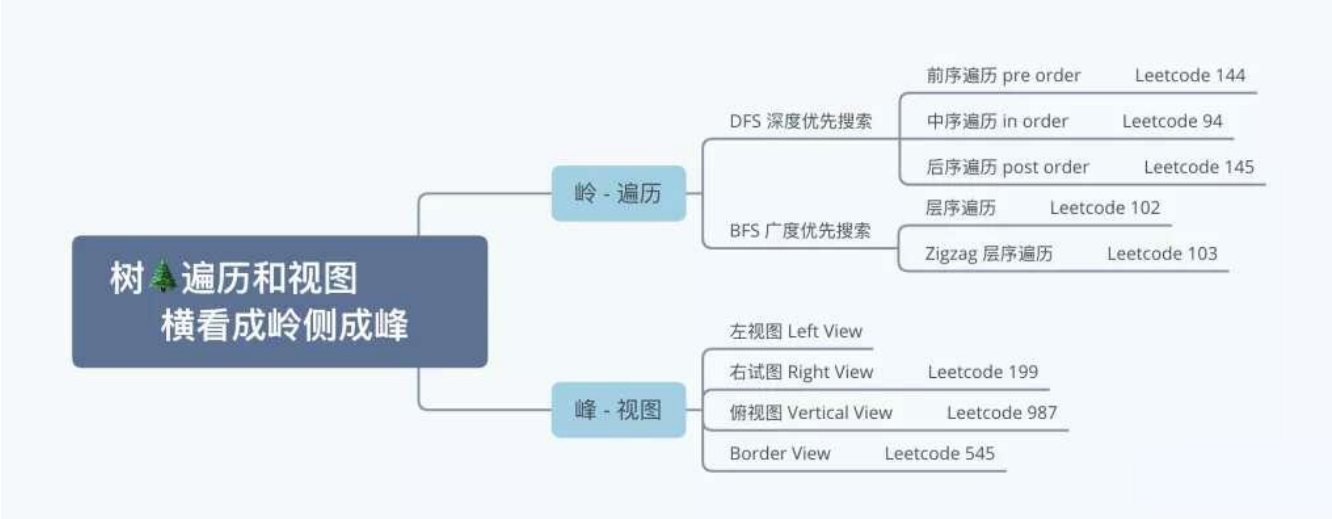
3）倒数第二层若存在叶子结点，一定在右部连续位置。

4）如果结点度为 1，则该结点只有左孩子，即没有右子树。

5）同样结点数目的二叉树，完全二叉树深度最小。

注：满二叉树一定是完全二叉树，但反过来不一定成立。

7、二叉树的遍历



二叉树的遍历，指的是如何按某种搜索路径巡防树中的每个结点，使得每个结点均被访问一次，而且仅被访问一次。

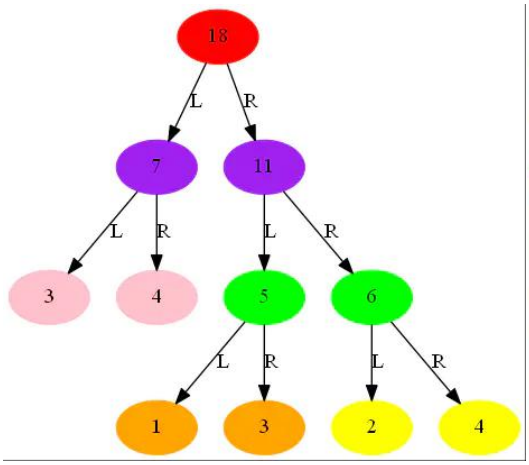
1、前序遍历的操作定义为：若二叉树为空，为空操作；否则（1）访问根节点；（2）先序遍历左子树；（3）先序遍历右子树。

2、中序遍历的操作定义为：若二叉树为空，为空操作；否则（1）中序遍历左子树；（2）访问根结点；（3）中序遍历右子树。

3、后序遍历的操作定义为：若二叉树为空，为空操作；否则（1）后序遍历左子树；（2）后序遍历右子树；（3）访问根结点。

4、层序遍历的操作定义为：若二叉树为空，为空操作；否则从上到下、从左到右按层

次进行访问。

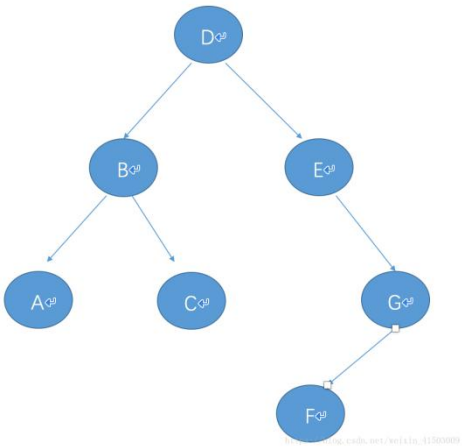


前序遍历为:
18 7 3 4 11 5 1 3 6 2 4
中序遍历为:
3 7 4 18 1 5 3 11 2 6 4
后序遍历为:
3 4 7 1 3 5 2 4 6 11 18
层序遍历为:
[[18], [7, 11], [3, 4, 5, 6], [1, 3, 2, 4]]

前序遍历为:
18 7 3 4 11 5 1 3 6 2 4
中序遍历为:
3 7 4 18 1 5 3 11 2 6 4
后序遍历为:
3 4 7 1 3 5 2 4 6 11 18
层序遍历为:
[[18], [7, 11], [3, 4, 5, 6], [1, 3, 2, 4]]

相关代码网址
https://blog.csdn.net/weixin_41503009/article/details/82356000
https://www.jianshu.com/p/9503238394df
https://segmentfault.com/a/1190000018265301?utm_source=tag-newest

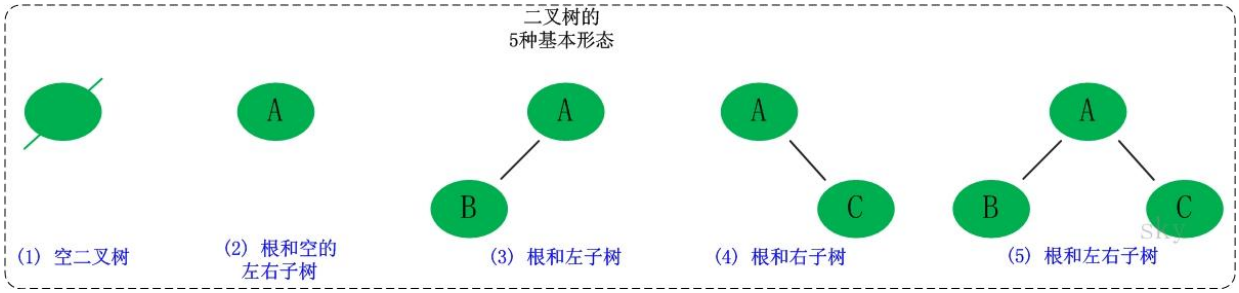
例子：用 python 代码实现，代码在下面



前序遍历：D B A C E G F
中序遍历：A B C D E F G
后序遍历：A C B F G E D
层序遍历：D B E A C G F

前序遍历：D B A C E G F
中序遍历：A B C D E F G

后序遍历：A C B F G E D
层序遍历：D B E A C G F



下面代码可直接运行，不用导入任何包，直接写 class Nodes 类，类名随便取

```python
class Nodes:
    def __init__(self, value=None, left=None, right=None):
        self.value = value
        self.left = left    # 左子树
        self.right = right  # 右子树

#前序遍历，这里必须有缩进
def preTraverse(root):
    if root == None:
        return
    print(root.value)
    preTraverse(root.left)
    preTraverse(root.right)

#中序遍历
def midTraverse(root):
    if root == None:
        return
    midTraverse(root.left)
    print(root.value)
    midTraverse(root.right)

#后序遍历
def afterTraverse(root):
    if root == None:
        return
    afterTraverse(root.left)
    afterTraverse(root.right)
    print(root.value)

#层序历遍
def layerTraverse(root):
    result=[]
    queue = [root.value, root.left, root.right]
    while queue:
        temp=queue.pop(0)
        if temp:
            if isinstance(temp,Nodes):
                queue.append(temp.value)
                queue.append(temp.left)
                queue.append(temp.right)
            else:
                result.append(temp)
    print(result)
```

调用函数，打印四种遍历

```
if __name__ == '__main__':
    roots = Nodes('D', Nodes('B', Nodes('A'), Nodes('C')), Nodes('E', right=Nodes('G', Nodes('F'))))
    print('前序遍历：')
    preTraverse(roots)      #前序遍历：D B A C E G F
    print('中序遍历：')
    midTraverse(roots)      #中序遍历：A B C D E F G
    print('后序遍历：')
    afterTraverse(roots)    #后序遍历：A C B F G E D
    print('层序遍历：')
    layerTraverse(roots)    #层序遍历：D B E A C G F
```

计算二叉树节点数和深度，直接在下面方法里接着写：

```
#递归计算二叉树结点个数
def node_count(root):
    if root and not isinstance(root, Nodes):
        return None

    if root:
        return node_count(root.left) + node_count(root.right) + 1
    else:
        return 0

#非递归计算二叉树结点个数，借用分层遍历计算
def node_count_not_recursion(root):
    if root and not isinstance(root, Nodes):
        return None

    return len(layerTraverse(root))

#递归计算二叉树深度
def tree_deep(root):
    if root and not isinstance(root, Nodes):
        return None

    if root:
        return 1 + max(tree_deep(root.left), tree_deep(root.right))
    else:
        return 0

#非递归计算二叉树深度，借用分层遍历计算
def tree_deep_not_recursion(root):
    if root and not isinstance(root, Nodes):
        return None
    result = 0
    queue = [(root, 1)]
    while queue:
        temp_node, layerTraverse = queue.pop(0)
        if temp_node:
            queue.append((temp_node.left, layerTraverse+1))
            queue.append((temp_node.right, layerTraverse+1))
            result = layerTraverse + 1

    return result-1
```

```
if __name__ == '__main__':
    roots = Nodes('D', Nodes('B', Nodes('A'), Nodes('C')), Nodes('E', right=Nodes('G', Nodes('F'))))
    print('前序遍历：')
    preTraverse(roots)      #前序遍历：D B A C E G F
    print('中序遍历：')
    midTraverse(roots)      #中序遍历：A B C D E F G
    print('后序遍历：')
    afterTraverse(roots)    #后序遍历：A C B F G E D
    print('层序遍历：')
    layerTraverse(roots)    #层序遍历：D B E A C G F
    print('递归二叉树结点个数：',node_count(roots))     #7
    print('非递归二叉树结点个数：',node_count_not_recursion(roots))    #7
    print('递归计算二叉树深度：',tree_deep(roots))      #4
    print('非递归计算二叉树深度：',tree_deep_not_recursion(roots))     #4
```

七十七、DefaultTestLoader 与 TestSuite 区别:

1. TestSuite 可以添加 TestCase 中所有的 Test 开头的方法和添加指定的 test 开头方法。
2. defaultTestloader()类，通过该类下面的 discover()方法可自动搜索指定目录下开头的测试用例文件(test*.px) 文件，并将查找到的测试用例组装到测试套件并添加 TestCase 内所有开头的方法，不能指定添加方法，因此可以直接通过 run()方法执行 discover。提示：defaultTestLoader 是属于 TestSuite 另外-种实现方式

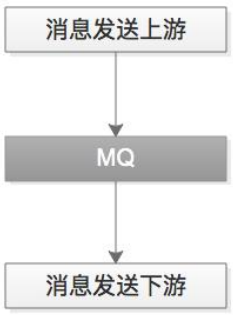
七十八、测试用例的通过率和覆盖度

假如测试用例有 20 条用例，测试了 10 条用例，覆盖率为 10/20*100%=50%，如果 10 条中有 5 条测试通过，5/20*100%=25%的通过率

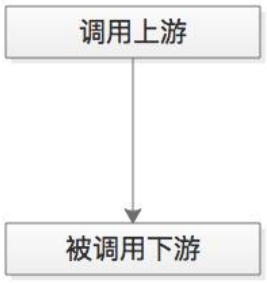
七十九、MQ 消息队列（Message Queue）

1、MQ，是一种跨进程的通信机制，用于上下游传递消息。在互联网架构中，MQ 是一种非常常见的上下游“逻辑解耦+物理解耦”的消息通信服务。使用了 MQ 之后，消息发送上游只需要依赖 MQ，逻辑上和物理上都不用依赖其他服务。
消息中间件可利用高效可靠的消息传递机制进行平台无关的数据交流，并基于数据通信来进行分布式系统的集成。通过提供消息传递和消息排队模型，可以在分布式环境下扩展进程间的通信。通过消息中间件，应用程序或组件之间可以进行可靠的异步通讯，从而降低系统之间的耦合度，提高系统的可扩展性和可用性。

什么时候使用 MQ：
1）数据驱动的任务依赖
2）上游不关心多下游执行结果
3）异步返回执行时间长



2、什么时候不使用 MQ 消息队列
调用与被调用的关系，是无法被 MQ 取代的。MQ 只用来传递上游任务执行完成的消息，并不用于传递真正的输入输出数据，调用方实时依赖执行结果的业务场景，请使用调用，而不是 MQ。上游实时关注执行结果



3、MQ 的不足是：
1）系统更复杂，多了一个 MQ 组件
2）消息传递路径更长，延时会增加
3）消息可靠性和重复性互为矛盾，消息不丢不重难以同时保证
4）上游无法知道下游的执行结果，这一点是很致命的

4、采用 MQ 的优点是：
1）解耦
2）冗余
3）扩展性
4）灵活性和峰值处理能力
5）可恢复性
6）顺序保证
7）缓冲
8）异步通信

八十、Python2 和 Python3 的区别

1、Python2 既可以使用带小括号的方式，或使用一个空格来分隔打印内容，比如 print 'hi'
Python3 使用 print 必须要以小括号包括打印内容，比如 print('hi')
2、python2 range(1,10)返回列表，python3 中返回迭代器，节约内存，xrange 重命名为 range
3、python2 中使用 ascii 编码，python3 中使用 utf-8 编码

4、python2 中 unicode 表示字符串序列，str 表示字节序列
python3 中 str 表示字符串序列，byte 表示字节序列
5、python2 中为正常显示中文，引入 coding 声明，python3 中不需要
6、python2 中是 raw_input()函数，python3 中是 input()函数
7、新的字符串格式化方法 format 取代%
8、整除:(没有太大影响)（Python3 中/表示真除，%表示取余，//表示地板除（结果取整）；Python2 中/表示根据除数被除数小数点位得到结果，//同样表示地板除）统一一下：Python3 中/表示真除，%表示取余，//结果取整；Python2 中带上小数点/表示真除，%表示取余，//结果取整

八十一、Python 常见数据结构及区别

Python 中常见的数据结构可以统称为容器（container）。序列（如列表 list 和元组 tuple）、映射（如字典 dict）、字符串(string)、以及集合（set）。

面试题一：list 和元祖的区别
list 是可变的对象，元组 tuple 是不可变的对象
tuple 是不可变的，这使得它可以作为 dict 的 key
创建 tuple 比 list 要快，存储空间比 list 占用更小
多线程并发的时候，tuple 是不需要加锁的，不用担心安全问题

字典dic、列表list、集合set、元祖tuple相互比较

比较	详情
字典 `dict` 和列表 `list` 的比较	`dict` 的查找和插入的速度极**快**，不会随着 `key` 的增加而变慢。但是需要占用**大量的内存**，内存浪费多。相反，`list` 查找和插入的时间随着元素的增加而增加。但是占用空间小，浪费**内存很少**。
字典 `dict` 和集合 `set` 比较	唯一区别仅在于**没有**存储对应的**value**，但是，set的原理和dict一样，所以，同样不可以放入可变对象，因为无法判断两个可变对象是否相等，也就无法保证set内部"不会有重复元素"。
元祖 `tuple` 与列表 `list` 比较	`tuple` 与 `list` **类似**，不同之处在于tuple中的元素**不**能进行**修改**。而且 `tuple` 使用小括号 `()`，`list` 使用方括号 `[]`。

列表的应用场景：
尽管 python 中的列表可以存储不同类型的数据，但是在开发中，更多的应用场景是：
1.列表存储相同类型的数据
2.通过迭代遍历，在循环体内部，针对列表中的每一项元素，执行相同的操作。

元组的应用场景：
尽管可以使用 for in 遍历元祖，但是在开发中，更多的应用场景是：
函数的参数和返回值，一个函数可以接受任意多个参数，或者依次返回多个数据
格式化字符串，格式化字符串后面的（）本质上就是一个元祖；
让列表不可以修改，以保护数据安全

字典的应用场景：
使用多个键值对，存储描述一个物体的相关信息－描述更复杂的数据信息
将多个字典放在一个列表中，再进行遍历，再循环体内针对每一个字典进行相同的处理

数据类型:
int ==> 整数. 主要用来进行数学运算
str ==> 字符串, 可以保存少量数据并进行相应的操作
bool==>判断真假, True, False
list==> 存储大量数据.用[ ]表示示
tuple=> 元组, 不可以发生改变 用( )表示示
dict==> 字典, 保存键值对, 一样可以保存大量数据
set==> 集合, 保存大量数据. 不可以重复. 其实就是不保存 value 的 dict

数据结构：
列表 list []: 可以修改,方括号表示,逗号区分各个子元素,其中字符串需引号区分["你好","中国",2018,"改革开放 40 周年"]
元祖 tuple (,):不可以修改,使用圆括号表示, 但可相互转换
字典 dict {key: values}: 键-值组成项
集合 set {key}:单纯键值,唯一

数据类型	**列表** (list)	**元祖** (tuple)	**集合** (set)	**字典** (dictionary)
表示	`[]`	`()`	`{}`	`{}`
举例	`a_list = [1, True, 'aa']`	`a_tuple = (1, True, 'aa');` `t2 = (1,)`	`a_set = {1, True, 'aa'};` `s = set([1, True, 'aa'])`	`a_dict = {'name': 'xuan', 'age': 21}`
是否**有序**	有序	有序	**无序**	**无序**
是否**读写**	读写	**只读**	读写	读写
空定义	`a_list = []`	`a_tuple = ()`	`a_set =set();` `not_sure = {}` **空**字典	`a_dict = {}`
元素**可修改**	`a_list[0] = 23`	**否**	**否**	`a_dict['age'] = 30`
下标访问	`a_list[0] = 23`	`a_tuple[0]`	**否**	`a_dict['age'] = 30`
添加元素	`+、append、extend、insert`	**不可添加**	`add、update`	`a_dict['new_key'] = 'value'`
删除元素	`del、remove、pop()、pop(1)、clear`	**不可删除**	`discard、remove、pop、clear`	`pop、popitem、clear`
元素**查找**	`index、count、in`	`in`	`in`	`a_dict['key']`
布尔真值	非空	非空	非空	非空
定义	列表(`list`)是 `有序` 的集合，可以存放不同数据类型的数据，并且 `list` 中的每个元素的都对应着一个 `索引` 来标记其位置，且索引从0开始。	`tuple` 与 `list` 类似，不同之处在于 `tuple` 中的 `元素不能` 进行 `修改` 。而且tuple使用小括号 `()` ， `list` 使用方括号 `[]` 。	`dic` 字典是另一种 `可变` 的容器模型，且可存储 `任意` 类型对象。字典的每个键值 `(key:value)` 对用冒号(`:`)分割，每个对之间用逗号(`,`)分割，整个字典包括在花括号 `{}` 中	`set()` 函数是创建一个 `无序不重复` 元素集，可添加，删除数据，还可以计算 `交集、差集、并集` 等。 `python` 的集合类型和 其他语言类似, 是一个 `无序` 不重复元素集, 基本功能包括关系测试和消除重复元素.集合对象还支持 `union(联合)`, `intersection(交)`, `difference(差)` 和 `sysmmetricdifference(对称差集)` 等数学运算,和我们初中数学学的集合的非常的相似。

八十二、Selenium 实现原理

Selenium3.0 主要由三大部分组成：SeleniumIDE、Selenium WebDriver、Selenoium Grid。
Selenium IDE：录制和回放脚本，可以模拟用户对页面的真实操作，区别于其他工具：是通过拦截 http 请求。
一般只把录制脚本当作一个辅助功能，因为一个 UI 节点的细微变化，都可能导致自动化测试工具无法识别，当测试项目项目大时，定位、更新十分困难。
其次，录制的脚本有时候人工难以理解。

Selenium Grid：实现在多台机器上、和异构环境中并行执行测试用例。并行执行不仅节省时间，而且可以同时在不同的浏览器、平台上运行自动化测试脚本。

Selenium Web Driver：针对各个浏览器而开发，通过原生浏览器支持或者扩展（Chrome webDrive、FireFox WebDriver）直接控制浏览器

selenium 1.0 = selenium IDE + selenium Grid + selenium RC
selenium IDE：实现简单的浏览器操作的录制和回放。
selenium Grid：实现多台机器上和异构环境中运行测试用例。
selenium RC：可作为代理服务器去访问应用。
Selenium1.0 引入了 Remote Control Server(远程控制服务) 这样一个代理 Server，JavaScript 脚本注入和与 Server 通讯都通过这个代理 Server 来进行
流程说明：
客户端建立与 selenium-RC server 的连接。
Selenium RC Server 启动一个浏览器（或是已经使用中），并注入 JS 代码
将 Selenium 代码传到客户端的 Selenium-Core(核心) 中。
Selenium-Core 翻译并解析执行用户录制的操作。
让代理 Server 进行通讯
Remote Control Server 负责跟远程 Web 应用服务器进行通讯。
操作完成，显示结果，并执行下一指令。

在 Selenium1.0 中，是通过 Selenium RC 服务器作为代理服务器去访问应用从而达到测试的目的。

Selenium RC 分为三个部分，Launcher、HttpProxy、Core。
Launcher 用于启动浏览器，把 Selenium Core 加载到浏览器中，并且把浏览器的代理设置为 Selenium Server 的 Http Proxy。
Core 是一堆 JavaScript 的集合，所以本质相当于运行这些 JavaScript 函数来实现对 Html 页面的操作。——这也是为什么可以运行在几乎所有主流的浏览器上。
然而直接运行 JavaScript 会有极大的安全漏洞，所以会受到“同源限制”，所谓“同源”指的是“三个相同”。协议相同，域名相同，端口相同。在这个基础上，Selenium2.0 引入了 WebDriver。

Web Driver 原理：

webDriver 是按照 client/server 模式设计的。client 是我们的测试脚本，发送请求；server 就是打开的浏览器，用来接收 client 的请求并作出响应。

具体的工作流程：
1. WebDriver 启动目标浏览器，并绑定到指定端口。该启动的浏览器实例，做为 WebDriver 的 remote server。
2. Client 端通过 CommandExcuter (命令执行者)发送 HTTPRequest 请求给远程服务器 remote server 的侦听端口（通信协议： the webriver wire protocol）

3. Remote server 远程服务需要依赖原生的浏览器组件（如：IEDriverServer.exe、chromedriver.exe），来转化，转化浏览器的 native(本地) 调用。

八十三、Django 实现原理

1、Django 已经成为 web 开发者的首选框架，是一个遵循 MVC 设计模式的框架。MVC 是 Model、View、Controller 三个单词的简写，分别代表模型、视图、控制器。Django 其实也是一个 MTV 的设计模式。MTV 是 Model、Template、View 三个单词的简写，分别代表模型、模版、视图。但是在 Django 中，控制器接受用户输入的部分由框架自行处理，所以 Django 里更关注的是模型（Model）、模板(Template)和视图（Views），称为 MTV 模式。

2、工作机制
1.用 manage .py runserver 启动 Django 服务器时就载入了在同一目录下的 settings .py。该文件包含了项目中的配置信息，如前面讲的 URLConf 等，其中最重要的配置就是 ROOT_URLCONF，它告诉 Django 哪个 Python 模块应该用作本站的 URLConf，默认的是 urls .py
2.当访问 url 的时候，Django 会根据 ROOT_URLCONF 的设置来装载 URLConf。
3.然后按顺序逐个匹配 URLConf 里的 URLpatterns。如果找到则会调用相关联的视图函数，并把 HttpRequest 对象作为第一个参数(通常是 request)
4.最后该 view 函数负责返回一个 HttpResponse 对象。

八十四、Java 中接口(interface)和抽象类(abstract class)的区别

相同点：
（1）都不能被实例化
（2）接口的实现类或抽象类的子类都只有实现了接口或抽象类中的方法后才能实例化。

不同点：
（1）接口只有定义，不能有方法的实现，java 1.8 中可以定义 default 方法体，而抽象类可以有定义与实现，方法可在抽象类中实现。
（2）实现接口的关键字为 implements，继承抽象类的关键字为 extends。一个类可以实现多个接口，但一个类只能继承一个抽象类。所以，使用接口可以间接地实现多重继承。
（3）接口强调特定功能的实现，而抽象类强调所属关系。

（4）接口成员变量默认为 public static final，必须赋初值，不能被修改；其所有的成员方法都是 public、abstract 的。抽象类中成员变量默认 default，可在子类中被重新定义，也可被重新赋值；抽象方法被 abstract 修饰，不能被 private、static、synchronized 和 native 等修饰，必须以分号结尾，不带花括号。
（5）接口被用于常用的功能，便于日后维护和添加删除，而抽象类更倾向于充当公共类的角色，不适用于日后重新对立面的代码修改。功能需要累积时用抽象类，不需要累积时用接口。

八十五、Spring 的工作原理

1、Spring 内部最核心的就是 IOC 了，动态注入，让一个对象的创建不用 new 了，可以自动的生产，这其实就是利用 java 里的反射，反射其实就是在运行时动态的去创建、调用对象，Spring 就是在运行时，跟 xml Spring 的配置文件来动态的创建对象和调用对象里的方法的 。
2、IOC（Inversion of Control）控制反转：
简单地说，由 spring 来负责控制对象的生命周期和对象间的关系。传统的 Java SE 程序设计，我们直接在对象内部通过 new 进行创建对象，是程序主动去创建依赖对象；而 IOC 是有专门一个容器来创建这些对象，即由 Ioc 容器来控制对象的创建；就是把 new 对象实例化的工作交给 spring 容器来完成，spring 帮我们负责销毁对象，控制对象的生命周期，在需要使用对象的时候直接向 spring 申请即可。控制对象生存周期的不再是引用它的对象，而是 spring。对于某个具体的对象而言，以前是它控制其他对象，现在是所有对象都被 spring 控制，所以这叫控制反转。

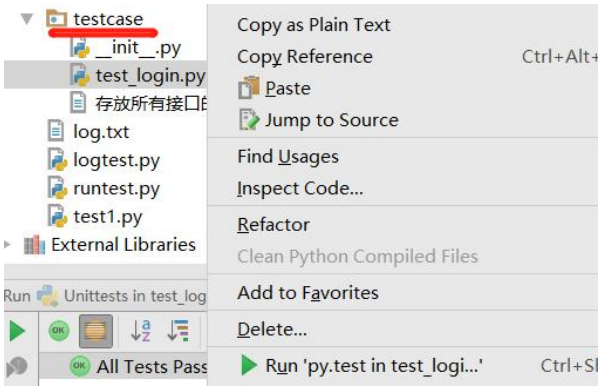
八十六、Python 自动化测试框架 Pytest 和 Unittest 区别

一、用例设计规则

1、Unittest
（1）测试类必须继承 unittest.TestCase
（2）测试函数必须以"test_"开头
（3）测试类必须有 unittest.main()方法
2、Pytest
（1）测试文件的文件名必须以"test_"开头，或者以"_test"结尾
（2）测试类命名必须以"Test"开头
（3）测试函数名必须以"test"开头
（4）测试类里面不能使用"__init__"方法

总结：pytest 是基于 unittest 衍生出来的新的测试框架，使用起来相对于 unittest 来说更简单、效率来说更高，pytest 兼容 unittest 测试用例，但是反过来 unittest 不兼容 pytest，所以说 pytest 的容错性更好一些！在使用交互逻辑上面 pytest 比 unittest 更全一些！unittest 是 python 自带的测试库。如果两个框架都安装，默认优先运行 pytest 模式，如果你的文件夹名叫 testcase 的，右键运行 test_login.py 只显示 pytest，即使设置 unittest 运行模式，右键也出不

来 unittest 运行方法，两种解决方法：1、非要用 testcase 这个文件夹名，只能改名 test_logins.py 就可以用 unittest 运行模式了，2、或者改文件夹名，testcases 就可以用 unittest 运行模式，或者别用 test 开头的文件夹名称即可。



二、断言对比

1、unittest 断言
assertEqual(a, b)　　# 判断 a 和 b 是否相等
assertNotEqual(a, b)　　# 判断 a 不等于 b
assertTrue(a)　　# 判断 a 是否为 Ture
assertFalse(a)　　#判断 a 是否为 False
assertIn(a, b)　　#a 包含在 b 里面
asserNotIn(a, b)　　#a 不包含在 b 里面

2、pytest 断言
pytest 只需要用 assert 来断言就行，assert 后面加需要断言的条件就可以了，例如：assert a = = b # 判断 a 是否等于 b、
assert a != b # 判断 a 不等于 b、assert a in b # 判断 b 包含 a
总结：从断言上面来看，pytest 的断言比 unittest 要简单些，unittest 断言需要记很多断言格式，pytest 只有 assert 一个表达式，用起来比较方便

三、用例前置和后置

1、unittest 前置和后置
（1）通过 setup 每个用例执行前执行，teardown 每个用例执行后执行
（2）通过 setupclass 类里面所有用例执行前执行，teardownclass 类里面所有用例执行后执行

2、pytest 前置和后置
（1）模块级别：setup_module/teardown_module，整个.py 全部用例开始前执行/全部用例执行完后执行
（2）函数级别：setup_function/teardown_function，只对函数级别生效，每个用例开始前和

结束后执行一次
（3）类级别：setup_class/teardown_ class，只对类级别生效，类里面所有用例开始前执行一次，所有用例执行完执行一次
（4）方法级别：setup_method/teardown_method，只是类里面方法级别生效，方法开始前执行一致，方法结束后执行一次
（5）方法级别：setup/teardown，这个与 setup_method/teardown_method 用法很类似，但是级别比 method 级别要低，也就是说在同一个方法中会先执行 setup_method 再执行 setup，方法结束后先执行 teardown 再执行 teardown_method

pytest 自定义设置前置和后置：
通过 firture 可以自定义 pytest 的前置和后置，格式 fixture(scope="function", params=None, autouse=False, ids=None, name=Noe)
scope:有四个级别，function（默认）,class,module,session
params:参数列表
autouse:False 为默认值，意思代表需要根据设置的条件（scope 级别）来激活 fixture，如果为 Ture，则表示所有 function 级别的都被激活 fixture
ids：每个字符串 id 的列表，感觉没啥实质性作用
name：fixture 的名字

四、参数化

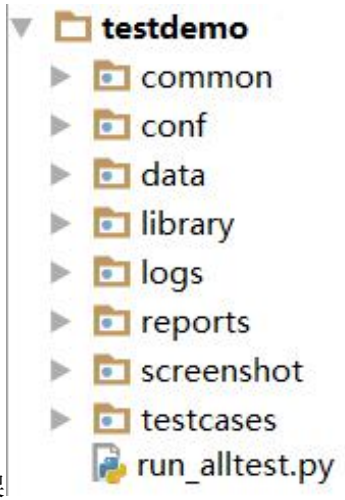
1、unittest 参数化
可以通过 nose_parameterized 来实现，格式：
@nose_parameterized.parameterized.expand(data), 'data'为 list 格式的参数化的数据
2、pytest 参数化
通过装饰器@pytest.mark.parametrize 来实现

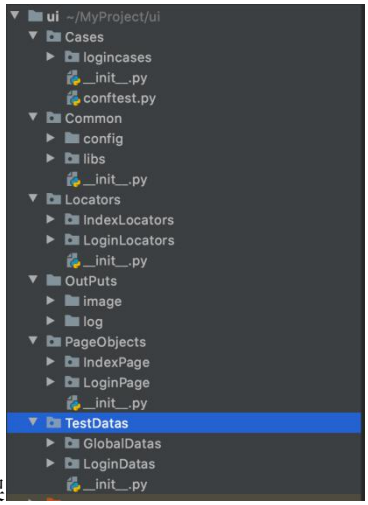
五、生成报告方式

1、unittest
通过 HTMLTestRunner 生成
2、pytest
（1）通过 pytest-html 生成 html 格式报告
（2）通过 allure 生成方案（很详细）

八十七、接口/UI 自动化框架设计

1、common：目录封装了发送接口请求、读取配置文件、处理日志、邮件和 excel 的工具类
2、conf：目录通过 ini 文件配置数据库和测试环境的 url 地址
3、data：目录存放所有接口的入参信息，比如 excel 文件
4、library：目录存放 ddt 和 thmltestrunner 模块
5、logs：目录存放 log 日志文件
6、reports：目录存放运行测试用例生成的报告文件
7、screenshot：目录存放截图
8、testcases：(这个必须加 s)目录存放所有接口的测试用例（这里如果安装 pytest 和 unittest 两个框架，起 testcase 作为文件夹的名称存放测试用例时，右键运行 py 文件只会显示 pytest 模式，所以不能起 testcase 这样的名称）
9、run_alltest：run_alltest.py 文件运行所有的测试用例，是单个文件，其他都是文件夹类型

接口框架 

ui 框架 

八十八、高并发下的流量控制

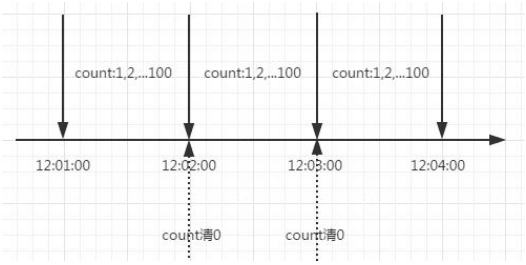
这个时候如果不做任何保护措施，服务器就会承受很大的处理压力，请求量很高，服务器负载也很高，并且当请求超过服务器承载极限的时候，系统就会崩溃，导致所有人都不能访问。为了应用服务的高可用，一个常用的办法是对大流量的请求（秒杀/抢购）进行限流，拦截掉大部分请求，只允许一部分请求真正进入后端服务器，这样就可以防止大量请求造成系统压力过大导致的系统崩溃，从而保护服务正常可用。
令牌桶(Token Bucket)、漏桶(leaky bucket)和滑动窗口、计数器算法是最常用的四种限流的算法。

限流算法

计数器

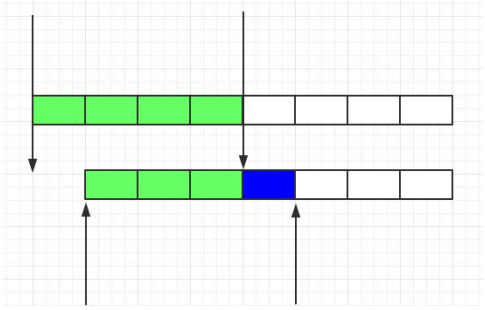
计数器限流算法也是比较常用的，主要用来限制总并发数。比如限流 qps 为 100，算法的实现思路就是从第一个请求进来开始计时，在接下去的 1s 内，每来一个请求，就把计数加 1，如果累加的数字达到了 100，那么后续的请求就会被全部拒绝。等到 1s 结束后，把计数恢

复成 0，重新开始计数。
这种实现方式有一个弊端：如果我在单位时间 1s 内的前 10ms，已经通过了 100 个请求，那后面的 990ms，只能眼巴巴的把请求拒绝，这种现象称为 突刺现象。



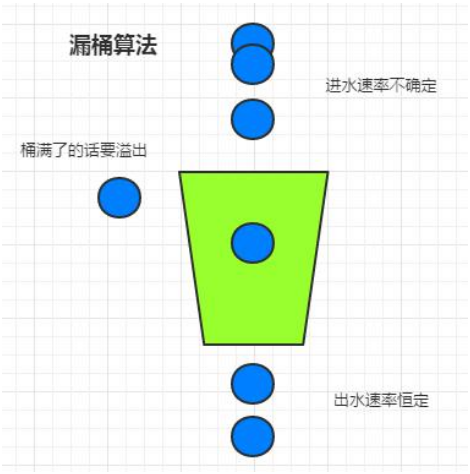
滑动窗口

滑动窗口的意思是说把固定时间片，进行划分，并且随着时间的流逝，进行移动，这样就巧妙的避开了计数器的临界点问题。也就是说这些固定数量的可以移动的格子，将会进行计数判断阀值，因此格子的数量影响着滑动窗口算法的精度。



漏桶

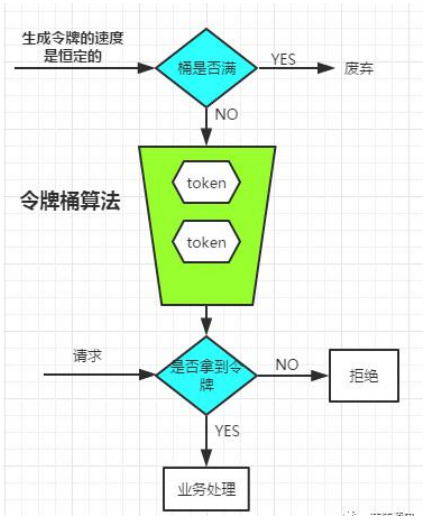
为了消除突刺现象，可以采用漏桶算法实现限流，漏桶算法这个名字就很形象，算法内部有一个容器，类似生活用到的漏斗，当请求进来时，相当于水倒入漏斗，然后从下端小口慢慢匀速的流出。不管上面流量多大，下面流出的速度始终保持不变。
不管服务调用方多么不稳定，通过漏桶算法进行限流，每 10 毫秒处理一次请求。因为处理的速度是固定的，请求进来的速度是未知的，可能突然进来很多请求，没来得及处理的请求就先放在桶里，既然是个桶，肯定是有容量上限，如果桶满了，那么新进来的请求就丢弃。



在算法实现方面，可以 准备一个队列，用来保存请求，另外通过一个线程池定期从队列中获取请求并执行，可以一次性获取多个并发执行。
这种算法，在使用过后也存在弊端：无法应对短时间的突发流量，同时它的优点也是可以平滑网络上的突发流量，请求可以被整形成稳定的流量。

令牌桶

从某种意义上讲，令牌桶算法是对漏桶算法的一种改进，桶算法能够限制请求调用的速率，而令牌桶算法能够在限制调用的平均速率的同时还允许一定程度的突发调用。
在令牌桶算法中，存在一个桶，用来存放固定数量的令牌。算法中存在一种机制，以一定的速率往桶中放令牌。每次请求调用需要先获取令牌，只有拿到令牌，才有机会继续执行，否则选择选择等待可用的令牌、或者直接拒绝。
放令牌这个动作是持续不断的进行，如果桶中令牌数达到上限，就丢弃令牌，所以就存在这种情况，桶中一直有大量的可用令牌，这时进来的请求就可以直接拿到令牌执行，比如设置 qps 为 100，那么限流器初始化完成一秒后，桶中就已经有 100 个令牌了，这时服务还没完全启动好，等启动完成对外提供服务时，该限流器可以抵挡瞬时的 100 个请求。所以，只有桶中没有令牌时，请求才会进行等待，最后相当于以一定的速率执行。



实现思路：可以 准备一个队列，用来保存令牌，另外通过一个线程池定期生成令牌放到队

列中，每来一个请求，就从队列中获取一个令牌，并继续执行。
漏桶 VS 令牌桶：两者主要区别在于“漏桶算法”能够强行限制数据的传输速率，而“令牌桶算法”在能够限制数据的平均传输速率外，还允许某种程度的突发传输。在“令牌桶算法”中，只要令牌桶中存在令牌，那么就允许突发地传输数据直到达到用户配置的门限，所以它适合于具有突发特性的流量。

集群限流

Redis 请求窗口

采用 redis 的计时和计数方式,在规定的时间窗口期,允许通过的最大请求数量
比如为了限制某个资源被每个用户或者商户的访问次数，5s 只能访问 2 次，或者一天只能调用 1000 次，这种需求，单机限流是无法实现的，这时就需要通过集群限流进行实现。
如何实现？为了控制访问次数，肯定需要一个计数器，而且这个计数器只能保存在第三方服务，比如 redis。
大概思路：每次有相关操作的时候，就向 redis 服务器发送一个 incr 命令，比如需要限制某个用户访问 /index 接口的次数，只需要拼接用户 id 和接口名生成 redis 的 key，每次该用户访问此接口时，只需要对这个 key 执行 incr 命令，在这个 key 带上过期时间，就可以实现指定时间的访问频率。

Nginx 限流

Nginx 按请求速率限速模块使用的是漏桶算法，即能够强行保证请求的实时处理速度不会超过设置的阈值。
Nginx 官方版本限制 IP 的连接和并发分别有两个模块： - limit_req_zone 用来限制单位时间内的请求数，即速率限制,采用的漏桶算法 “leaky bucket”。 - limit_req_conn 用来限制同一时间连接数，即并发限制。

八十九、什么是性能测试、负载测试、压力测试？

性能测试（Performance Test）：通常收集所有和测试有关的所有性能，被不同人在不同场合下进行使用。关注点：how much 和 how fast

负载测试（Load Test）：负载测试是一种性能测试，指数据在超负荷环境中运行，程序是否能够承担。关注点：How much

压力测试（Stress Test）：压力测试（又叫强度测试）也是一种性能测试，它在系统资源特别低的情况下软件系统运行情况，目的是找到系统在哪里失效以及如何失效的地方。

九十、简述一下缺陷的生命周期？

打开 ： 表示问题被提交等待有人处理。重新指派 ： 问题被重新指派给某人处理。
处理 ： 问题在处理中，尚未完成。
固定 ： 确认此问题存在，但暂时不进行处理。
回归 ： 对已经修复的问题进行回归确认。Reopened。
关闭 ： 问题的最后一个状态。
重新指派 ： 问题被重新指派给某人处理。
处理 ： 问题在处理中，尚未完成。
固定 ： 确认此问题存在，但暂时不进行处理。
回归 ： 对已经修复的问题进行回归确认。Reopened。
关闭 ： 问题的最后一个状态。

九十一、测试用例设计方法都有哪些？

1.等价类划分法
顾名思义，等价类划分，就是将测试的范围划分成几个互不相交的子集，他们的并集是全集，从每个子集选出若干个有代表性的值作为测试用例。

2.边界值分析法
长期的测试工作经验告诉我们，大量的错误是发生在输入或输出范围的边界上，而不是发生在输入输出范围的内部。因此针对各种边界情况设计测试用例，可以查出更多的错误。选出的测试用例，应选取正好等于、刚刚大于、刚刚小于边界的值，例如，对于在区间 min，max 的值，测试用例可以记为 min，min+，max，max-。

3.错误推测法
错误推测法是指：在测试程序时，人们可以根据经验或直觉推测程序中可能存在的各种错误，从而有针对性地编写检查这些错误的测试用例的方法。

4.判定表法
又称为策略表，基于策略表的测试，是功能测试中最严密的测试方法。该方法适合于逻辑判断复杂的场景，通过穷举条件获得结果，对结果再进行优化合并，会得到一个判断清晰的策略表。

5.正交实验法
用语言描述正交实验法会很抽象难懂，简单说，就是在各因素互相独立的情况下，设计出一种特殊的表格，找出能以少数替代全面的测试用例。
其中，上面所说的特殊表格就是正交表，是按照一定规则生成的表。
虽然说是特殊的表格，实际表现形式跟一般的表格没有什么区别，正交表的主要特征是，“均匀分布，整齐划一”，正是因为“均匀”的，所以才能以少数代替全部。

九十二、unittest 五个各个组件的介绍

1、TestCase：测试用例，测试用例里面会有很多测试方法，是单元测试中最小维度的测试行为。所有用例都要继承这个类

2、TestFixure：测试固件、夹具，测试固件是执行测试时的准备工作和收尾工作（打开和关闭浏览器，链接和关闭数据库）。

1.setUp：在写测试用例的时候，每次操作其实都是基于打开浏览器输入对应网址这些操作，这个就是执行用例的前置条件。

2.tearDown：执行完用例后，为了不影响下一次用例的执行，一般有个数据还原的过程，这就是执行用例的后置条件。

3、TestSuite：测试套件，是测试用例的集合，用 addTest 加载测试用例的容器

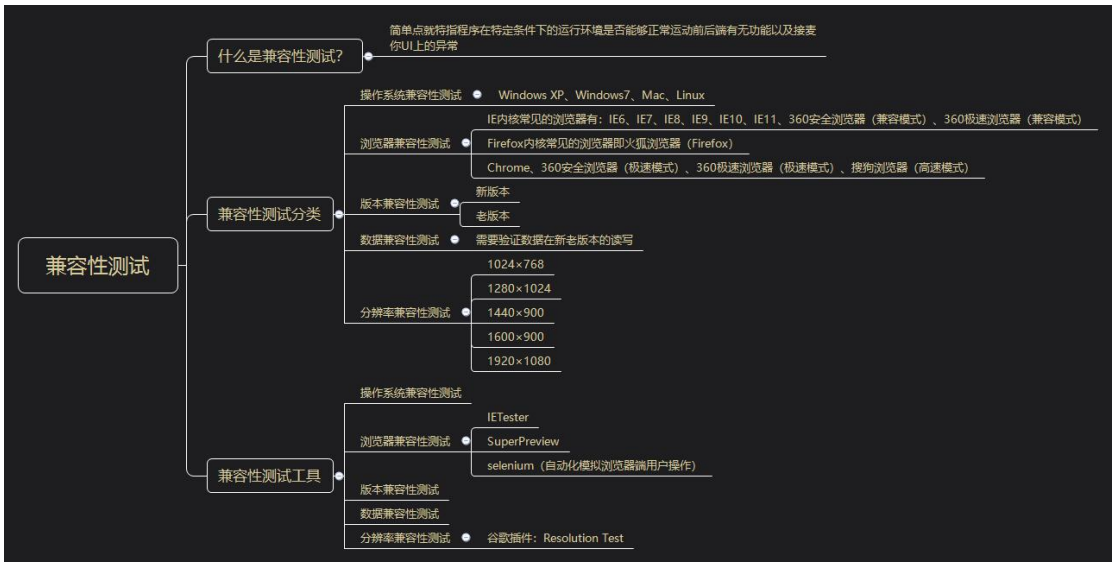
4、defaultTestLoader：用来收集测试用例，discover 跑指定路径下的 test 开头的.py 文件，并加载到测试套件里

5、TextTestRunner：测试运行器，运行测试用例，并生成文本形式报告，
unittest.TextTestRunner().run(suite)

1、HTMLTestRunner：将测试结果呈现给用户，生成 html 测试报告，
HTMLTestReportCN.HTMLTestRunner

九十三、浏览器/APP 兼容性测试

web 工具：https://www.browserling.com/　　http://browsershots.org/
APP(花钱)工具：https://wetest.qq.com/
代码测试用 selenium



九十四、python 魔术方法

在 Python 中，所有以“__”双下划线包起来的方法，都统称为“Magic Method”，中文称『魔术方法』,例如类的初始化方法 __init__：构造函数，构造器，当一个实例被创建的时候调用的初始化方法
__new__：实例化魔术方法，是在一个对象实例化的时候所调用的第一个方法
__del__：销毁魔术方法，析构器，当一个实例被销毁的时候调用的方法

九十五、cookie、session、token、认证、授权、凭证、jwt 含义

1、什么是认证（Authentication）

通俗地讲就是验证当前用户的身份，证明“你是你自己”（比如：你每天上下班打卡，都需要通过指纹打卡，当你的指纹和系统里录入的指纹相匹配时，就打卡成功）
互联网中的认证：
用户名密码登录
邮箱发送登录链接
手机号接收验证码
只要你能收到邮箱/验证码，就默认你是账号的主人

2、什么是授权（Authorization）

用户授予第三方应用访问该用户某些资源的权限
你在安装手机应用的时候，APP 会询问是否允许授予权限（访问相册、地理位置等权限）
你在访问微信小程序时，当登录时，小程序会询问是否允许授予权限（获取昵称、头像、地区、性别等个人信息）
实现授权的方式有：cookie、session、token、OAuth

3、什么是凭证（Credentials）

实现认证和授权的前提
是需要一种
媒介（证书）
来标记访问者的身份

4、什么是 Cookie

HTTP 是无状态的协议（对于事务处理没有记忆能力，每次客户端和服务端会话完成时，服务端不会保存任何会话信息）：每个请求都是完全独立的，服务端无法确认当前访问者的身份信息，无法分辨上一次的请求发送者和这一次的发送者是不是同一个人。所以服务器与浏

览器为了进行会话跟踪（知道是谁在访问我），就必须主动的去维护一个状态，这个状态用于告知服务端前后两个请求是否来自同一浏览器。而这个状态需要通过 cookie 或者 session 去实现。
cookie 存储在客户端： cookie 是服务器发送到用户浏览器并保存在本地的一小块数据，它会在浏览器下次向同一服务器再发起请求时被携带并发送到服务器上。
cookie 是不可跨域的： 每个 cookie 都会绑定单一的域名，无法在别的域名下获取使用，一级域名和二级域名之间是允许共享使用的（靠的是 domain）。

5、什么是 Session

session 是另一种记录服务器和客户端会话状态的机制
session 是基于 cookie 实现的，session 存储在服务器端，sessionId 会被存储到客户端的 cookie 中
session 认证流程：
用户第一次请求服务器的时候，服务器根据用户提交的相关信息，创建对应的 Session
请求返回时将此 Session 的唯一标识信息 SessionID 返回给浏览器
浏览器接收到服务器返回的 SessionID 信息后，会将此信息存入到 Cookie 中，同时 Cookie 记录此 SessionID 属于哪个域名
当用户第二次访问服务器的时候，请求会自动判断此域名下是否存在 Cookie 信息，如果存在自动将 Cookie 信息也发送给服务端，服务端会从 Cookie 中获取 SessionID，再根据 SessionID 查找对应的 Session 信息，如果没有找到说明用户没有登录或者登录失效，如果找到 Session 证明用户已经登录可执行后面操作。

6、什么是 Token（令牌）Acesss Token

访问资源接口（API）时所需要的资源凭证
简单 token 的组成： uid(用户唯一的身份标识)、time(当前时间的时间戳)、sign（签名，token 的前几位以哈希算法压缩成的一定长度的十六进制字符串）
特点：服务端无状态化、可扩展性好、支持移动端设备、安全、支持跨程序调用

7、token 的身份验证流程：

客户端使用用户名跟密码请求登录
服务端收到请求，去验证用户名与密码
验证成功后，服务端会签发一个 token 并把这个 token 发送给客户端
客户端收到 token 以后，会把它存储起来，比如放在 cookie 里或者 localStorage 里
客户端每次向服务端请求资源的时候需要带着服务端签发的 token
服务端收到请求，然后去验证客户端请求里面带着的 token ，如果验证成功，就向客户端返回请求的数据
每一次请求都需要携带 token，需要把 token 放到 HTTP 的 Header 里。
基于 token 的用户认证是一种服务端无状态的认证方式，服务端不用存放 token 数据。用解析 token 的计算时间换取 session 的存储空间，从而减轻服务器的压力，减少频繁的查询数据库

8、另外一种 token——refresh token

refresh token 是专用于刷新 access token 的 token。如果没有 refresh token，也可以刷新 access token，但每次刷新都要用户输入登录用户名与密码，会很麻烦。有了 refresh token，可以减少这个麻烦，客户端直接用 refresh token 去更新 access token，无需用户进行额外的操作。

Access Token 的有效期比较短，当 Acesss Token 由于过期而失效时，使用 Refresh Token 就可以获取到新的 Token，如果 Refresh Token 也失效了，用户就只能重新登录了。

Refresh Token 及过期时间是存储在服务器的数据库中，只有在申请新的 Acesss Token 时才会验证，不会对业务接口响应时间造成影响，也不需要向 Session 一样一直保持在内存中以应对大量的请求。

9、什么是 JWT

JSON Web Token（简称 JWT）是目前最流行的跨域认证解决方案。

是一种认证授权机制。

JWT 是为了在网络应用环境间传递声明而执行的一种基于 JSON 的开放标准（RFC 7519）。

JWT 的声明一般被用来在身份提供者和服务提供者间传递被认证的用户身份信息，以便于从资源服务器获取资源。比如用在用户登录上。

可以使用 HMAC 算法或者是 RSA 的公/私秘钥对 JWT 进行签名。因为数字签名的存在，这些传递的信息是可信的。

WT 认证流程：

用户输入用户名/密码登录，服务端认证成功后，会返回给客户端一个 JWT

客户端将 token 保存到本地（通常使用 localstorage，也可以使用 cookie）

当用户希望访问一个受保护的路由或者资源的时候，需要请求头的 Authorization 字段中使用 Bearer 模式添加 JWT，其内容看起来是下面这样

Authorization: Bearer <token>

服务端的保护路由将会检查请求头 Authorization 中的 JWT 信息，如果合法，则允许用户的行为

因为 JWT 是自包含的（内部包含了一些会话信息），因此减少了需要查询数据库的需要

因为 JWT 并不使用 Cookie 的，所以你可以使用任何域名提供你的 API 服务而不需要担心跨域资源共享问题（CORS）

因为用户的状态不再存储在服务端的内存中，所以这是一种无状态的认证机制

JWT 的使用方式：客户端收到服务器返回的 JWT，可以储存在 Cookie 里面，也可以储存在 localStorage。

10、Token 和 Session 的区别

Session 是一种记录服务器和客户端会话状态的机制，使服务端有状态化，可以记录会话信息。而 Token 是令牌，访问资源接口（API）时所需要的资源凭证。Token 使服务端无状态化，不会存储会话信息。
Session 和 Token 并不矛盾，作为身份认证 Token 安全性比 Session 好，因为每一个请求都有签名还能防止监听以及重放攻击，而 Session 就必须依赖链路层来保障通讯安全了。如果你需要实现有状态的会话，仍然可以增加 Session 来在服务器端保存一些状态。
所谓 Session 认证只是简单的把 User 信息存储到 Session 里，因为 SessionID 的不可预测性，暂且认为是安全的。而 Token ，如果指的是 OAuth Token 或类似的机制的话，提供的是 认证 和 授权 ，认证是针对用户，授权是针对 App 。其目的是让某 App 有权利访问某用户的信息。这里的 Token 是唯一的。不可以转移到其它 App 上，也不可以转到其它用户上。Session 只提供一种简单的认证，即只要有此 SessionID ，即认为有此 User 的全部权利。是需要严格保密的，这个数据应该只保存在站方，不应该共享给其它网站或者第三方 App。所以简单来说：如果你的用户数据可能需要和第三方共享，或者允许第三方调用 API 接口，用 Token 。如果永远只是自己的网站，自己的 App，用什么就无所谓了。

11、Token 和 JWT 的区别

相同：
都是访问资源的令牌
都可以记录用户的信息
都是使服务端无状态化
都是只有验证成功后，客户端才能访问服务端上受保护的资源
区别：
Token：服务端验证客户端发送过来的 Token 时，还需要查询数据库获取用户信息，然后验证 Token 是否有效。
JWT：将 Token 和 Payload 加密后存储于客户端，服务端只需要使用密钥解密进行校验（校验也是 JWT 自己实现的）即可，不需要查询或者减少查询数据库，因为 JWT 自包含了用户信息和加密的数据。

12、使用 cookie 时需要考虑的问题

因为存储在客户端，容易被客户端篡改，使用前需要验证合法性
不要存储敏感数据，比如用户密码，账户余额
使用 httpOnly 在一定程度上提高安全性
尽量减少 cookie 的体积，能存储的数据量不能超过 4kb
设置正确的 domain 和 path，减少数据传输
cookie 无法跨域
一个浏览器针对一个网站最多存 20 个 Cookie，浏览器一般只允许存放 300 个 Cookie
移动端对 cookie 的支持不是很好，而 session 需要基于 cookie 实现，所以移动端常用的是 token

13、使用 session 时需要考虑的问题

将 session 存储在服务器里面，当用户同时在线量比较多时，这些 session 会占据较多的内

存，需要在服务端定期的去清理过期的 session
当网站采用集群部署的时候，会遇到多台 web 服务器之间如何做 session 共享的问题。因为 session 是由单个服务器创建的，但是处理用户请求的服务器不一定是那个创建 session 的服务器，那么该服务器就无法拿到之前已经放入到 session 中的登录凭证之类的信息了。
当多个应用要共享 session 时，除了以上问题，还会遇到跨域问题，因为不同的应用可能部署的主机不一样，需要在各个应用做好 cookie 跨域的处理。
sessionId 是存储在 cookie 中的，假如浏览器禁止 cookie 或不支持 cookie 怎么办？ 一般会把 sessionId 跟在 url 参数后面即重写 url，所以 session 不一定非得需要靠 cookie 实现
移动端对 cookie 的支持不是很好，而 session 需要基于 cookie 实现，所以移动端常用的是 token

14、使用 token 时需要考虑的问题

如果你认为用数据库来存储 token 会导致查询时间太长，可以选择放在内存当中。比如 redis 很适合你对 token 查询的需求。
token 完全由应用管理，所以它可以避开同源策略
token 可以避免 CSRF 攻击(因为不需要 cookie 了)
移动端对 cookie 的支持不是很好，而 session 需要基于 cookie 实现，所以移动端常用的是 token

15、使用 JWT 时需要考虑的问题

因为 JWT 并不依赖 Cookie 的，所以你可以使用任何域名提供你的 API 服务而不需要担心跨域资源共享问题（CORS）
JWT 默认是不加密，但也是可以加密的。生成原始 Token 以后，可以用密钥再加密一次。
JWT 不加密的情况下，不能将秘密数据写入 JWT。
JWT 不仅可以用于认证，也可以用于交换信息。有效使用 JWT，可以降低服务器查询数据库的次数。
JWT 最大的优势是服务器不再需要存储 Session，使得服务器认证鉴权业务可以方便扩展。但这也是 JWT 最大的缺点：由于服务器不需要存储 Session 状态，因此使用过程中无法废弃某个 Token 或者更改 Token 的权限。也就是说一旦 JWT 签发了，到期之前就会始终有效，除非服务器部署额外的逻辑。
JWT 本身包含了认证信息，一旦泄露，任何人都可以获得该令牌的所有权限。为了减少盗用，JWT 的有效期应该设置得比较短。对于一些比较重要的权限，使用时应该再次对用户进行认证。
JWT 适合一次性的命令认证，颁发一个有效期极短的 JWT，即使暴露了危险也很小，由于每次操作都会生成新的 JWT，因此也没必要保存 JWT，真正实现无状态。
为了减少盗用，JWT 不应该使用 HTTP 协议明码传输，要使用 HTTPS 协议传输。

16、哈希算法及使用加密算法时需要考虑的问题

哈希算法(Hash Algorithm)又称散列算法、散列函数、哈希函数，是一种从任何一种数据中创建小的数字“指纹”的方法。哈希算法将数据重新打乱混合，重新创建一个哈希值。
哈希算法主要用来保障数据真实性(即完整性)，即发信人将原始消息和哈希值一起发送，收

信人通过相同的哈希函数来校验原始数据是否真实。

使用加密算法时需要考虑的问题：
绝不要以明文存储密码
永远使用 哈希算法 来处理密码，绝不要使用 Base64 或其他编码方式来存储密码，这和以明文存储密码是一样的，使用哈希，而不要使用编码。编码以及加密，都是双向的过程，而密码是保密的，应该只被它的所有者知道， 这个过程必须是单向的。哈希正是用于做这个的，从来没有解哈希这种说法， 但是编码就存在解码，加密就存在解密。
绝不要使用弱哈希或已被破解的哈希算法，像 MD5 或 SHA1 ，只使用强密码哈希算法。
绝不要以明文形式显示或发送密码，即使是对密码的所有者也应该这样。如果你需要 “忘记密码” 的功能，可以随机生成一个新的 一次性的（这点很重要）密码，然后把这个密码发送给用户。

17、哈希算法通常有以下几个特点：
2 的 128 次方为 340282366920938463463374607431768211456，也就是 10 的 39 次方级别
2 的 160 次方为 1.461501637330902918203684832716 3e+48，也就是 10 的 48 次方级别
2 的 256 次方为 1.157920892373161954235709850086 9 × 10 的 77 次方，也就是 10 的 77 次方
正像快速：原始数据可以快速计算出哈希值
逆向困难：通过哈希值基本不可能推导出原始数据
输入敏感：原始数据只要有一点变动，得到的哈希值差别很大
冲突避免：很难找到不同的原始数据得到相同的哈希值，宇宙中原子数大约在 10 的 60 次方到 80 次方之间，所以 2 的 256 次方有足够的空间容纳所有的可能，算法好的情况下冲突碰撞的概率很低：

九十六、AD 压测命令

https://www.apachehaus.com/cgi-bin/download.plx
最新版本不支持 https，指支持 http 一个协议，旧版支持 https，不支持参数化

压测工具	Apache Benchmark(ab)	LoadRunner	Jmeter
学习成本	低	高	高
安装部署成本	低	高	高
是否免费	是	否	是
是否支持多协议	否	是	是
图形化结果展示	否	是	是
是否支持TPS模式	否	否	否
场景设计	否	是	是
场景录制	否	是	是
生态环境	弱	弱	弱
监控指标是否完备	否	否	否
是否原生支持流量地域定制	否	否	否

```
-V          Print version number and exit 显示ab工具的版本号
-h          Display usage information (this message)显示ab可选参数列表
-n requests    Number of requests to perform 参数用来设置模拟请求的总次数
-c concurrency  Number of multiple requests to make at a time参数用来设置模拟请求的并发数
-t timelimit   Seconds to max. to spend on benchmarking参数用来设置模拟请求的时间，单位是秒，默认是50000秒
               This implies -n 50000
-w          Print out results in HTML tables 采用HTML表格打印结果
-m method      Method name 方法名称
-s timeout     Seconds to max. wait for each response 请求超时时间，单位是秒，默认30秒
               Default is 30 seconds
-T content-type Content-type header to use for POST/PUT data, eg. 请求content-type
          'application/x-www-form-urlencoded'
          Default is 'text/plain'
-p postfile    File containing data to POST. Remember also to set -T POST时的数据文件
               参数文档内容使用 "key=value&key=value key=value..." 方式进行拼接
```

ab –V 查看版本
ab –n100 http://biudu.com -n 指总共的请求，模拟一个用户访问一百次
ab –c10 –n50 http://biudu.com c 模拟 10 个用户，n 访问 50 次，每个用户访问 5 次
ab –c10 –n50 –t100 http://biudu.com t 模拟压测 100 秒
ab –c10 –n50 –w http://biudu.com >>d:\a.html w 转换成网页形式报告
ab –c10 –n50 –p d:\1.txt http://biudu.com post 形式传参，txt 文件里是 name=1&=2 格式
ab –c10 –n50 –p d:\1.txt –T application/x-www-form-urlencoded http://biudu.com T 指类型

九十七、怎么区分 App 页面是 H5 还是原生？

原生是 Native APP
H5 就是 Web App

1.看断网的情况
把手机的网络断掉。然后点开页面。然后可以正常显示的东西就是原生写的。
显示 404 或则错误页面的是 html 页面。

2.看布局边界
打开开发者选项中的显示布局边界
页面元素很多、布局是一整块的就是 h5 的；
布局密密麻麻的是原生控件。





3.看复制文章的提示，需要你通过对比才能得出结果。
比如是文章资讯页面可以长按页面试试，如果出现文字选择、粘贴功能的是 H5 页面，否则是 native 原生的页面。
有些原生 APP 开放了复制粘贴功能或者关闭了。而 H5 的 css 屏蔽了复制选择功能等等情况。需要通过对目标测试 APP 进行对比才可知。
这个在支付宝 APP、蚂蚁聚宝都是可以判断的。

4.看加载的方式
如果在打开新页面导航栏下面有一条加载的线的话，这个页面就是 H5 页面，如果没有就是原生的。 微信里面打开我们的 H5 页面常见的有个绿色的 加载线条。如下图红框里面所示：



5、看 app 顶部 导航栏是否会有关闭的操作
如果 APP 顶部导航栏当中出现了关闭按钮或者有关闭的图标，那么当前的页面肯定的 H5，原生的不会出现（除非设计开发者故意弄的）
美团的、大众点评的 APp、微信 APP 当加载 h5 过多的时候，左上角会出现关闭 2 字。

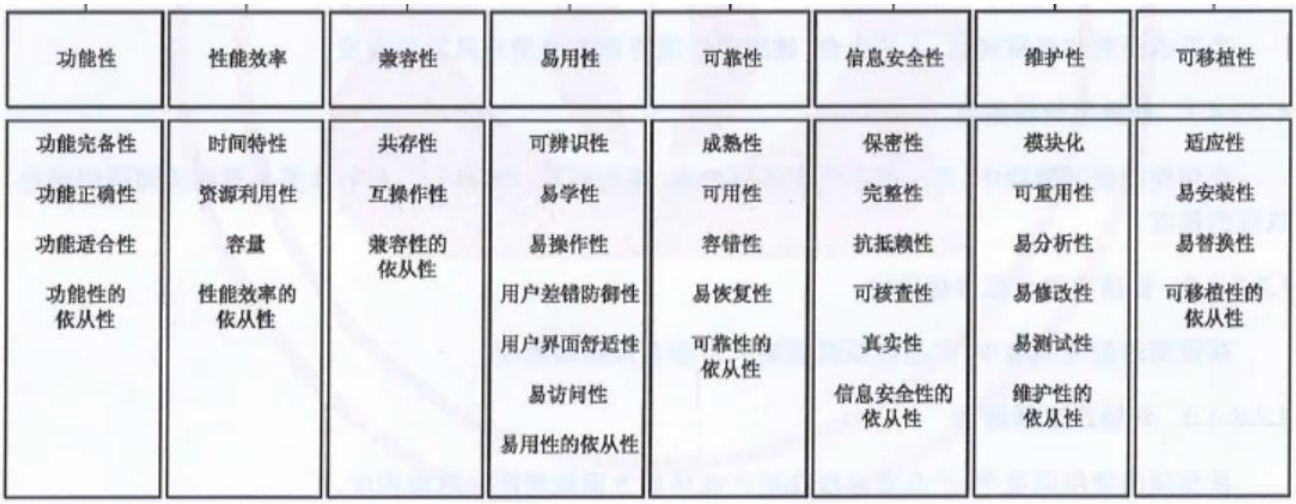
6、判断页面 下拉刷新的时候（前提是要有下拉刷新的功能）
如果界面没有明显刷新现象的是原生的，如果有明显刷新现象（比如闪一下）的是 H5 页面（ios 和 android）。
比如淘宝的众筹页面。

7、下拉页面的时候显示网址提供方的一定是 H5。



九十八、用例八大质量属性

质量属性：功能性、性能效率、兼容性、易用性、可靠性、信息安全性、维护性、可移植性



九十九、Ubuntu 和 Centos 区别

1、centos 中普通用户没有 sudo 权限，需在 sudoers 文件中添加用户的权限。
centos 中新建的非 root 用户是没有 sudo 的权限的，如果需要使用 sudo 权限必须在 /etc/sudoers 中加入账户和权限，所以切换到 root 账号的时候只需要输入：su,加入 root 账号的密码即可。
2、ubuntu 中普通用户，直接使用 sudo+命令行的方式即可。
在 Ubuntu 中，一般使用 sudo+命令，如果是第一次使用会提示输入当前用户的密码（而不是 root 的密码），如果要切换到 root 用户，需要输入 sudo passwd 设置 root 用户密码。然后再 su root。
3、ubuntu 与 centos 安装软件包命令的格式不一样。
在线安装软件中，centos 使用的是 yum 命令，而 ubuntu 中使用的是 apt-get 命令。除此之外 yum 中还有一个从软件源中搜索某个软件的方法：yum search +软件名
4、centos 是支持 rpm 包的，ubuntu 不支持 rpm 包。RPM 的全名是 Red Hat Package Manager，本意是 Red Hat 软件包管理，centos 是来自于 redhat，所以 centos 支持 rpm 格式的安装，而 ubuntu 显然是不支持的。

ubuntu 安装：
网络安装：sudo apt-get install 软件名
安装 deb 包：sudo dpkg -i 软件包名.deb

ubuntu 卸载：
apt 卸载软件：sudo apt-get remove 软件名
dpkg 方式卸载：sudo dpkg -r 软件名

centos：
安装软件包命令：yum install ~
删除软件包命令：yum remove ~

系统升级
CentOS：升级不频繁，省事省心。
Ubuntu：升级比较频繁，经常需要更新系统。

系统稳定性
CentOS：目前 CentOS 系统更加安全稳定。
Ubuntu：升级包可能会造成系统的不稳定和不安全。

Centos 是基于 Redhat 开源构建的，服务器系统用的最多，Ubuntu 是程序员开发环境，桌面环境用的最多得。
对于 ubuntu 而言，就是 linux 操作系统的具体，而 linux 对于 ubuntu 来说就是他的抽象；在 linux 操作系统中，因为应用程序和管理策略的不同，有多个版本，例如：
ubuntu，fedora，redhat，centos 等。

非常多的商业公司部署在生产环境上的服务器都是使用 CentOS 系统，Centos 是从 Redhat 源代码编译重新发布版，Centos 去除很多与服务器功能无关的应用，系统简单但非常稳定，命令行操作可以方便管理系统和应用，并且有帮助文档和社区的支持。

一百、正则匹配

import re　　#正则

\w　匹配字母数字及下划线
\W　匹配非字母数字及下划线
\s　匹配任意空白字符，等价于 [\t\n\r\f].
\S　匹配任意非空字符
\d　匹配任意数字，等价于 [0-9].
\D　匹配任意非数字
\A　匹配字符串开始
\Z　匹配字符串结束，如果是存在换行，只匹配到换行前的结束字符串。
\z　匹配字符串结束
\G　匹配最后匹配完成的位置。

* 匹配前一个字符 0 或无限次　abc*　ab　abcccc
+ 匹配前一个字符 1 次或无限次 abc+　abc　abcccc
？匹配前一个字符 0 次或 1 次　abc?　ab　abc

```
#例子
d='www.baidu.com?a=1&b=2&c=3'
#表示匹配一个不是"？"的字符任意次
k=re.compile('\w[^?]*')         #['www.baidu.com', 'a=1&b=2&c=3']
kk=re.compile('\w=\d*')         #['a=1', 'b=2', 'c=3']
#k.search(d)         #只匹配到所有字符串里的第一个
#k.match(d)             #只能从头匹配，只匹配一个
k.findall(d)
kk.findall(d)
print (k.findall(d),'----')         #['www.baidu.com', 'a=1&b=2&c=3']
print (kk.findall(d),'///////////////')       #['a=1', 'b=2', 'c=3']

aa='abefffa1cgbeda'
bb=re.compile('a[^f]*')       # ['abe', 'a1cgbeda']
bb.findall(aa)
print(bb.findall(aa))           # ['abe', 'a1cgbeda']


cc=re.compile('bef+')           #+号匹配前一个字符 1 次或无限次
cc.findall(aa)
print(cc.findall(aa))           #['befff']

dd=re.compile('bef*')           #*号匹配前一个字符 0 或无限次
dd.findall(aa)
print(dd.findall(aa))           #['befff', 'be']


print('aaa'+'\n'+'bbb')
#print(f"1"+'\n'+f"1")
#print(f'1\n'+f'1')
print(str(1)+'\n'+str(1))

ab='<a  href="http://baidu.com/s?id=14"mon="col=1&amp;a=2&amp;pn=1"  target="_blank"> 速看！河南、江西等..</a>'
g=re.compile('href="(.+?)".*>(.+?)<')
print ("正则")
print (g.findall(ab))     # [('http://baidu.com/s?id=14', '速看！河南、江西等..')]


#k=re.compile('<"_blank">.*</a>')
#k=re.compile('href="(.+?)"')
#k=re.compile('>(.+?)<')
g=re.compile('href="(.+?)".*>(.+?)<')
print ("正则")
```

print (g.findall(ab)) # [('http://baidu.com/s?id=14', '速看！ 河南、江西等..')]

Python 使用正则表达式匹配 URL 网址

```
str = "http://www.baidu.com/"
regular = re.compile(r'[a-zA-Z]+://[^\s]*[.com|.cn]')
re.findall(regular, str)
```